

Advanced

BACKEND

+

AI

PART 2

Full Course

**AI POWERED
ADVANCED BACKEND COURSE**



PART2

1

2

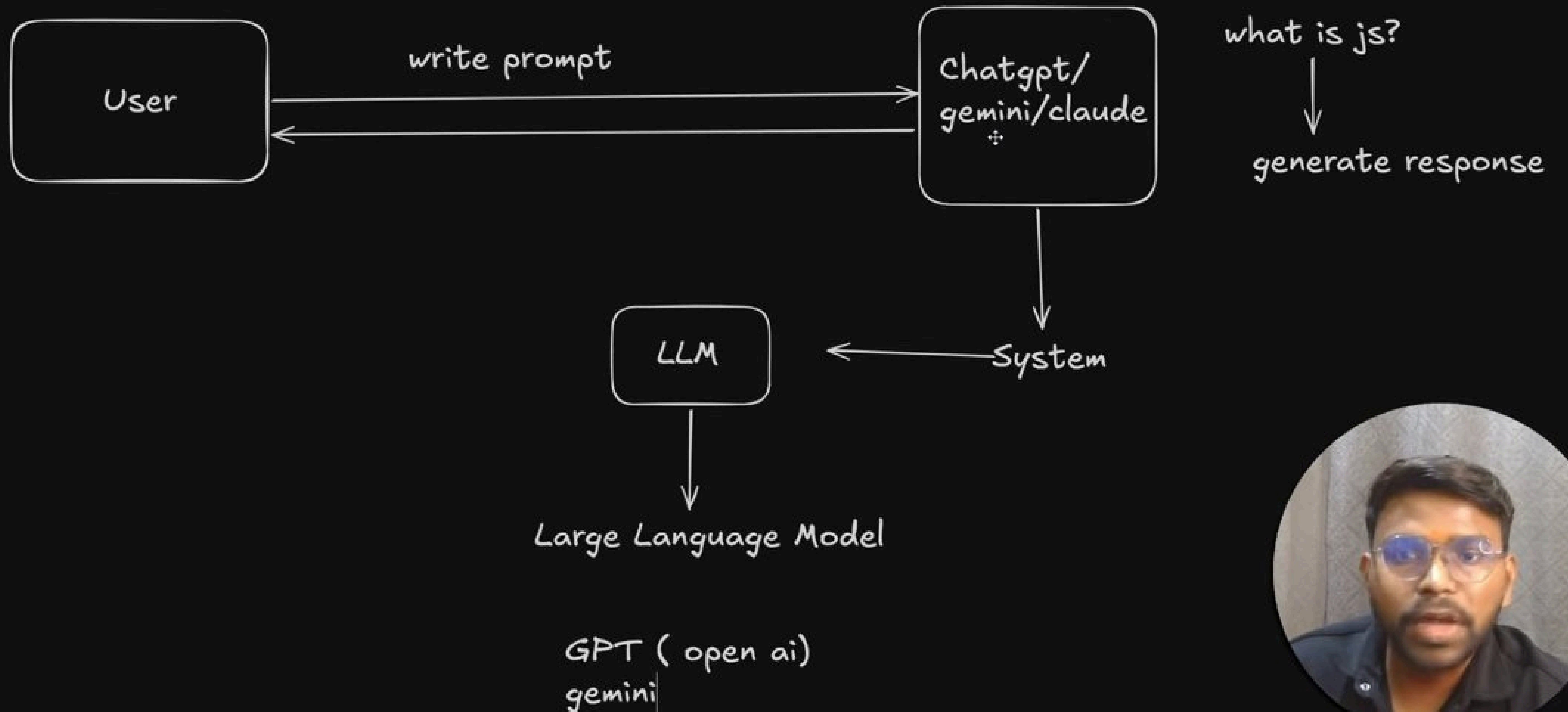


LEVEL 4

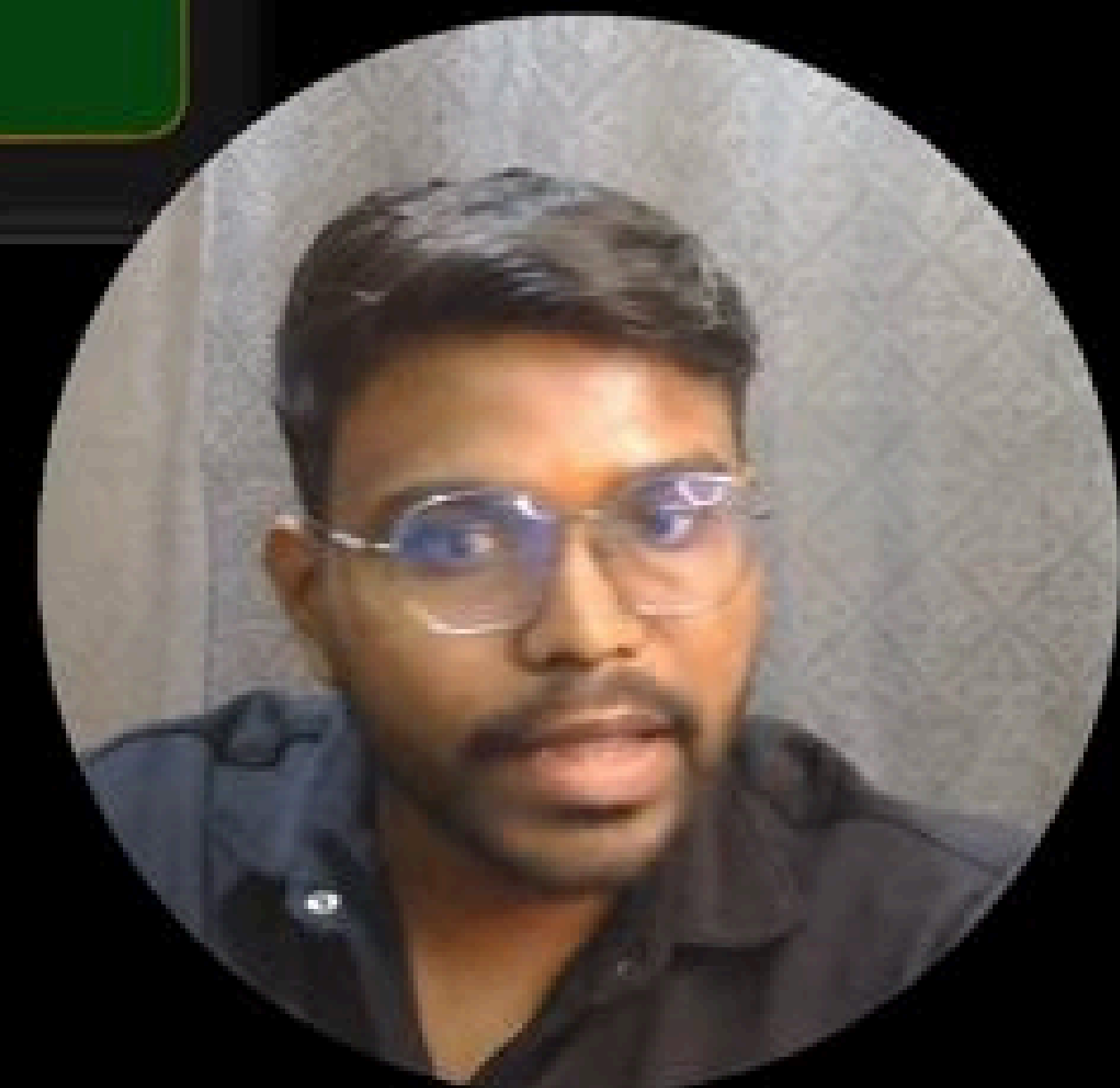
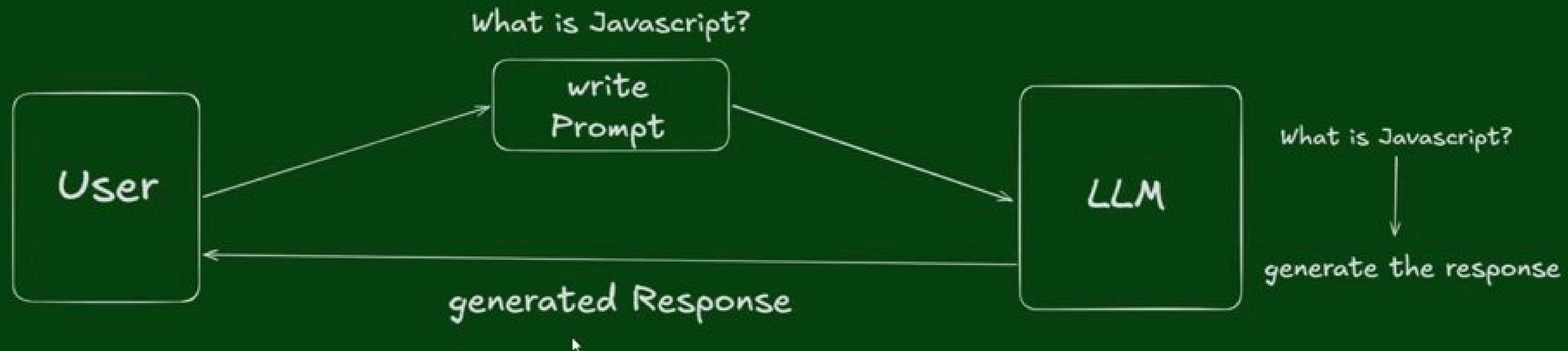
AI Full Course



AI Full Course



simple AI Response Generation



What is an LLM?

LLM stands for Large Language Model.

It is an AI model trained on massive amounts of text data to understand and generate human language.

Examples:

- OpenAI GPT models
- Anthropic Claude
- Google Gemini
- Meta Llama



 package.json X

```
{
  "name": "level4",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}
```

Ln 1, Col 1 Sp

Ln 1, Col 1 Spaces: 4 UTF-8 CRLF { } JavaScript  Go Live 





▼ ADVA...    

```
>  level1
```

```
>  level2
```

```
> █ level3
```

level4

```
> node_modules
```

JS index.js

 package-lock.json package.json

> TIMELINE

> OUTLINE

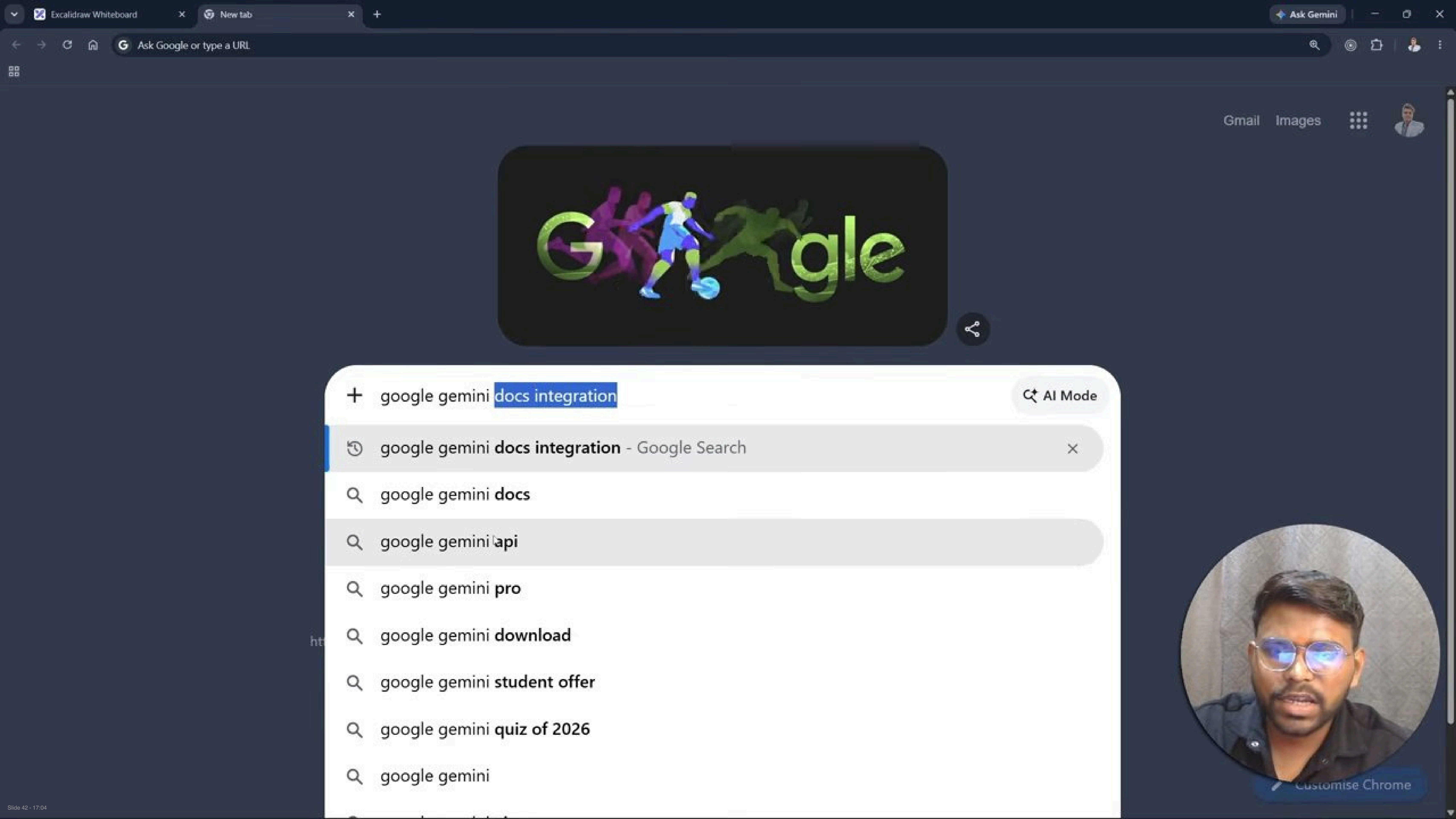
main   0  0

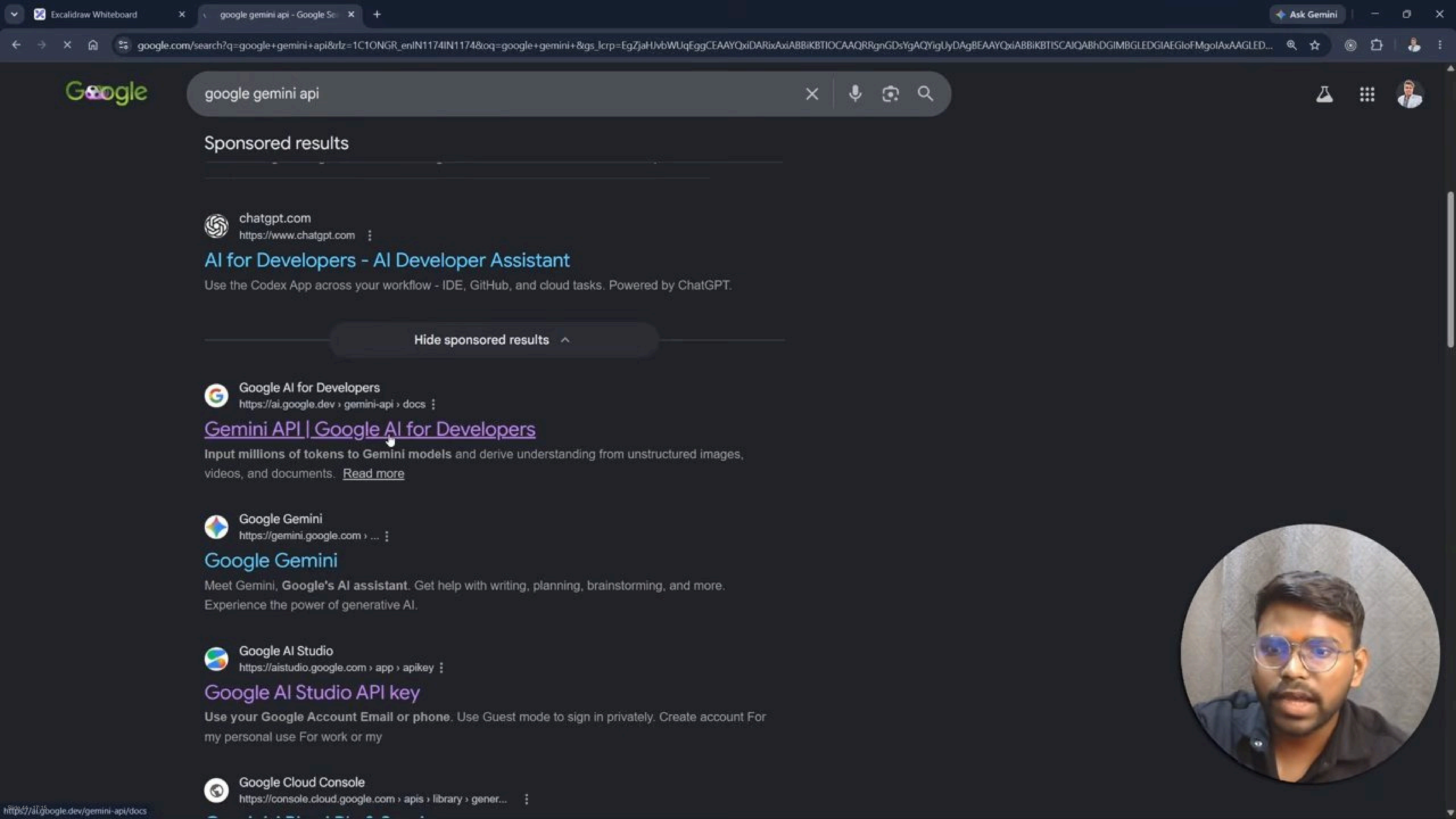
 package.json package-lock.json

JS index.js

```
1  import express from "express"
2
3  const app=express()
4  const port=5000
5
6
7  app.get("/",(req,res)=>{
8    return res.json({message:"hello from level4"})
9  })
10
11
12 app.listen(port,()=>{
13   console.log("server started")
14 })
```







Get started

Overview

Quickstart

API keys

Libraries

Pricing

Interactions API

Coding agent setup

Models

All models

Current: Gemini 3.5

Nano Banana

Ve

Lyria 3

Lyria RealTime

Imagen

Text-to-speech

Live

Gemini Deep Research is now available in preview with collaborative planning, visualization, MCP support, and more.

Home > Gemini API > generateContent API > Docs

Gemini API

The fastest path from prompt to production with Gemini, Veo, Nano Banana, and more.

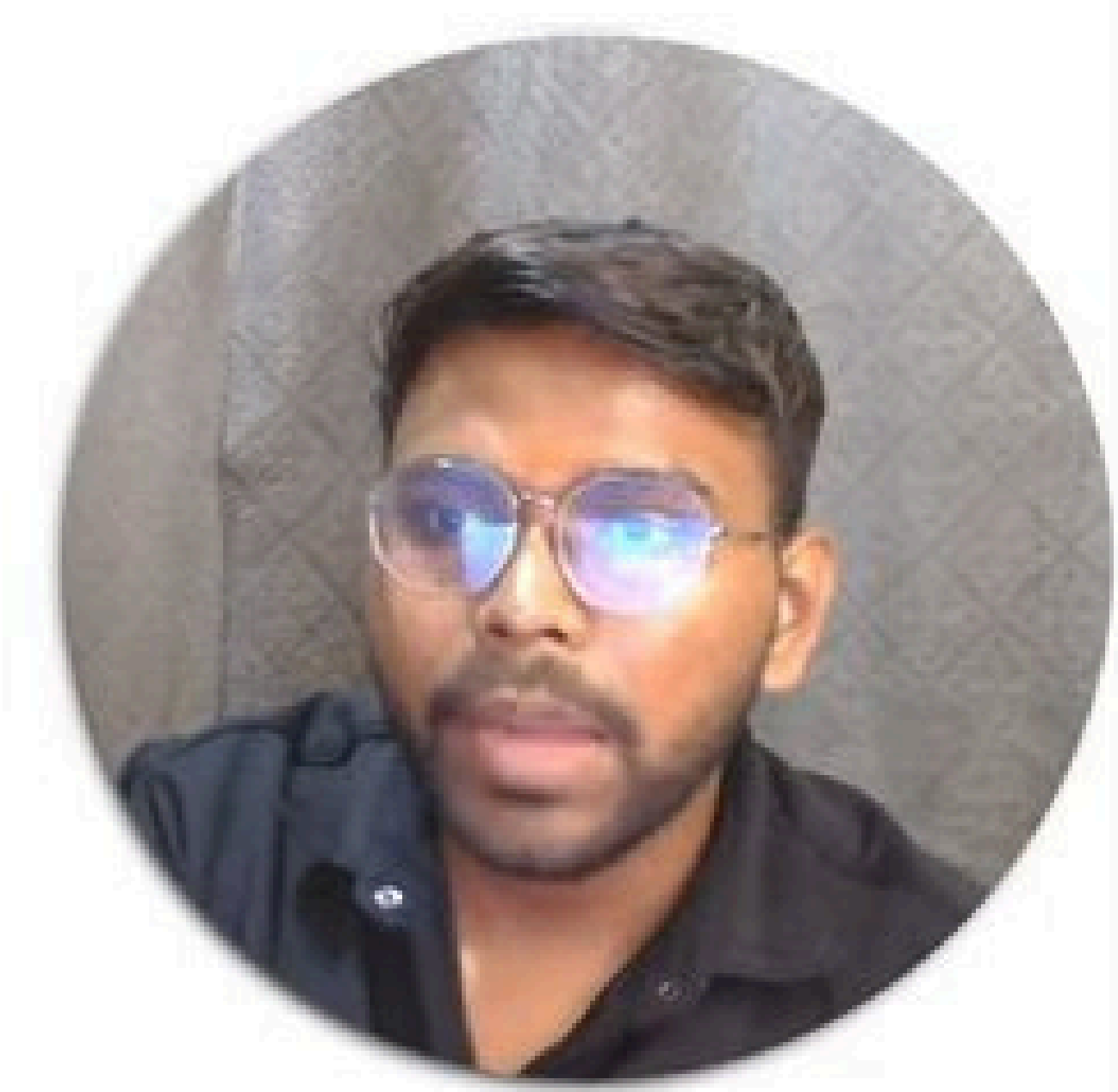
Python JavaScript Go Java C# REST

```
import { GoogleGenAI } from "@google/genai";

const ai = new GoogleGenAI({});

async function main() {
  const response = await ai.models.generateContent({
    model: "gemini-3.5-flash",
    contents: "Explain how AI works in a few words",
  });
  console.log(response.text);
}

await main();
```



Excalidraw Whiteboard

Gemini API quickstart - general

ai.google.dev/gemini-api/docs/quickstart

Gemini API

Docs

API reference

Get started

Overview

Quickstart

API keys

Libraries

Pricing

Interactions API

Coding agent setup

Models

All models

Current: Gemini 3.5

Nano Banana

Veo

Lyria 3

Lyria RealTime

Imagen

Text-to-speech

Live

Search

English

Get API key

Cookbook

Community

Sign in

Install the Google GenAI SDK

Python

JavaScript

Using Node.js v18+, install the Google Gen AI SDK for TypeScript and JavaScript using the following npm command:

npm install @google/genai

Generate text

Use the models.generate_content method to generate a text response.

Python

JavaScript

REST

import { GoogleGenAI } from "@google/genai";

const ai = new GoogleGenAI({});

export function main() {

On this page

Before you begin

Install the Google GenAI SDK

Generate text

Stream responses

Multi-turn conversations

Use tools

Call custom functions

What's next

Slide 50 - 18/19

Excalidraw Whiteboard

Gemini API quickstart - general

ai.google.dev/gemini-api/docs/quickstart#rest

Gemini API

DocsAPI reference

Get started

Overview

Quickstart

API keys

Libraries

Pricing

Interactions API

Coding agent setup

Models

All models

Current: Gemini 3.5

Nano Banana

Veo

Lyria 3

Lyria RealTime

Imagen

Text-to-speech

Live

Search

English

Get API key

Cookbook

Community

Sign in

Use the `models.generate_content` method to generate a text response.

PythonJavaScriptREST

```
curl "https://generativelanguage.googleapis.com/v1beta/models/gemini-3
-H "x-goog-api-key: $GEMINI_API_KEY" \
-H 'Content-Type: application/json' \
-X POST \
-d '{
  "contents": [
    {
      "parts": [
        {
          "text": "Explain how AI works in a few words"
        }
      ]
    }
  ]
}'
```

On this page

Before you begin

Install the Google GenAI SDK

Generate text

Stream responses

Multi-turn conversations

Use tools

Call custom functions

What's next

Stream responses

By default, the model returns a response only after the entire generation process is complete. For a faster, more interactive experience, you can stream the response

Ask Gemini

Slide 51 - 18:32

Excalidraw Whiteboard

Gemini API quickstart - general

+

ai.google.dev/gemini-api/docs/quickstart#rest

Gemini API

Docs

API reference

Get started

Overview

Quickstart

API keys

Libraries

Pricing

Interactions API

Coding agent setup

Models

All models

Current: Gemini 3.5

Nano Banana

Veo

Lyria 3

Lyria RealTime

Imagen

Text-to-speech

Live

Search

/

English

Get API key

Cookbook

Community

Sign in

Python

JavaScript

Using [Node.js v18+](#), install the Google Gen AI SDK for TypeScript and JavaScript using the following [npm command](#):

```
npm install @google/genai
```

Generate text

Use the `models.generate_content` method to [generate a text response](#).

Python

JavaScript

REST

```
curl "https://generativelanguage.googleapis.com/v1beta/models/gemini-3" \
-H "x-goog-api-key: $GEMINI_API_KEY" \
-H 'Content-Type: application/json' \
-X POST \
-d '{
  "contents": [
    {
      "parts": [
```

On this page

Before you begin

Install the Google GenAI SDK

Generate text

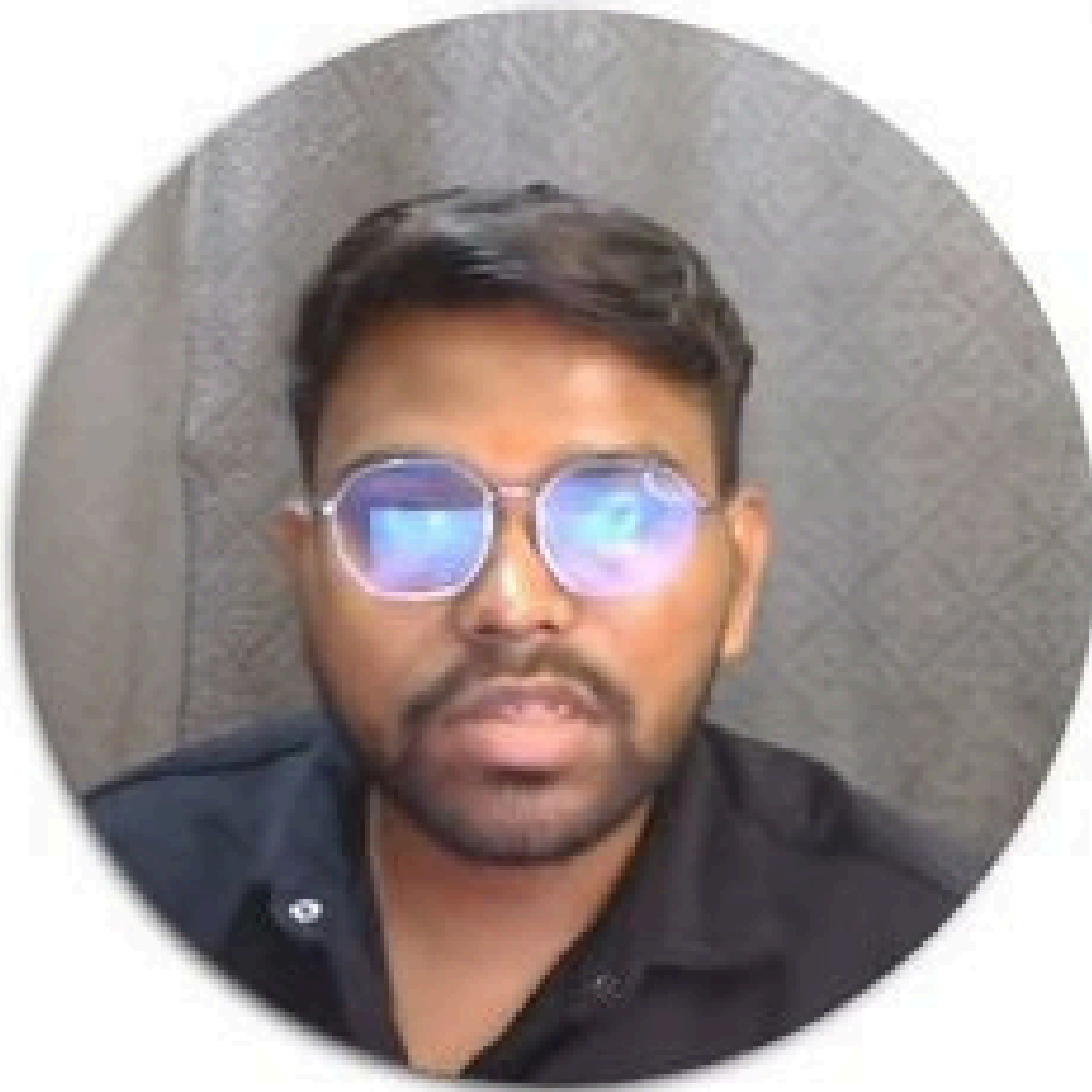
Stream responses

Multi-turn conversations

Use tools

Call custom functions

What's next



Slide 52 - 18:43



> **SEARCH**

▼ ADVANCED BACKEND CO...

```
>  level1
```

```
>  level2
```

```
> █ level3
```

level4

```
> node_modules
```

JS index.js

 package-lock.json

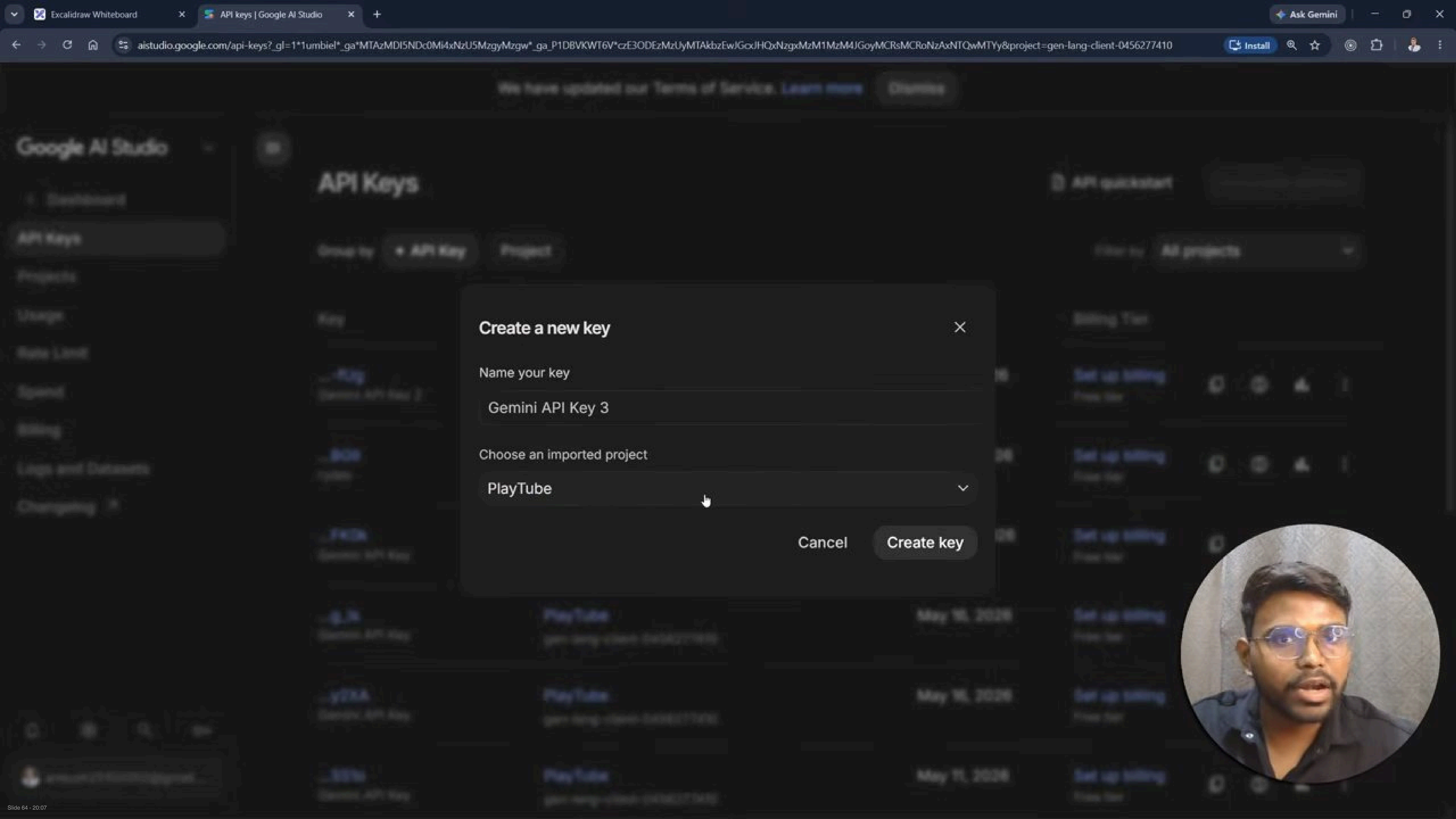
 package.json package.json X

package-lock.json X

index.js

```
3   "version": "1.0.0",
4   "description": "",
5   "main": "index.js",
6   "scripts": {
7     "dev": "nodemon index.js"
8   },
9   "keywords": [],
10  "author": "",
11  "license": "ISC",
12  "type": "module",
13  "dependencies": {
14    "@google/genai": "^2.8.0",
15    "express": "^5.2.1",
16    "nodemon": "^3.1.14"
17  }
18 }
19
```





FileEditSelectionView

Advanced Backend Course

package.jsonpackage-lock.jsonindex.js.env

1GEMINI_API_KEY="AQ.Ab8RN6K83SorGP-PFaSePR6_3paD5Nd0YqG1FfnmBxjUY16Tew"

EXPLORE

SEARCH

ADVANCED BACKEND CO...

level1

level2

level3

level4

node_modules

.env

index.js

package-lock.json

package.json

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

node - level4

PS D:\desktop\Advanced Backend Course\level4> npm install @google/genai

found 0 vulnerabilities

PS D:\desktop\Advanced Backend Course\level4> npm run dev

> level4@1.0.0 dev

> nodemon index.js

[nodemon] 3.1.14

[nodemon] to restart at any time, enter `rs`

[nodemon] watching path(s): *.*

[nodemon] watching extensions: js,mjs,cjs,json

[nodemon] starting `node index.js`

server started

main

Ln 1, Col 1 (14 selected)

Spaces: 4

UTF-8

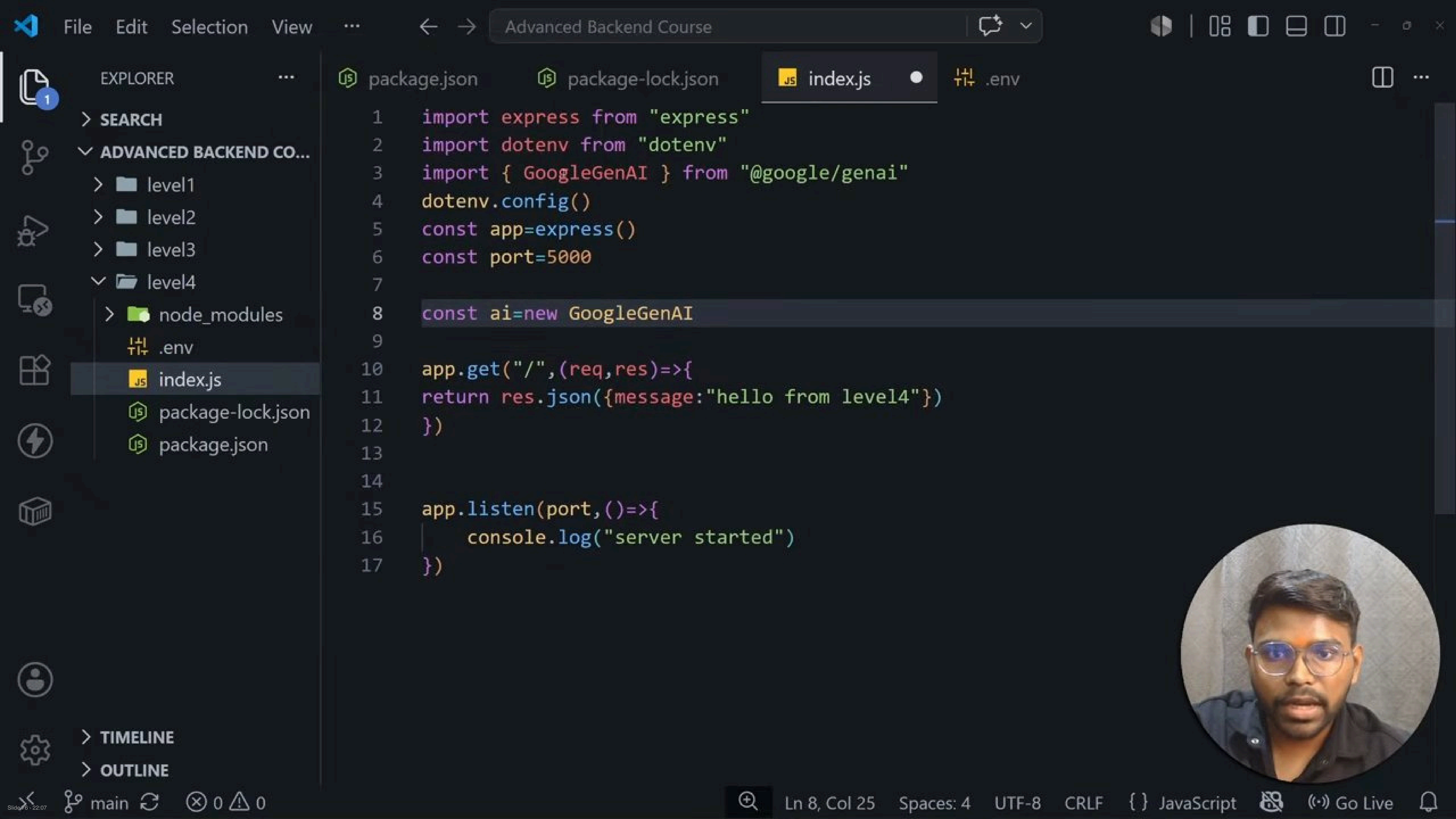
CRLF

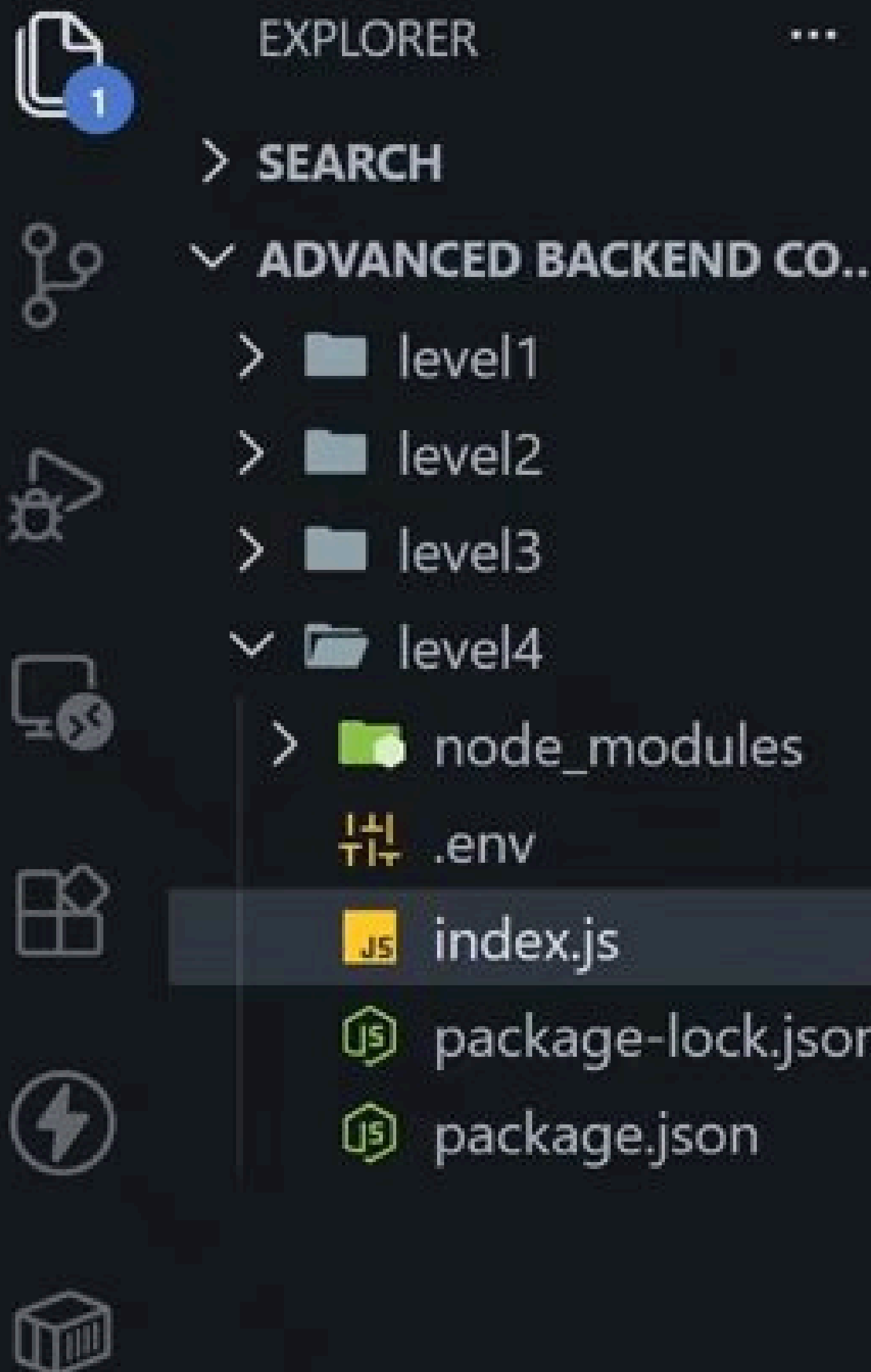
{ }

Dotenv

Go Live







package.json package-lock.json index.js .env

```
1 import express from "express"
2 import dotenv from "dotenv"
3 import { GoogleGenAI } from "@google/genai"
4 dotenv.config()
5 const app=express()
6 const port=5000
7
8 const ai=new GoogleGenAI()
```

The Google GenAI SDK.

@remarks

Provides access to the GenAI features through either the [Gemini API](#) or the [Vertex AI API](#).

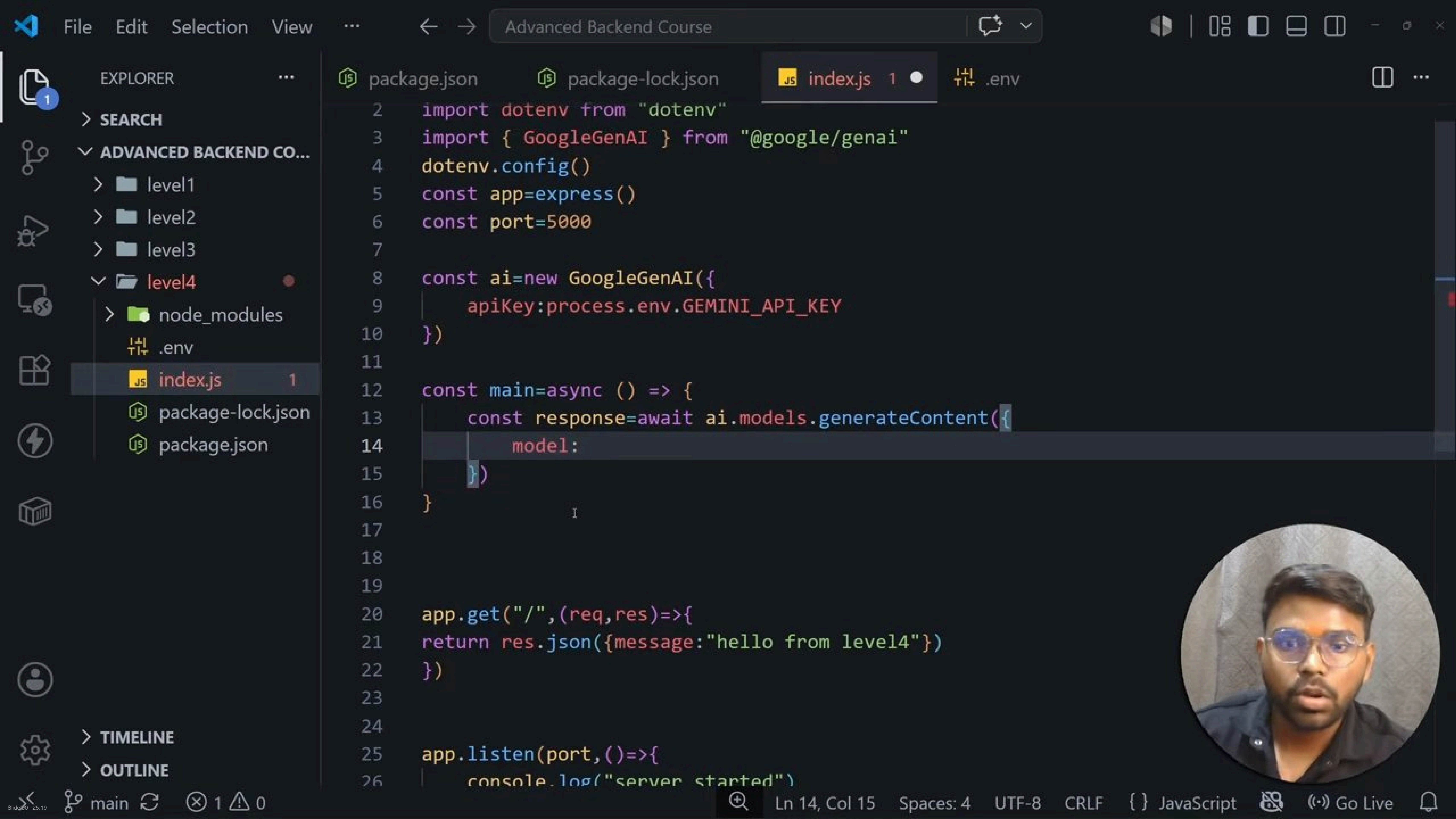
The `GoogleGenAIOptions.vertexai` value determines which of the API services to use.

When using the Gemini API, a `GoogleGenAIOptions.apiKey` must also be set, when using Vertex AI, `GoogleGenAIOptions.project` and `GoogleGenAIOptions.location` must also be set.

@example

Initializing the SDK for using the Gemini API:





```
File Edit Selection View ... < > Advanced Backend Course [chat icon] [dropdown]  
[1 icon] EXPLORER ...  
> SEARCH  
v ADVANCED BACKEND CO...  
  > level1  
  > level2  
  > level3  
  v level4 [red dot]  
    > node_modules  
    .env  
    index.js 1  
    package-lock.json  
    package.json  
[lightning bolt icon]  
[cube icon]  
[person icon]  
> TIMELINE  
> OUTLINE  
main [refresh icon] [close icon] 1 [warning icon] 0  
Slide 25:19  
package.json package-lock.json index.js 1 [diff icon] .env  
2 import dotenv from "dotenv"  
3 import { GoogleGenAI } from "@google/genai"  
4 dotenv.config()  
5 const app=express()  
6 const port=5000  
7  
8 const ai=new GoogleGenAI({  
9   apiKey:process.env.GEMINI_API_KEY  
10 })  
11  
12 const main=async () => {  
13   const response=await ai.models.generateContent({  
14     model:  
15   })  
16 }  
17  
18  
19  
20 app.get("/",(req,res)=>{  
21   return res.json({message:"hello from level4"})  
22 })  
23  
24  
25 app.listen(port,()=>{  
26   console.log("server started")  
})  
Ln 14, Col 15 Spaces: 4 UTF-8 CRLF { } JavaScript [no error icon] [Go Live icon] [bell icon]
```

Excalidraw Whiteboard

Gemini API quickstart - general

ai.google.dev/gemini-api/docs/quickstart#javascript

Gemini API

DocsAPI reference

Get started

Overview

Quickstart

API keys

Libraries

Pricing

Interactions API

Coding agent setup

Models

All models

Current: Gemini 3.5

Nano Banana

Ve

Lyria 3

Lyria RealTime

Imagen

Text-to-speech

Live

Search

English

Get API key

Cookbook

Community

Sign in

PythonJavaScriptREST

```
import { GoogleGenAI } from "@google/genai";

const ai = new GoogleGenAI({});

async function main() {
  const response = await ai.models.generateContent({
    model: "gemini-3.5-flash",
    contents: "Explain how AI works in a few words",
  });

  console.log(response.text);
}

main();
```

Light code theme

On this page

Before you begin

Install the Google GenAI SDK

Generate text

Stream responses

Multi-turn conversations

Use tools

Call custom functions

What's next

Stream responses

By default, the model returns a response only after the entire generation process is complete. For a faster, more interactive experience, you can stream the response chunks as they are generated.

PythonJavaScriptREST



Slide 92 - 25:35

Excalidraw Whiteboard

Gemini API quickstart - general

ai.google.dev/gemini-api/docs/quickstart#javascript

Gemini API

Search

English

Get API key

Cookbook

Community

Sign in

Docs

API reference

Get started

Overview

Quickstart

API keys

Libraries

Pricing

Interactions API

Coding agent setup

Models

All models

Current: Gemini 3.5

Nano Banana

Ve

Lyria 3

Lyria RealTime

Imagen

Text-to-speech

Live

Python

JavaScript

REST

```
import { GoogleGenAI } from "@google/genai";

const ai = new GoogleGenAI({});

async function main() {
  const response = await ai.models.generateContent({
    model: "gemini-3.5-flash",
    contents: "Explain how AI works in a few words",
  });

  console.log(response.text);
}

main();
```

On this page

Before you begin

Install the Google GenAI SDK

Generate text

Stream responses

Multi-turn conversations

Use tools

Call custom functions

What's next

Stream responses

By default, the model returns a response only after the entire generation process is complete. For a faster, more interactive experience, you can stream the response chunks as they are generated.

Python

JavaScript

REST



Slide 93 - 25:44

Excalidraw Whiteboard

Gemini API quickstart - general

ai.google.dev/gemini-api/docs/quickstart#javascript

Gemini API

Search

English

Get API key

Cookbook

Community

Sign in

Docs

API reference

Get started

Overview

Quickstart

API keys

Libraries

Pricing

Interactions API

Coding agent setup

Models

All models

Current: Gemini 3.5

Nano Banana

Ve

Lyria 3

Lyria RealTime

Imagen

Text-to-speech

Live

Python

JavaScript

REST

```
import { GoogleGenAI } from "@google/genai";

const ai = new GoogleGenAI({});

async function main() {
  const response = await ai.models.generateContent({
    model: "gemini-3.5-flash",
    contents: "Explain how AI works in a few words",
  });

  console.log(response.text);
}

main();
```

On this page

Before you begin

Install the Google GenAI SDK

Generate text

Stream responses

Multi-turn conversations

Use tools

Call custom functions

What's next

Stream responses

By default, the model returns a response only after the entire generation process is complete. For a faster, more interactive experience, you can stream the response chunks as they are generated.

Python

JavaScript

REST



Slide 96 - 26:11

FileEditSelectionView

Advanced Backend Course

VS Code icons

EXPLORER

SEARCH

ADVANCED BACKEND CO...

level1

level2

level3

package.json

package-lock.json

index.js

.env

const main: () => Promise<void>

.env

index.js

package-lock.json

package.json

main

MaskReferenceImage @google/genai

MediaDeviceInfo @google/genai

MatchOperation @google/genai

MediaEncryptedEvent

MutationObserver

MutationRecord

MediaSession

MIDIMessageEvent

MediaStreamAudioDestinationNode

MediaSourceHandle

MediaStreamAudioSourceNode

2 import dotenv from "dotenv"

3 import { GoogleGenAI } from "@google/genai"

4 dotenv.config()

5 const app=express()

6 const port=5000

7

10

11

12 cons

13

14

15

16

17

18 }

19

20 main

21

22

23

24

25 app.get("/",(req,res)=>{

26 return res.json({message:"hello from level4"})

main

Ln 20, Col 5

Spaces: 4

UTF-8

CRLF

{ } JavaScript

Go Live



EXPLORER

> SEARCH

▼ ADVANCED BACKEND CO...

```
>  level1
```

```
>  level2
```

```
>  level3
```

level4

```
> node_modules
```

 .env index.js package-lock.json package.json

> TIMELINE

> OUTLINE

main   0  0

package.json package-lock.json index.js .env

```
2 import dotenv from "dotenv"
3 import { GoogleGenAI } from "@google/genai"
4 dotenv.config()
5 const app=express()
6 const port=5000
7
8 const ai=new GoogleGenAI({
9   apiKey:process.env.GEMINI_API_KEY
10 })
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

 node - level4     ...  

```
PS D:\desktop\Advanced Backend Course\level4> npm run dev
```

```
thoughtsTokenCount: 184,  
serviceTier: 'standard'
```

7

1

```
[nodemon] restarting due to changes...
```

```
[nodemon] starting `node index.js`
```

```
◇ injected env (1) from .env // tip: % enable debugging { debug: true }
```

```
server started
```

```
[nodemon] restarting due to changes...
```

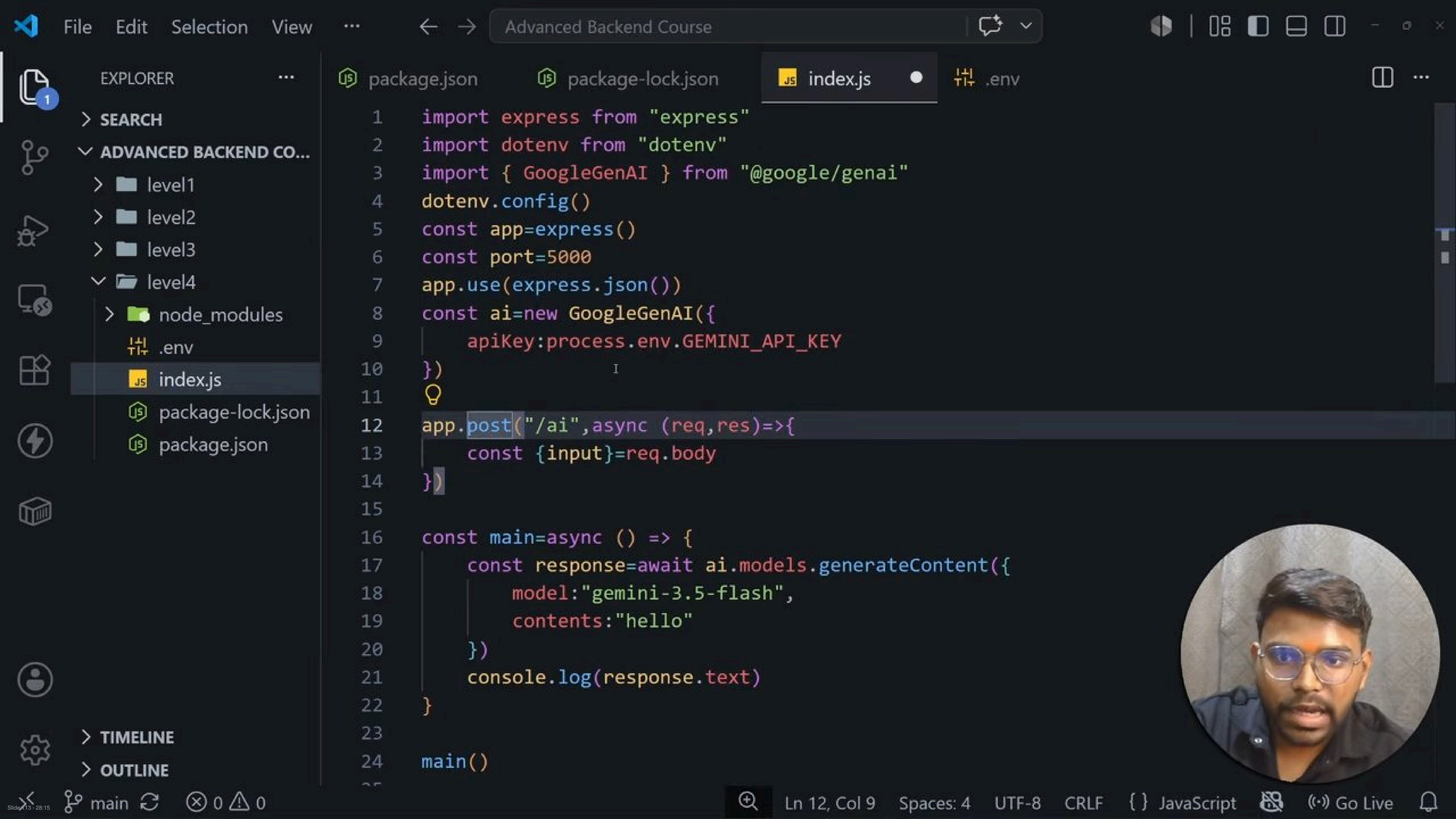
```
[nodemon] starting `node index.js`
```

```
◇ injected env (1) from .env // tip: ◇ secrets for agents [www.dotenvx.com]
```

```
server started
```

Hello! How can I help you today?





THUNDER CLIE...

New Request

ActivityCollectionsEnv

filter activity

GETlocalhost:3000/a...

2 days ago

POSTlocalhost:8000/ai

2 days ago

GETlocalhost:8000/s...

7 days ago

...

http://localhost:8080/auth

GETlocalhost:8000/c...

9 days ago

GETlocalhost:5000

11 days ago

POSTlocalhost:3000/...

15 days ago

GETlocalhost:8000/a...

package.jsonpackage-lock.jsonindex.js.env

1import express from "express"

2import dotenv from "dotenv"

3import { GoogleGenAI } from "@google/genai"

4dotenv.config()

5const app=express()

6const port=5000

7app.use(express.json())

8const ai=new GoogleGenAI({

9|apiKey:process.env.GEMINI_API_KEY

10|})

11

12app.post("/ai",async (req,res)=>{

13|const {input}=req.body

14|const response=await ai.models.generateContent({

15|model:"gemini-3.5-flash",

16|contents:input

17|})

18

19return res.status(200).json({"ai":response.text})

20})

21

22

23

24

25

Ln 22, Col 1Spaces: 4UTF-8CRLF{ }JavaScriptGo Live

FileEditSelectionView

THUNDER CLIE...

New Request

ActivityCollectionsEnv

filter activity

GETlocalhost:3000/a...

2 days ago

POSTlocalhost:8000/ai

2 days ago

GETlocalhost:8000/s...

7 days ago

GETlocalhost:8000/c...

9 days ago

GETlocalhost:5000

11 days ago

POSTlocalhost:3000/...

15 days ago

GETlocalhost:8000/a...

package.jsonpackage-lock.jsonindex.jslocalhost:3000/api/auth/r...

GETlocalhost:8000/upload

Send

QueryHeaders 2AuthBody 1TestsPre Run

JSONXMLTextFormForm-encodeGraphQLBinary

12345

"name":"virtual code",

"email":"vc@gmail.com",

"password":"vc123"

Status:Size:Time:

Response

JetBrains Plugin

New

Thunder Client plugin is now available for JetBrains IDEs.

Download

Go Live

main

00

THUNDER CLIE...

New Request

Activity

filter activity

GET localhost:3000/a...

2 days ago

POST localhost:8000/ai

2 days ago

GET localhost:8000/s...

7 days ago

GET localhost:8000/c...

9 days ago

GET localhost:5000

11 days ago

POST localhost:3000/...

15 days ago

GET localhost:8000/a...

package.json

package-lock.json

index.js

localhost:3000/api/auth/r.. *

.env

POSThttp://localhost:5000/ai

Send

Query

Headers²

Auth

Body¹

Tests

Pre Run

JSON

XML

Text

Form

Form-encode

GraphQL

Binary

1 {

2 : "what is javascript?"

3 }

Processing, Please Wait...

Response

Processing...

Cancel Request

main

0

0

Go Live

THUNDER CLIE...

New Request

Activity

filter activity

POSTlocalhost:3000/...

just now

POSTlocalhost:8000/ai

2 days ago

GETlocalhost:8000/s...

7 days ago

GETlocalhost:8000/c...

9 days ago

GETlocalhost:5000

11 days ago

POSTlocalhost:3000/...

15 days ago

GETlocalhost:8000/a...

package.jsonpackage-lock.jsonindex.jslocalhost:3000/api/auth/r...

.env

POSThttp://localhost:5000/ai

Send

QueryHeaders 2AuthBody 1TestsPre Run

JSONXMLTextFormForm-encodeGraphQLBinary

1 {
2 "input": "what is javascript?"
3 }

Status: 200 OKSize: 3.3 KBTime: 17.36 s

1 {
2 "ai:": "**JavaScript (often abbreviated as JS)** is a popular, high-level programming language that is one of the core technologies of the World Wide Web. To understand what JavaScript does, the easiest way is to look at the Pillars of Web Development\":
 1. **HTML (Hypertext Markup Language)**: The structure of a website (e.g., paragraphs, headings, images, and styling).
 2. **CSS (Cascading Style Sheets)**: The styling and design (e.g., colors, fonts, and spacing).
 3. **JavaScript**: The behavior and interactivity that makes a page alive and responsive.

What does JavaScript accomplish?

Slide 27 - 31:25

main

00

THUNDER CLIE...

New Request

Activity

filter activity

POST localhost:3000/...

just now

POST localhost:8000/ai

2 days ago

GET localhost:8000/s...

7 days ago

GET localhost:8000/c...

9 days ago

GET localhost:5000

11 days ago

POST localhost:3000/...

15 days ago

GET localhost:8000/a...

package.jsonpackage-lock.jsonindex.jslocalhost:3000/api/auth/r...

POST

http://localhost:5000/ai

Send

Query

Headers²

Auth

Body¹

Tests

Pre Run

JSON

XML

Text

Form

Form-encode

GraphQL

Binary

1

{

2

3

}

Status: 200 OK

Size: 3.3 KB

Time: 17.36 s

Response

JavaScript, a website is just a static document (like a printed flyer). With JavaScript, a website becomes an interactive application. Here are some common examples of JavaScript in action:

Interactive elements

Drop-down menus, image carousels, zoom-in features, and modal pop-up windows.

Asynchronous updates

Loading new content (like new posts in a Facebook or Twitter feed) without having to reload the entire webpage.

Form validation

Checking if you entered a valid email address or password before submitting to the server.

Animations

Making elements slide, fade, or change color on mouseover.

Web-based games

Directing gameplay and physics in browser-based games.

Copy

Response

main

0

0

THUNDER CLIE...

1

New Request

Activity

filter activity

POST localhost:3000/...

just now

POST localhost:8000/ai

2 days ago

GET localhost:8000/s...

7 days ago

GET localhost:8000/c...

9 days ago

GET localhost:5000

11 days ago

POST localhost:3000/...

15 days ago

GET localhost:8000/a...

package.jsonpackage-lock.jsonindex.jslocalhost:3000/api/auth/r...

.env

8const ai=new GoogleGenAI({

10})

11

12app.post("/ai",async (req,res)=>{

13const {input}=req.body

14const response=await ai.models.generateContent({

15model:"gemini-3.5-flash",

16contents:[

17{role:"system",

18parts:[{text:"you are a assistant and your name is jarvis."}]

19},

20{

21role:"user",

22parts:[{text}]

23}

24]}

25})

26

27return res.status(200).json({"ai":response.text})

28})

29

30

31

32

Ln 22, Col 28

Spaces: 4

UTF-8

CRLF

{ }

JavaScript

Go Live

FileEditSelectionView

Advanced Backend Course

localhost:3000/api/auth/r..

Explorer (Ctrl+Shift+E)

SEARCH

ADVA...

level1

level2

level3

level4

node_modules

.env

index.js

package-lock.json

package.json

package.json

package-lock.json

index.js

POST

http://localhost:5000/ai

Send

Query

Headers 2

Auth

Body 1

Tests

Pre Run

JSON

XML

Text

Form

Form-encode

GraphQL

Binary

1 {

2 "input": "what is javascript?"

3 }

Status: 200 OK

Size: 3.3 KB

Time: 17.36 s

Response

1 {

2 "ai:": "**JavaScript (often abbreviated as JS)** is a popular, high-level programming language that is one of the core technologies of the World Wide Web. To understand what JavaScript does, the easiest way is to look at the Pillars of Web Development":\n\n1. **HTML (Hypertext Markup Language)**: The **structure** of a website (e.g., paragraphs, headings, images, and links). **CSS (Cascading Style Sheets)**: The **styling** and design (e.g., colors, fonts, and spacing).\n3. **JavaScript**: The **behavior** and interactivity that makes a page alive and responsive.\n\n---\n\n### What does JavaScript accomplish?

main

0

0

Slide 37 - 35:39

Go Live

File

Edit

Selection

View

...

←

→

Advanced Backend Course

🗨

▼

📄

📁 package.json

📁 package-lock.json

📄 index.js

🔗 GET localhost:3000/api/auth/r..

🔗 .env

📁 ...

🔗

POST

▼

http://localhost:5000/ai

Send

🔍

Query

Headers²

Auth

Body¹

Tests

Pre Run

📄

JSON

XML

Text

Form

Form-encode

GraphQL

Binary

JSON Content

Format

1 {

2 "input": "what is javascript?"

3 }

Status: 200 OK Size: 3.3 KB Time: 17.36 s

Response Headers⁶ Cookies Results Docs </> ≡

1 {

2 "ai:": **JavaScript (often abbreviated as JS)** is a popular, high-level programming language that is one of the core technologies of the World Wide Web.

If you want to understand what JavaScript does, the easiest way is to look at the **"Three Pillars of Web Development"**:

1. **HTML (Hypertext Markup Language)**: The **structure** of a website (e.g., paragraphs, headings, images, and text).
2. **CSS (Cascading Style Sheets)**: The **styling** and design, including colors, fonts, layouts, and spacing.
3. **JavaScript**: The **behavior** and interactivity that makes the page alive and responsive.

What does JavaScript do?
Without JavaScript, a web page is just a static document (like a PDF).

Slide 38 - 35:42

main

🔄

🚫 0

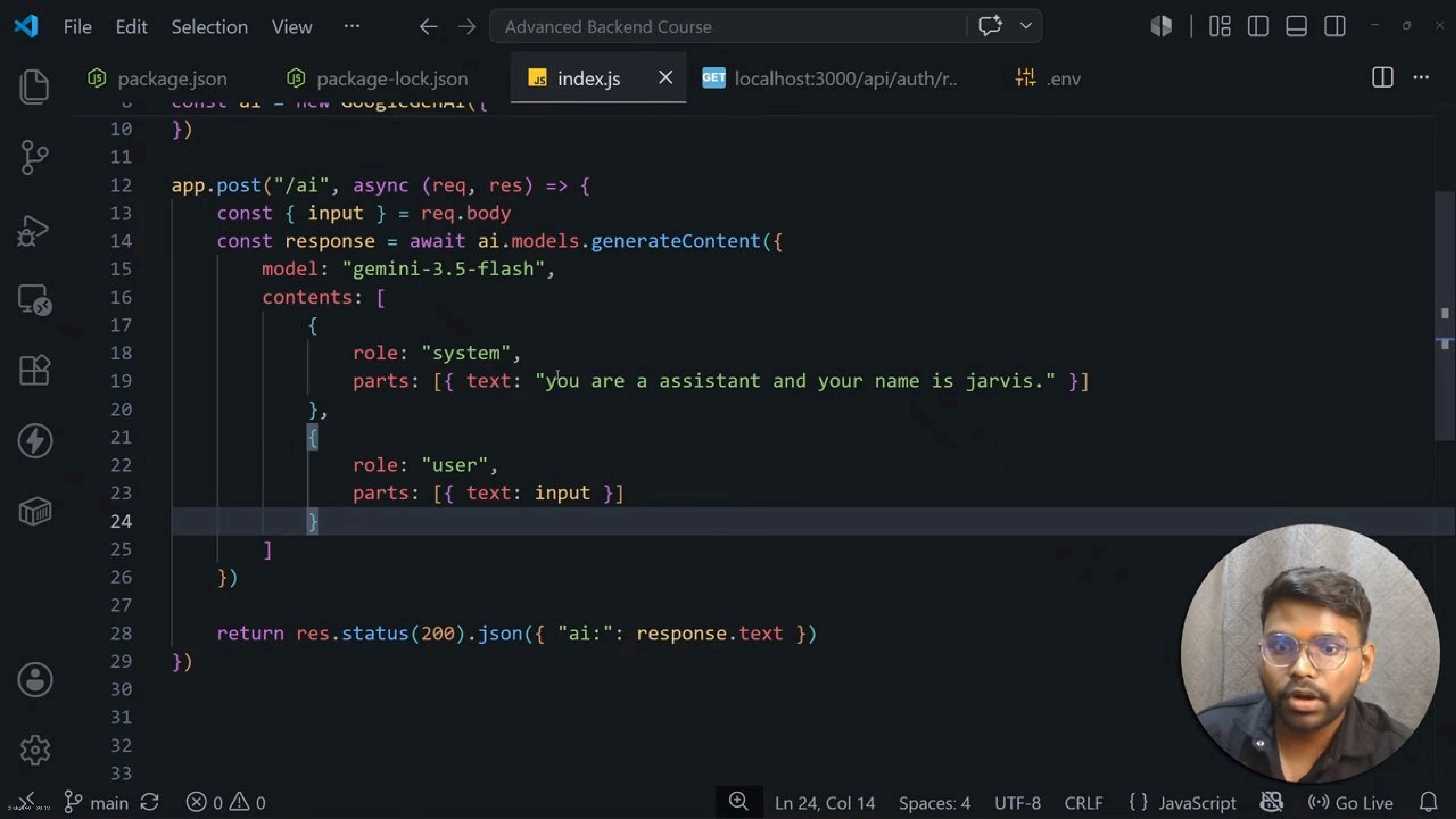
⚠️ 0

🔍

👤

🔗 Go Live

🔔



```
10  })
11
12  app.post("/ai", async (req, res) => {
13    const { input } = req.body
14    const response = await ai.models.generateContent({
15      model: "gemini-3.5-flash",
16      contents: [
17        {
18          role: "system",
19          parts: [{ text: "you are a assistant and your name is jarvis." }]
20        },
21        {
22          role: "user",
23          parts: [{ text: input }]
24        }
25      ]
26    })
27
28    return res.status(200).json({ "ai:": response.text })
29  })
30
31
32
33
```



VS Code

File Edit Selection View ...

Advanced Backend Course

localhost:3000/api/auth/r..

package.json package-lock.json index.js

GET localhost:3000/api/auth/r..

.env

POST http://localhost:5000/ai Send

Query Headers 2 Auth Body 1 Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content

Format

1 {

2 "input": "what is your name?"

3 }

Status: 200 OK Size: 93 Bytes Time: 9.22 s

Response Headers 6 Cookies Results Docs

</>

1 {

2 "ai": "My name is Jarvis. I am an AI assistant, built by Google. How can I help you today?"

3 }

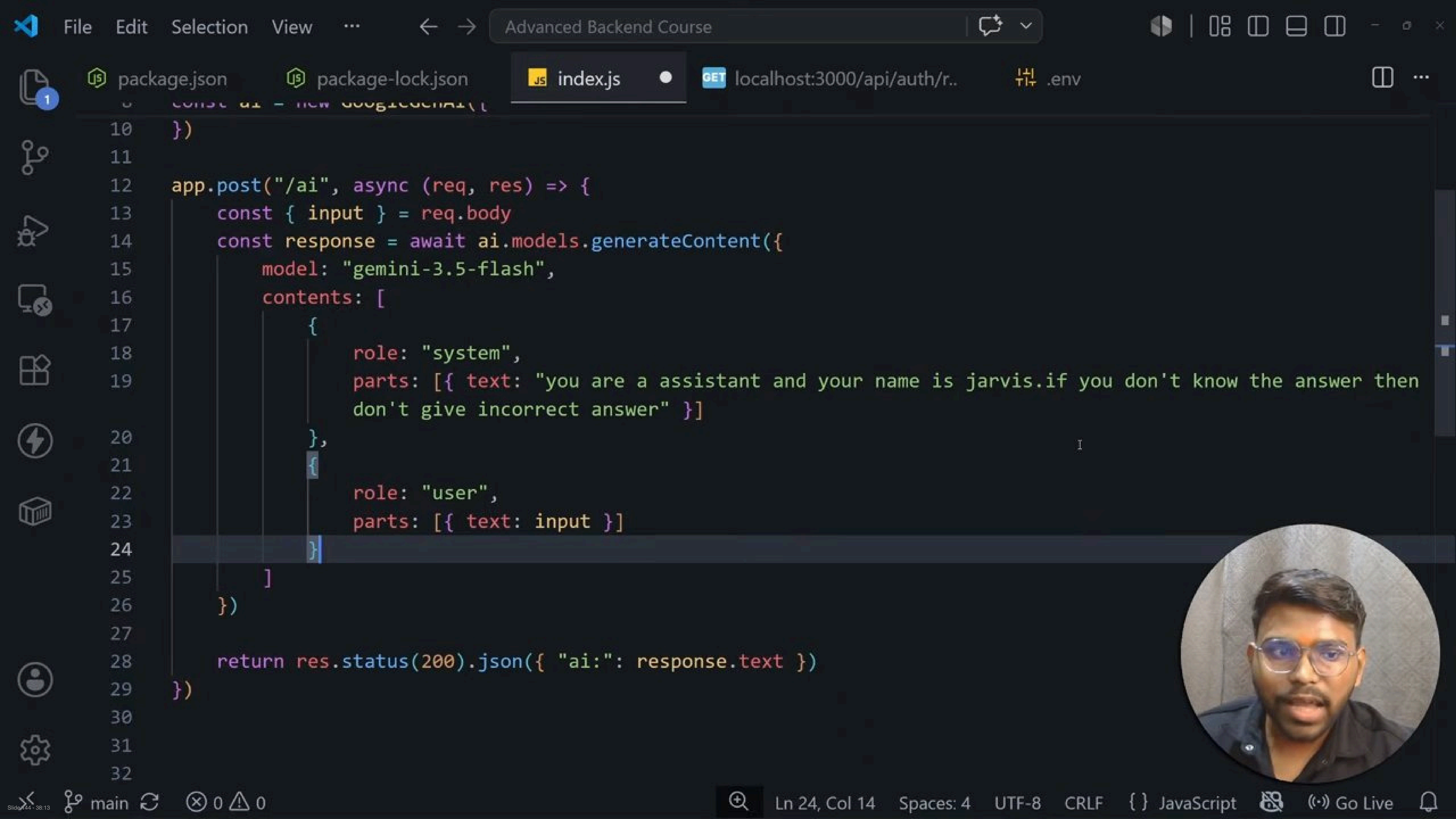
main

0 0

Go Live



Resp



```
10  })
11
12  app.post("/ai", async (req, res) => {
13    const { input } = req.body
14    const response = await ai.models.generateContent({
15      model: "gemini-3.5-flash",
16      contents: [
17        {
18          role: "system",
19          parts: [{ text: "you are a assistant and your name is jarvis.if you don't know the answer then don't give incorrect answer" }]
20        },
21        {
22          role: "user",
23          parts: [{ text: input }]
24        }
25      ]
26    })
27
28    return res.status(200).json({ "ai:": response.text })
29  })
30
31
32
```

Send

Status: 200 OK Size: 74 Bytes Time: 2.04 s

Pre Run

Binary

Format

```
1 {
2   "input": "what is date today?"
3 }
```

```
1 {
2   "ai:": "Today is Thursday, January 30, 2025. How
   may I assist you today?"
3 }
```



Excalidraw Whiteboard

Gemini API quickstart - general

Ask Gemini

excalidraw.com

Stroke

Font family

Font size

Text align

Opacity

Layers

Actions

user

write prompt

LLM

no access of Realtime data

no memory

LLM + Realtime data + Memory + action

AI Agent

Excalidraw+

Share

100%

Slide 152 - 43:43

Excalidraw Whiteboard

Gemini API quickstart - general

Ask Gemini

excalidraw.com

Excalidraw+ Share

Stroke

Font family

Font size

Text align

Opacity

Layers

Actions

user

LLM

no access of Realtime data
no memory

LLM + Realtime data + Memory + action

AI Agent

100%

Slide 153 - 43:46

Excalidraw Whiteboard

Gemini API quickstart - general

Ask Gemini

excalidraw.com

Excalidraw+

Share

Stroke

Font family

A

</>

A

Font size

S

M

L

XL

Text align

Opacity

0

100

Layers

Actions

no access of Realtime data

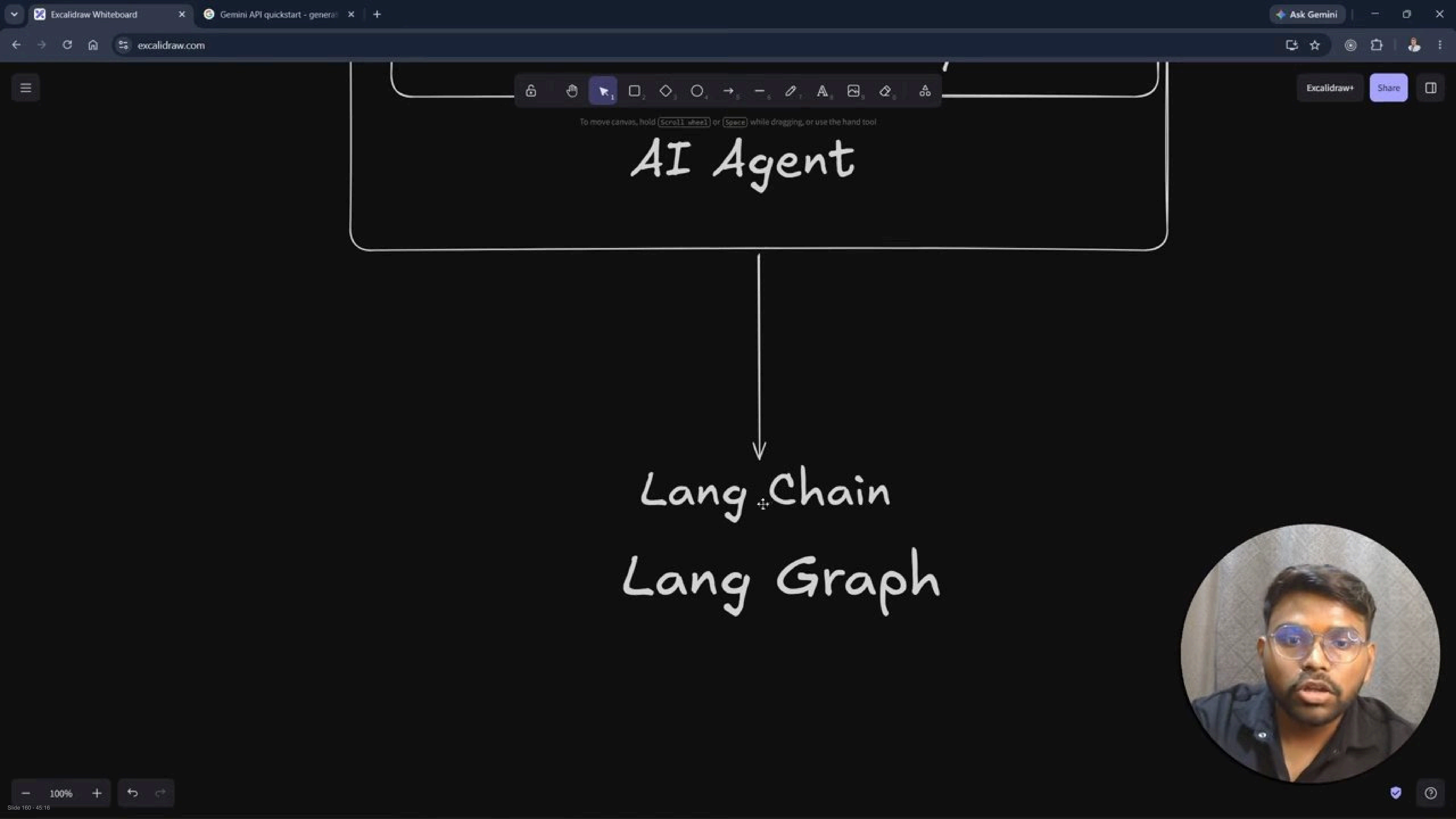
no memory

LLM + Realtime data + Memory + action

AI Agent

100%

Slide 154 - 43:49



What is an AI Agent?

An AI Agent is a system that uses an LLM plus tools, memory, and decision-making to perform tasks autonomously.

Think of it like:

```
LLM = Brain
```

```
Agent = Brain + Tools + Actions + Memory
```

Agent Components

An agent usually has:

- LLM (reasoning)
- Memory
- Tool calling
- Planning
- Execution loop
- APIs/databases access



Difference Between LLM and Agent

Feature	LLM	Agent
Generates text	✓	✓
Uses tools/APIs	✗	✓
Makes decisions	Limited	✓
Multi-step tasks	Weak	Strong
Memory	Minimal	Persistent
Autonomous actions	✗	✓



Popular Agent Frameworks

Developers commonly use:

- [LangChain ↗](#)
- [LangGraph ↗](#)
- [LlamaIndex ↗](#)
- [CrewAI ↗](#)
- [AutoGen ↗](#)



What is LangChain?

LangChain is a framework used to build:

- AI chatbots
- RAG systems
- AI agents
- Tool-calling apps
- Multi-step AI workflows

It helps connect:

LLM + Memory + Tools + Vector DB + APIs

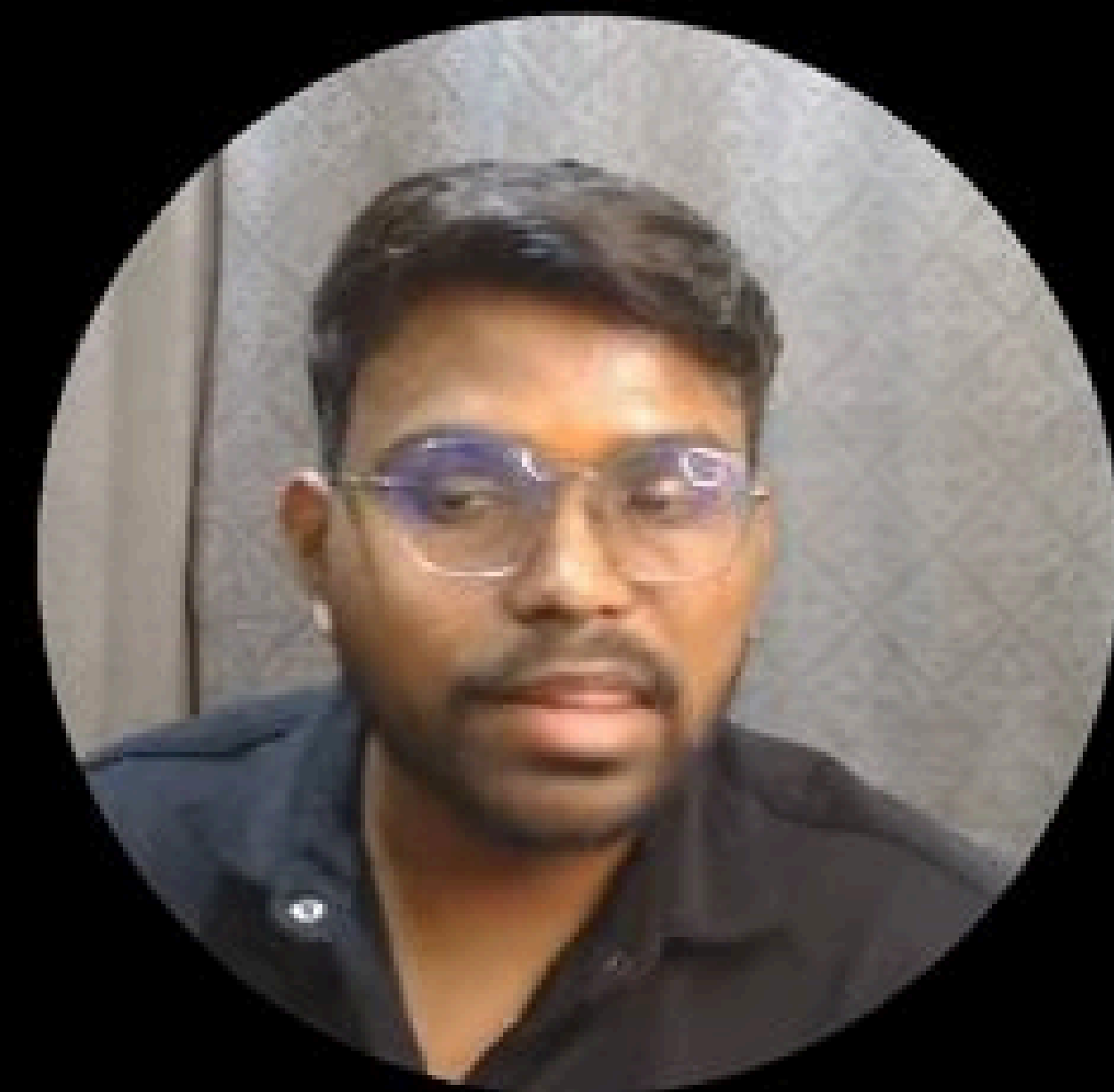


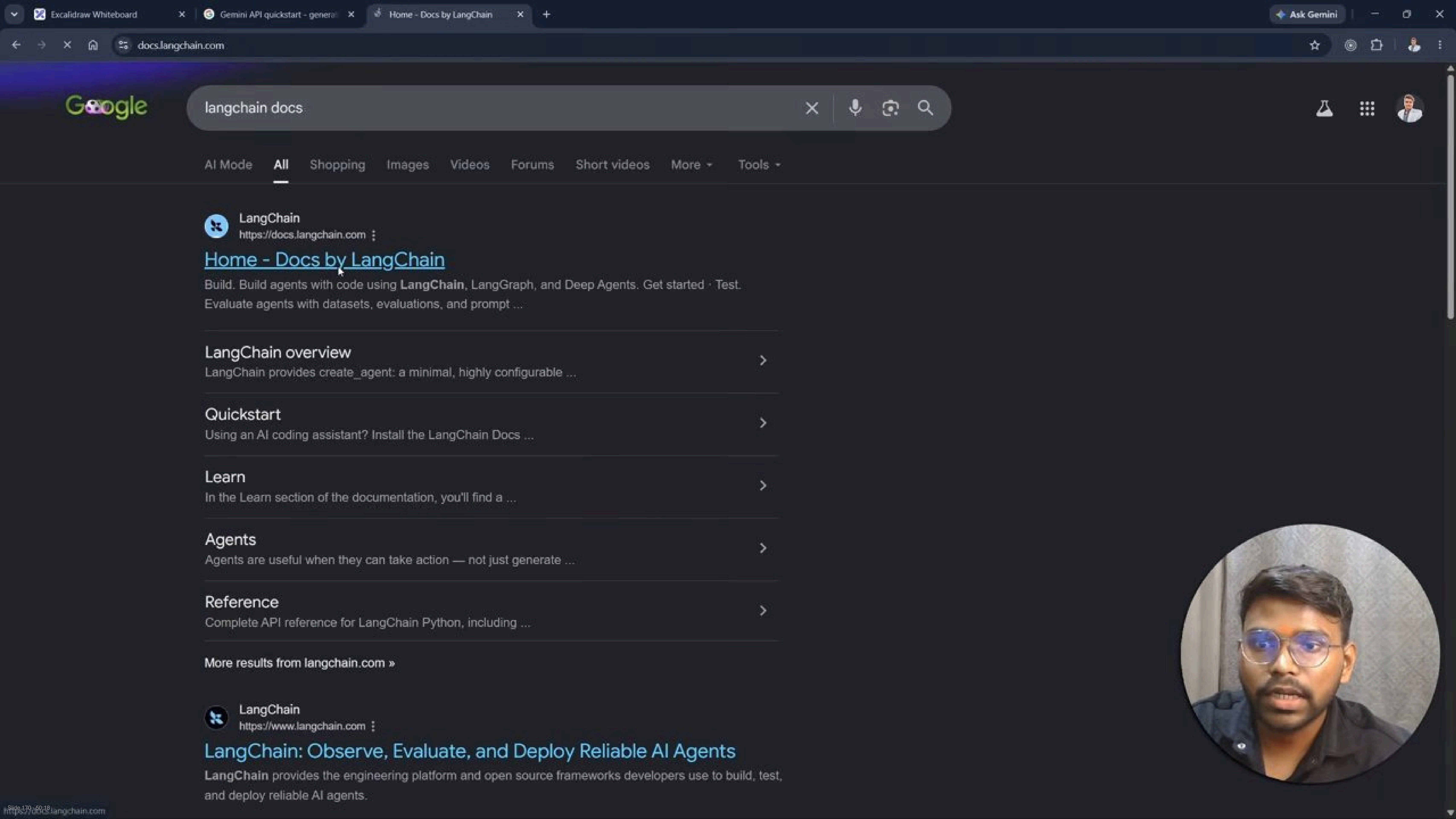
LangGraph

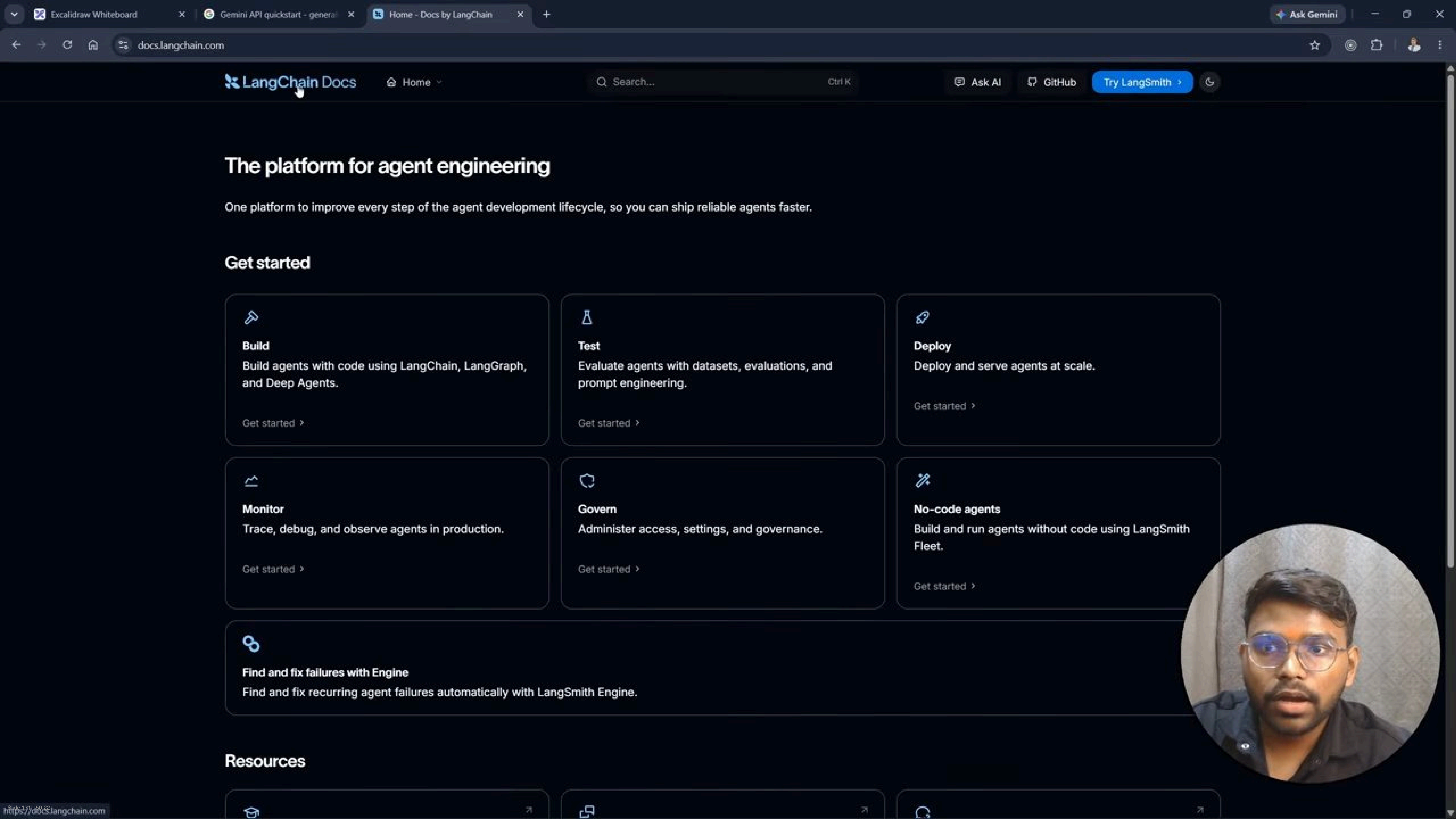
LangGraph is used for:

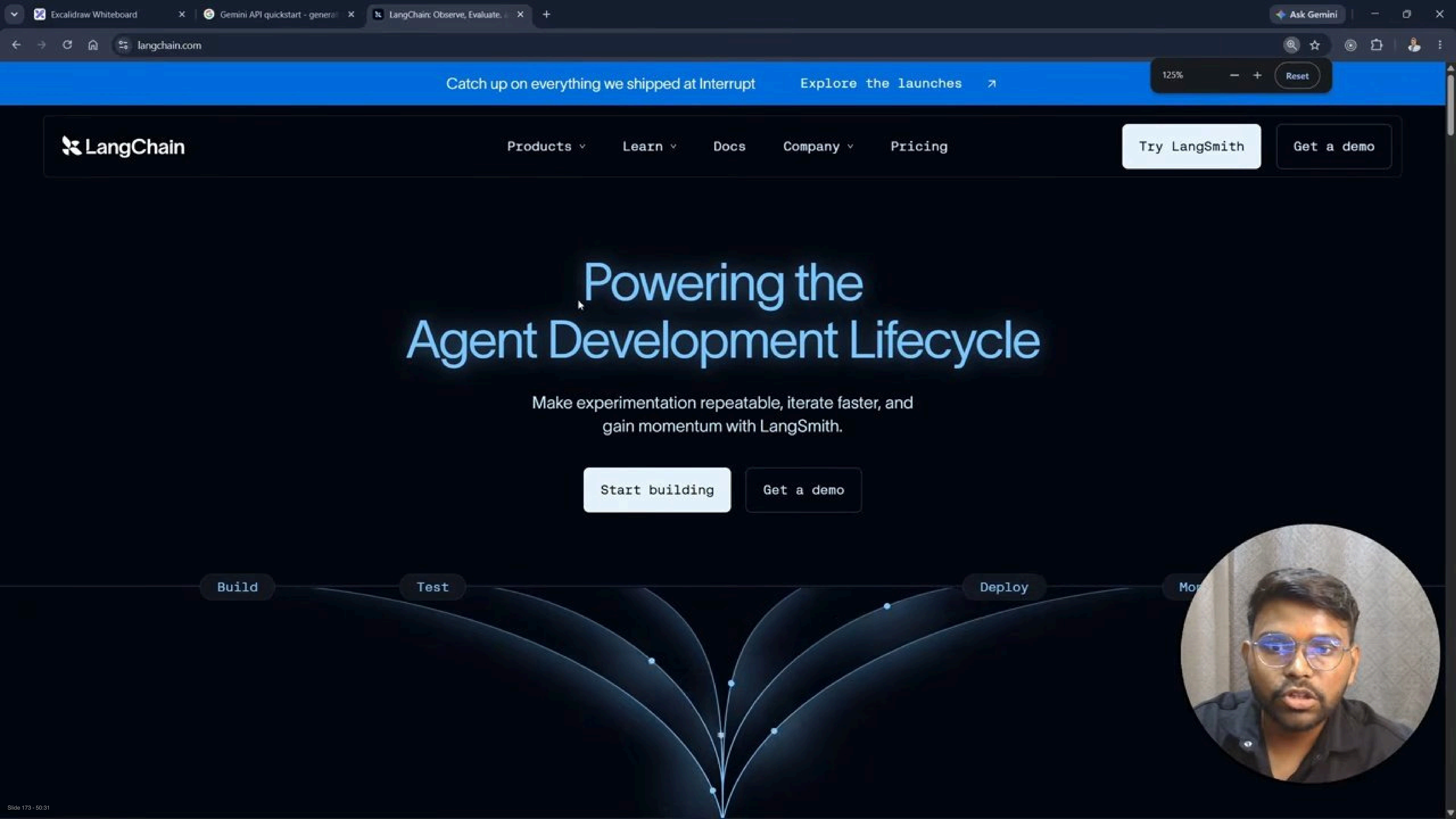
- Stateful AI agents
- Memory
- Multi-agent systems
- Workflow orchestration

This is production-level AI engineering.









Catch up on everything we shipped at Interrupt

Explore the launches ↗

125%

— +

Reset

LangChain

Products ▾

Learn ▾

Docs

Company ▾

Pricing

Try LangSmith

Get a demo

Powering the Agent Development Lifecycle

Make experimentation repeatable, iterate faster, and
gain momentum with LangSmith.

Start building

Get a demo

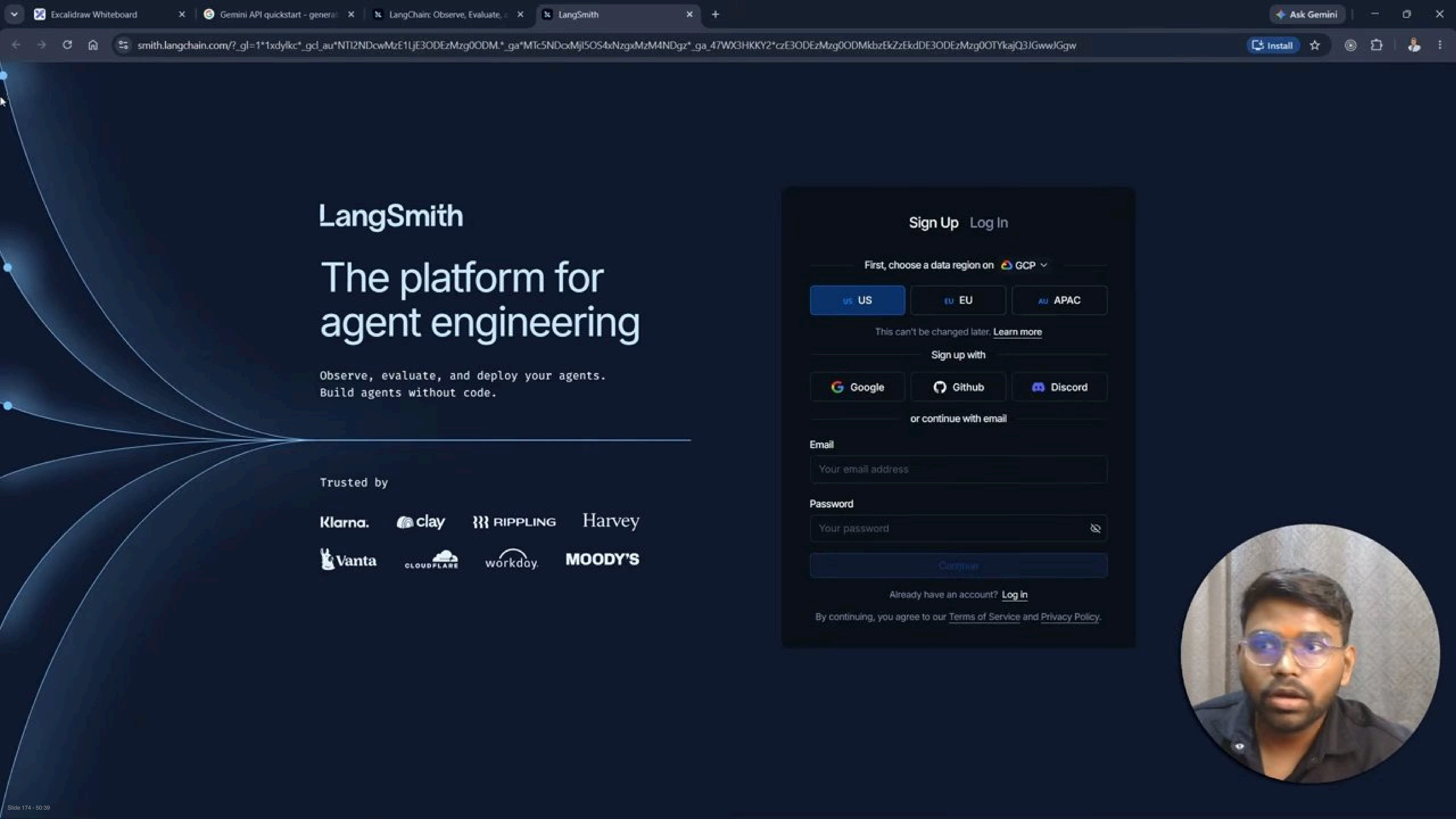
Build

Test

Deploy

Monitor





LangSmith

The platform for agent engineering

Observe, evaluate, and deploy your agents.
Build agents without code.

Trusted by



Sign Up Log In

First, choose a data region on GCP

US

EU

APAC

This can't be changed later. [Learn more](#)

Sign up with

Google

Github

Discord

or continue with email

Email

Your email address

Password

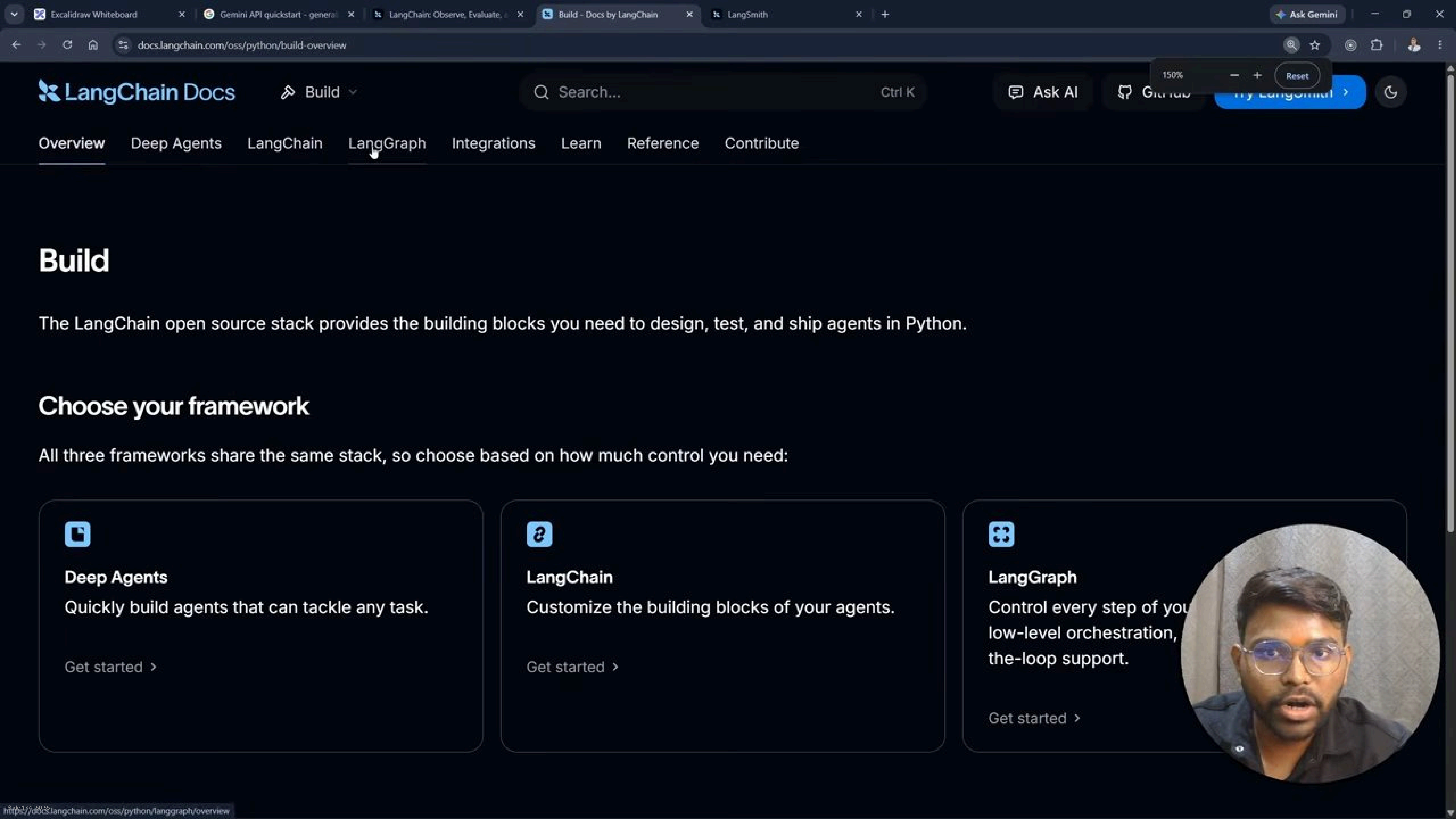
Your password

Continue

Already have an account? [Log in](#)

By continuing, you agree to our [Terms of Service](#) and [Privacy Policy](#).





Build

The LangChain open source stack provides the building blocks you need to design, test, and ship agents in Python.

Choose your framework

All three frameworks share the same stack, so choose based on how much control you need:



Deep Agents

Quickly build agents that can tackle any task.

[Get started >](#)



LangChain

Customize the building blocks of your agents.

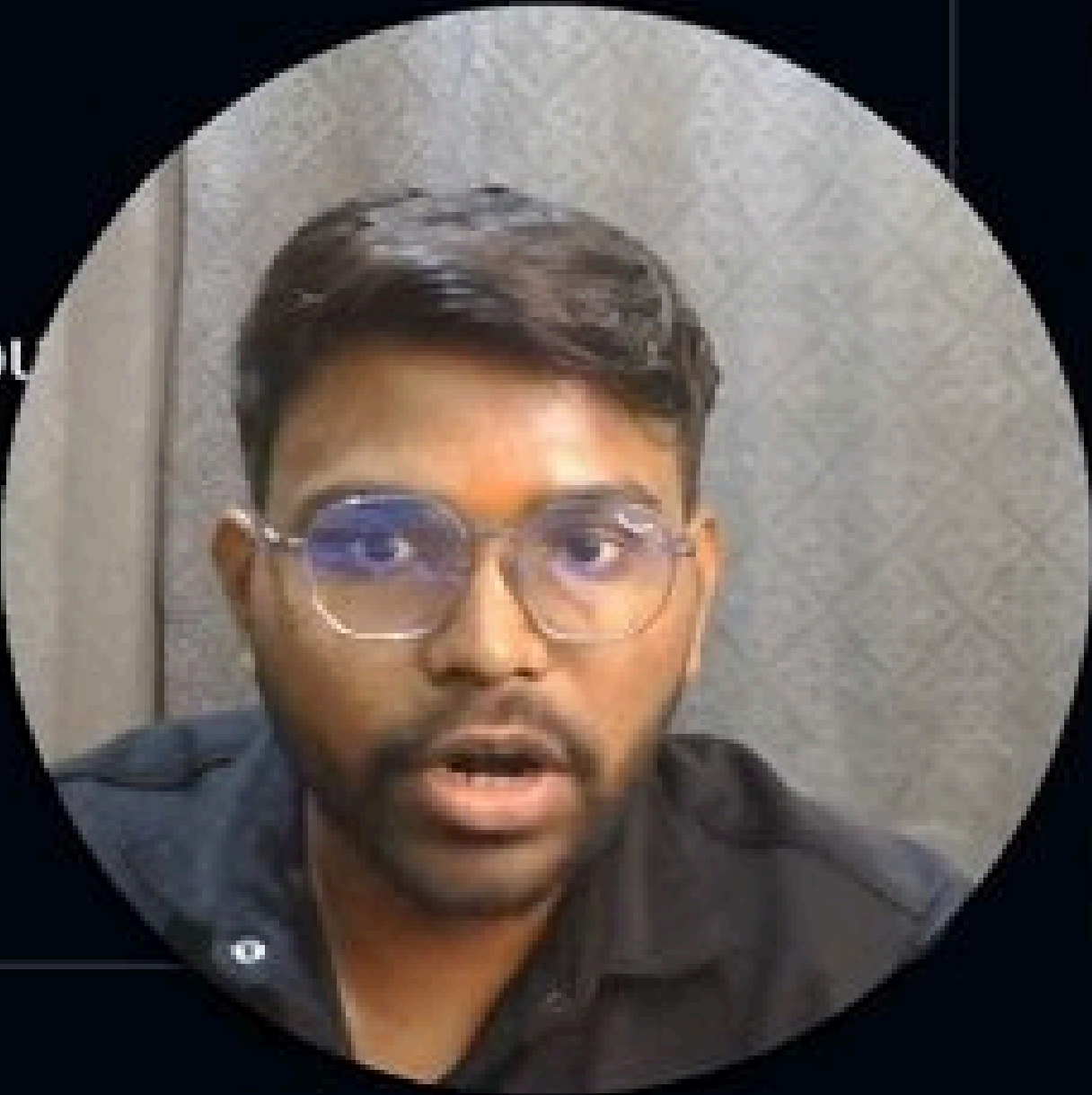
[Get started >](#)

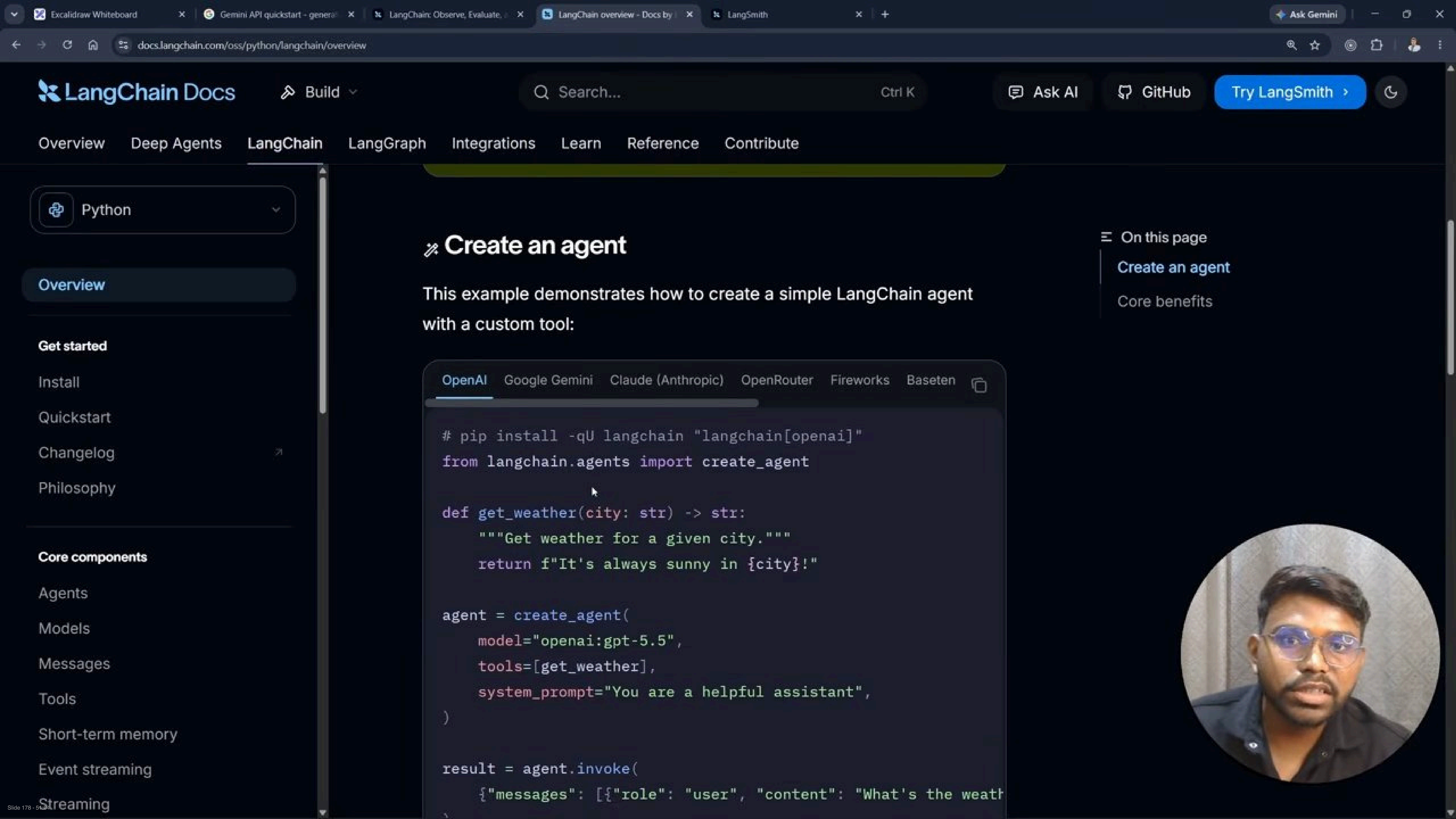


LangGraph

Control every step of your workflow with fine-grained, low-level orchestration, state, and the-loop support.

[Get started >](#)





Create an agent

This example demonstrates how to create a simple LangChain agent with a custom tool:

```
# pip install -qU langchain "langchain[openai]"
from langchain.agents import create_agent

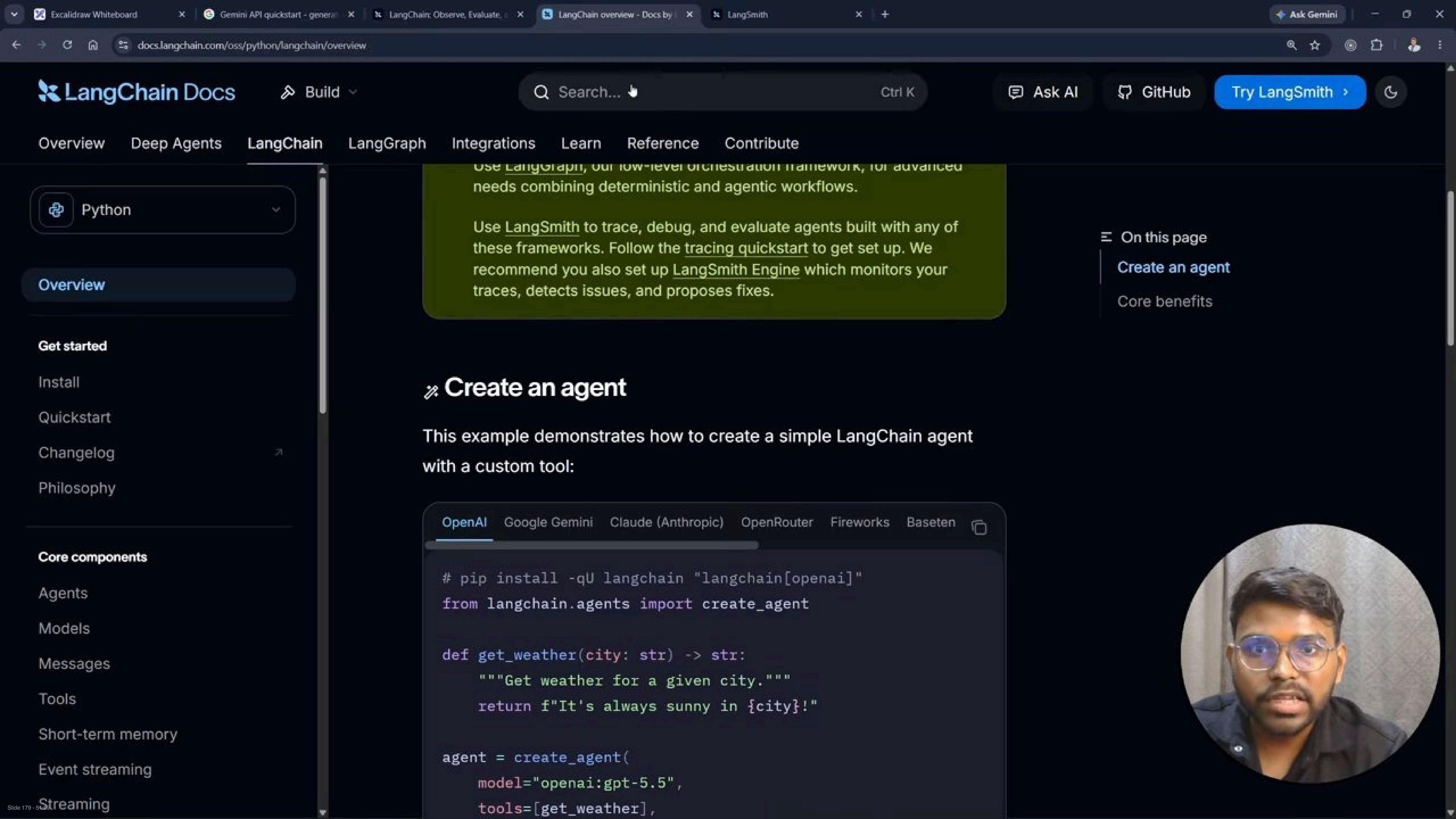
def get_weather(city: str) -> str:
    """Get weather for a given city."""
    return f"It's always sunny in {city}!"

agent = create_agent(
    model="openai:gpt-5.5",
    tools=[get_weather],
    system_prompt="You are a helpful assistant",
)

result = agent.invoke(
    {"messages": [{"role": "user", "content": "What's the weath"}
```

- On this page
- Create an agent
- Core benefits





Use [LangGraph](#), our low-level orchestration framework, for advanced needs combining deterministic and agentic workflows.

Use [LangSmith](#) to trace, debug, and evaluate agents built with any of these frameworks. Follow the [tracing quickstart](#) to get set up. We recommend you also set up [LangSmith Engine](#) which monitors your traces, detects issues, and proposes fixes.

On this page

[Create an agent](#)[Core benefits](#)

Create an agent

This example demonstrates how to create a simple LangChain agent with a custom tool:

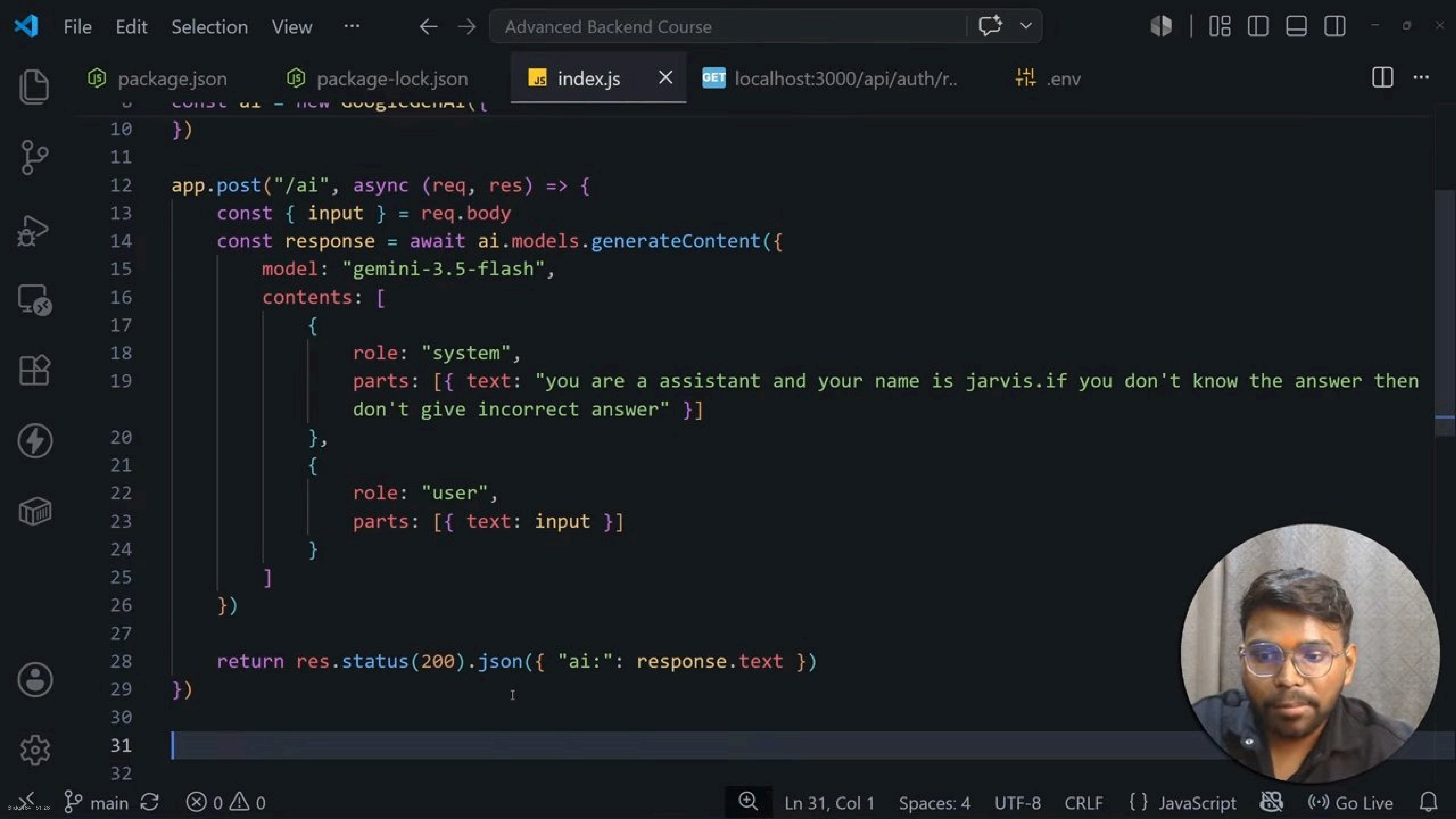
OpenAI Google Gemini Claude (Anthropic) OpenRouter Fireworks Baseten

```
# pip install -qU langchain "langchain[openai]"
from langchain.agents import create_agent
```

```
def get_weather(city: str) -> str:
    """Get weather for a given city."""
    return f"It's always sunny in {city}!"
```

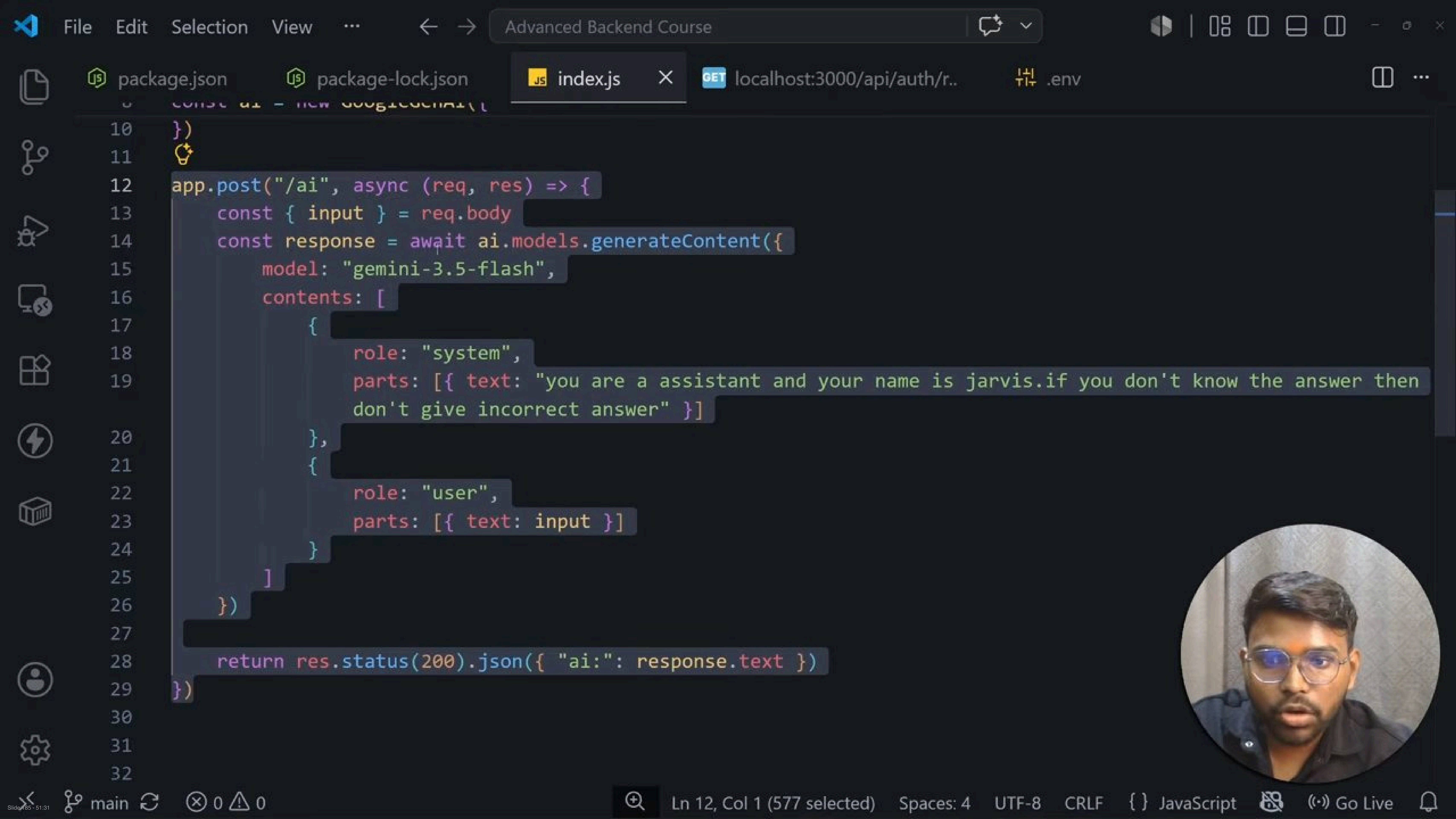
```
agent = create_agent(
    model="openai:gpt-5.5",
    tools=[get_weather],
```

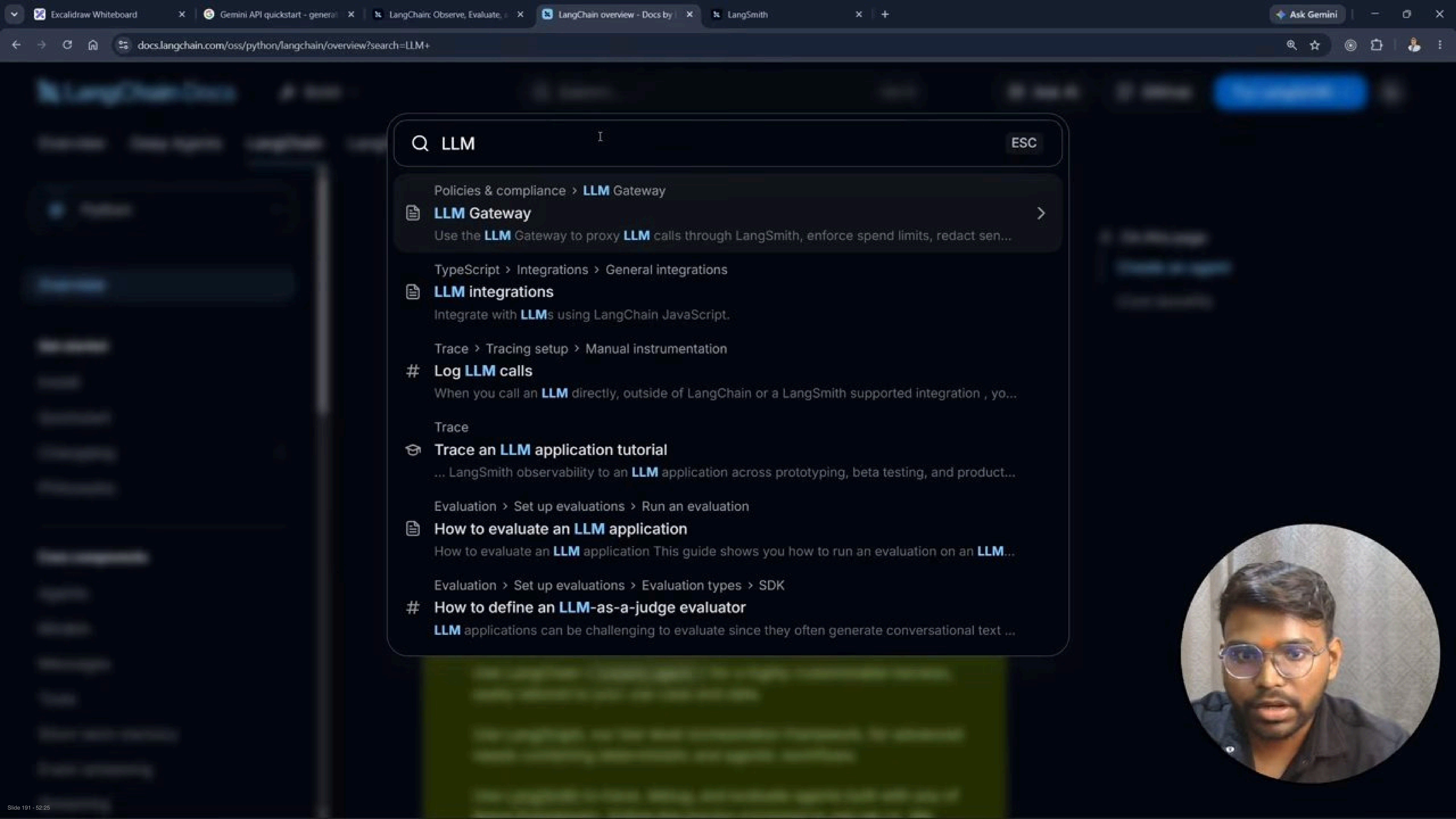




```
10  })
11
12  app.post("/ai", async (req, res) => {
13      const { input } = req.body
14      const response = await ai.models.generateContent({
15          model: "gemini-3.5-flash",
16          contents: [
17              {
18                  role: "system",
19                  parts: [{ text: "you are a assistant and your name is jarvis.if you don't know the answer then
20                      don't give incorrect answer" }]
21              },
22              {
23                  role: "user",
24                  parts: [{ text: input }]
25              }
26          ]
27      })
28      return res.status(200).json({ "ai:": response.text })
29  })
30
31
32
```







LLM

ESC

Policies & compliance > **LLM** Gateway



LLM Gateway



Use the **LLM** Gateway to proxy **LLM** calls through LangSmith, enforce spend limits, redact sen...

TypeScript > Integrations > General integrations



LLM integrations

Integrate with **LLMs** using LangChain JavaScript.

Trace > Tracing setup > Manual instrumentation



Log LLM calls

When you call an **LLM** directly, outside of LangChain or a LangSmith supported integration , yo...

Trace



Trace an LLM application tutorial

... LangSmith observability to an **LLM** application across prototyping, beta testing, and product...

Evaluation > Set up evaluations > Run an evaluation



How to evaluate an LLM application

How to evaluate an **LLM** application This guide shows you how to run an evaluation on an **LLM**...

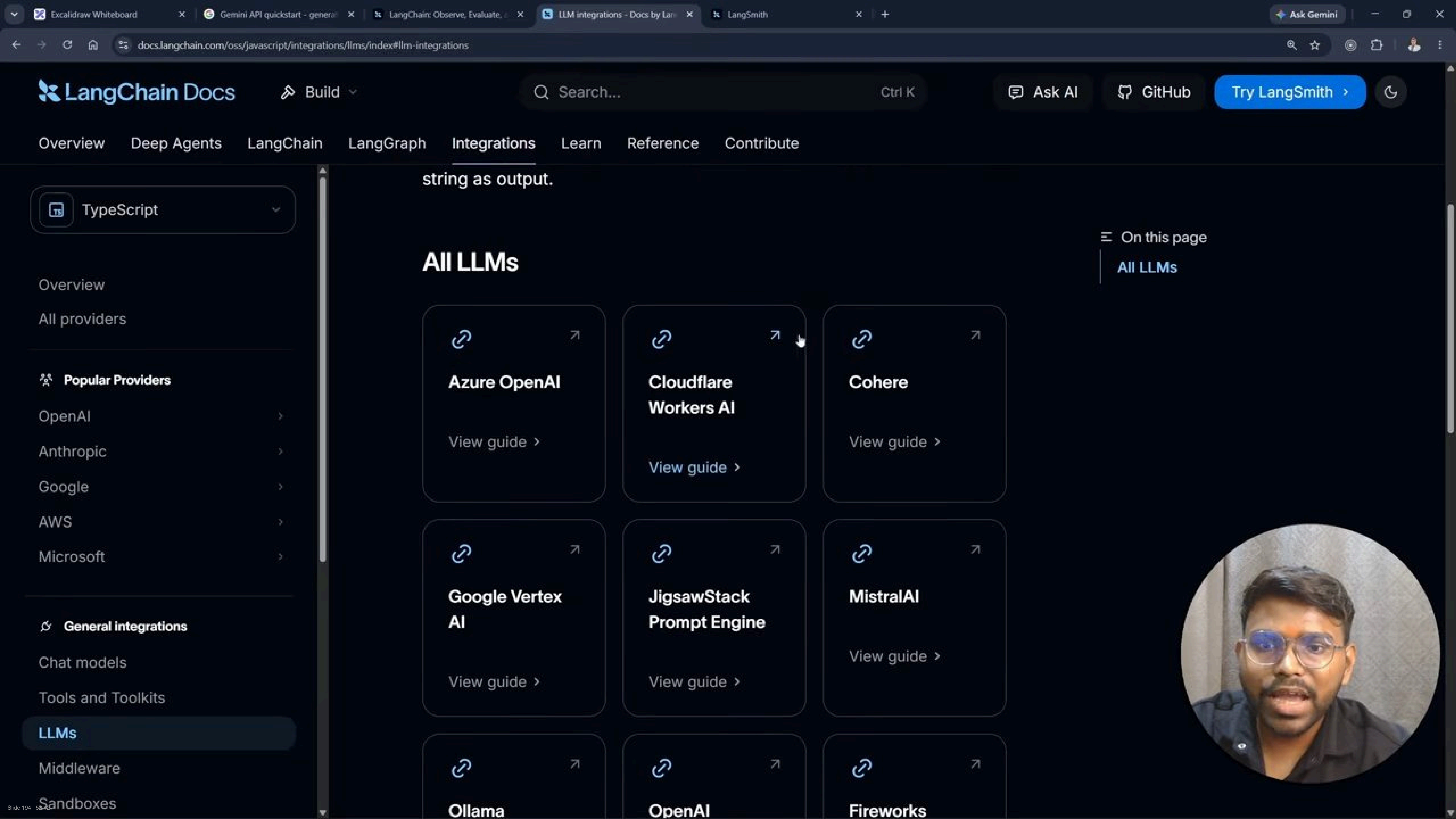
Evaluation > Set up evaluations > Evaluation types > SDK

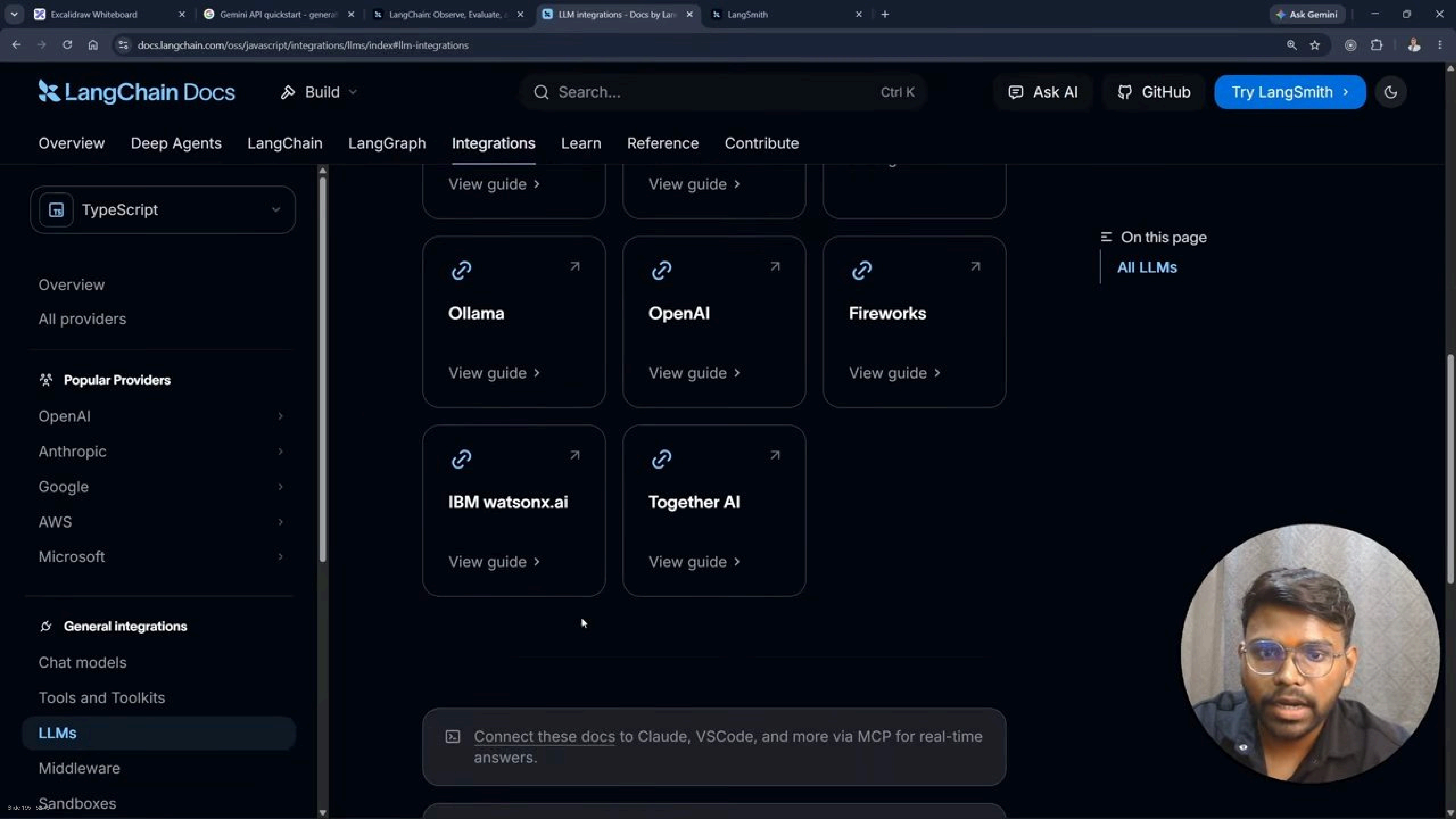


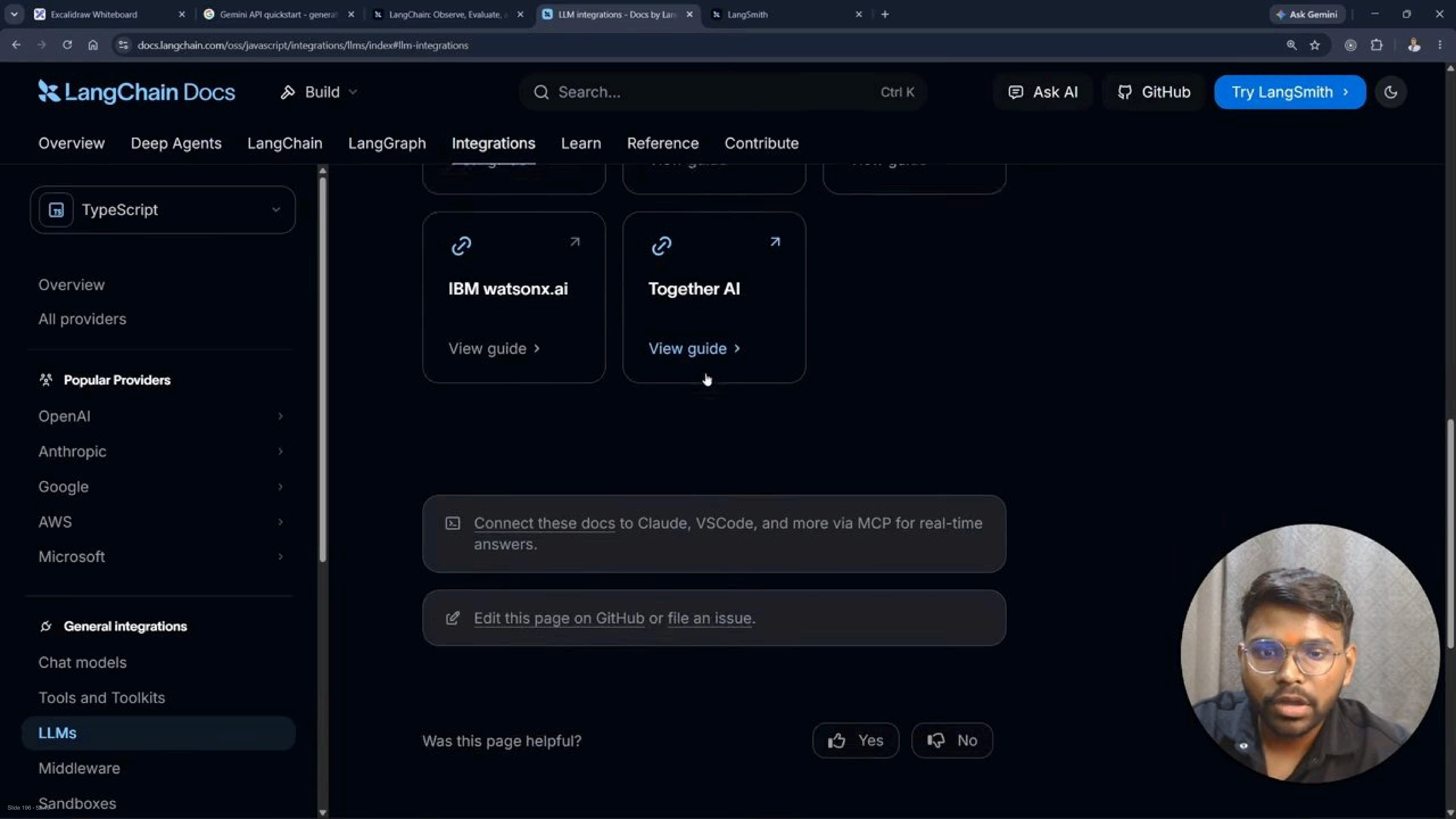
How to define an LLM-as-a-judge evaluator

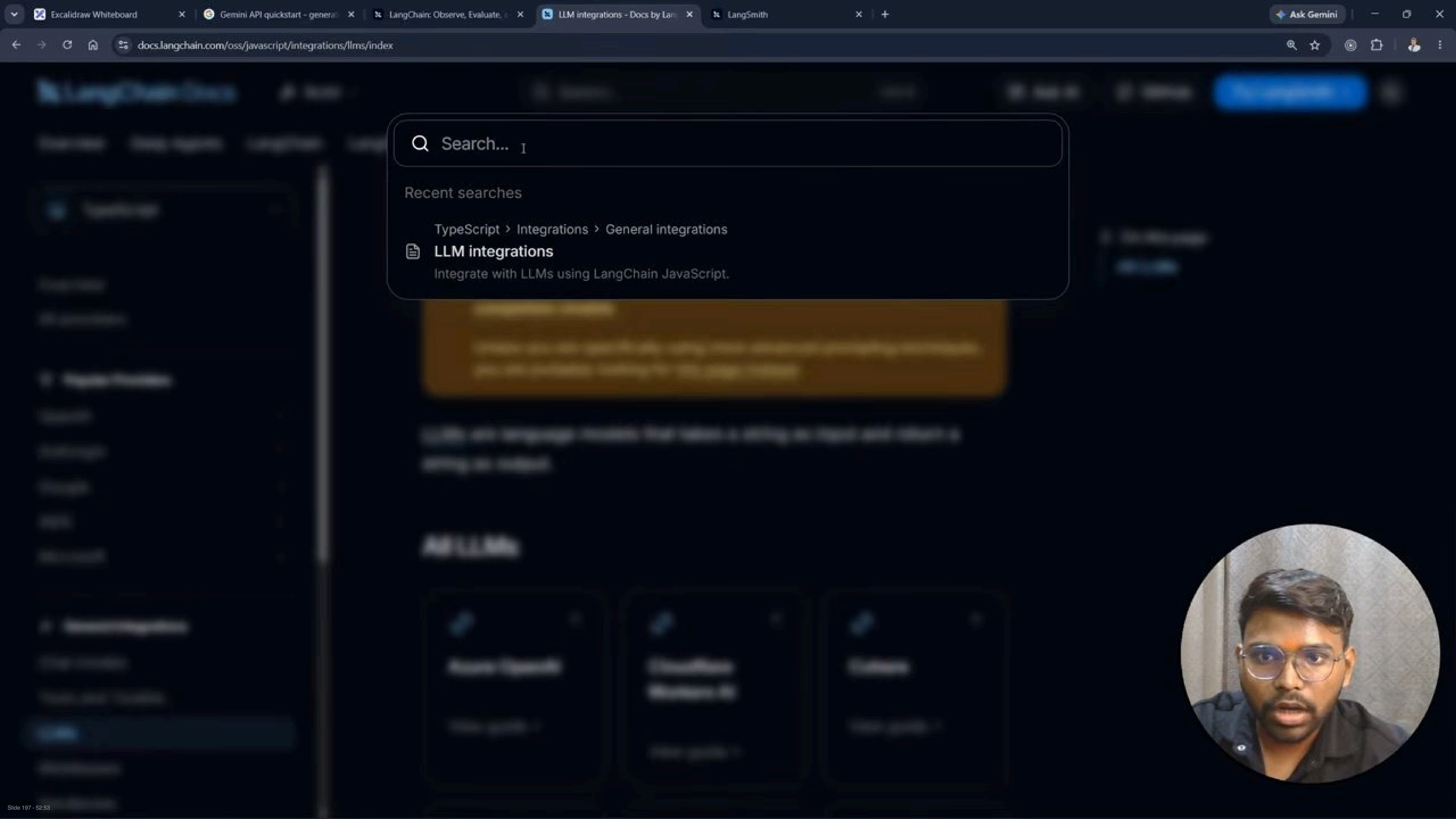
LLM applications can be challenging to evaluate since they often generate conversational text ...











Search... I

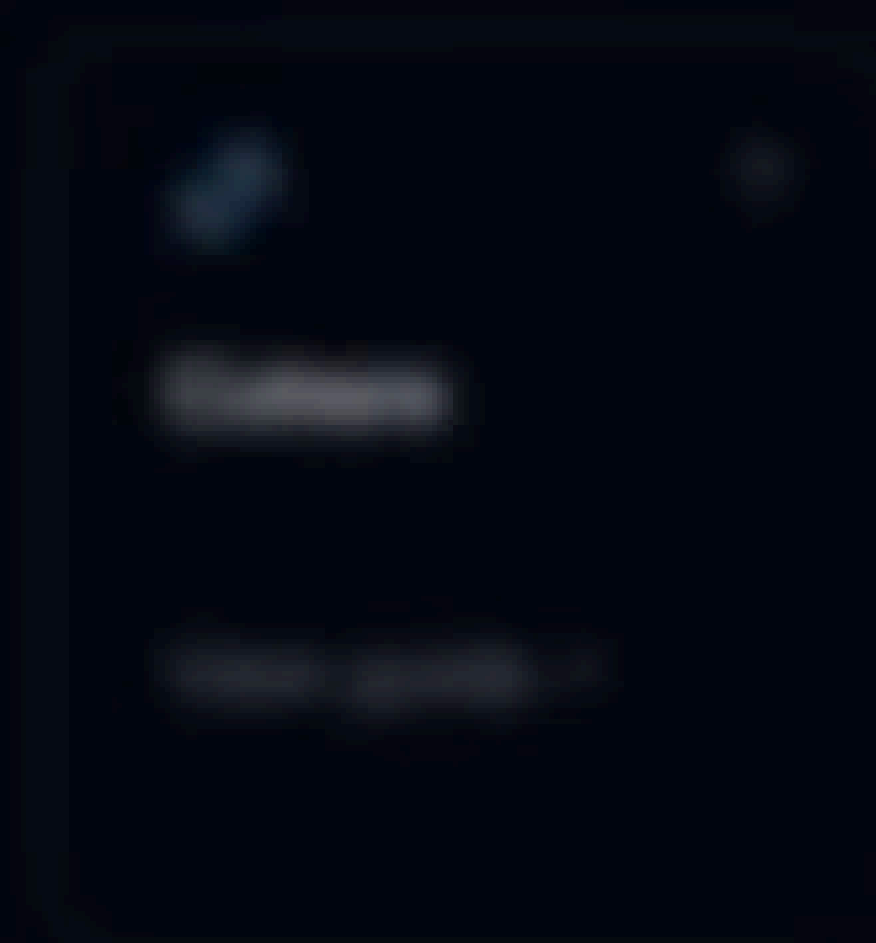
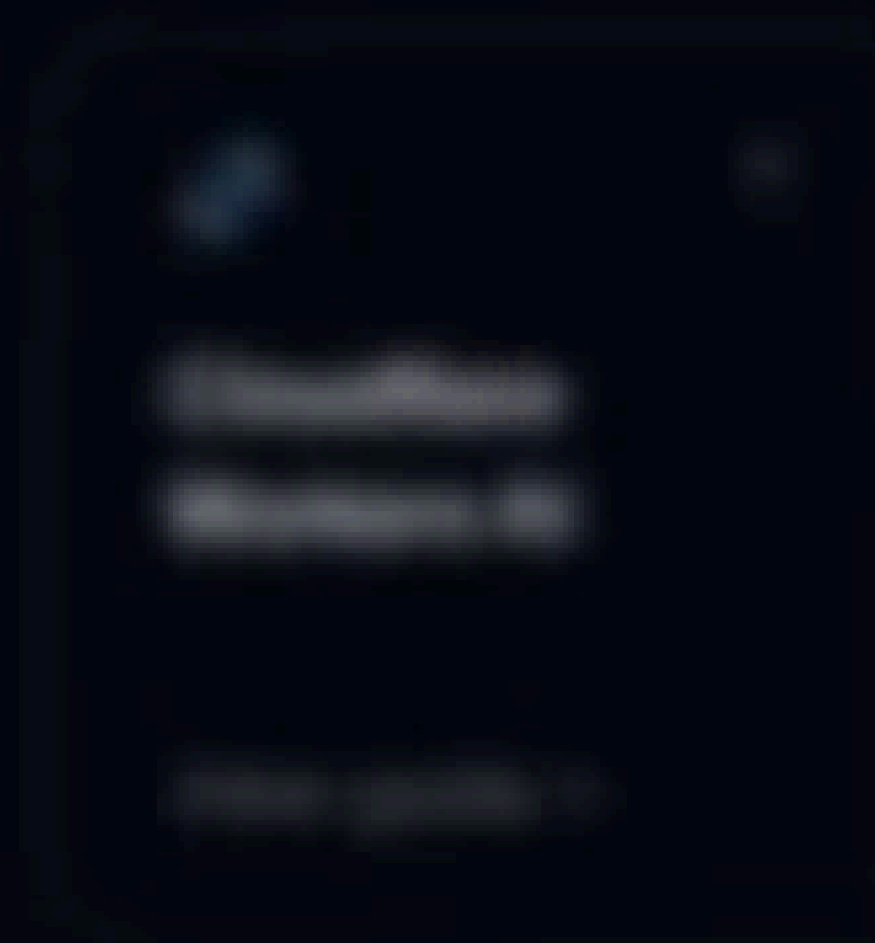
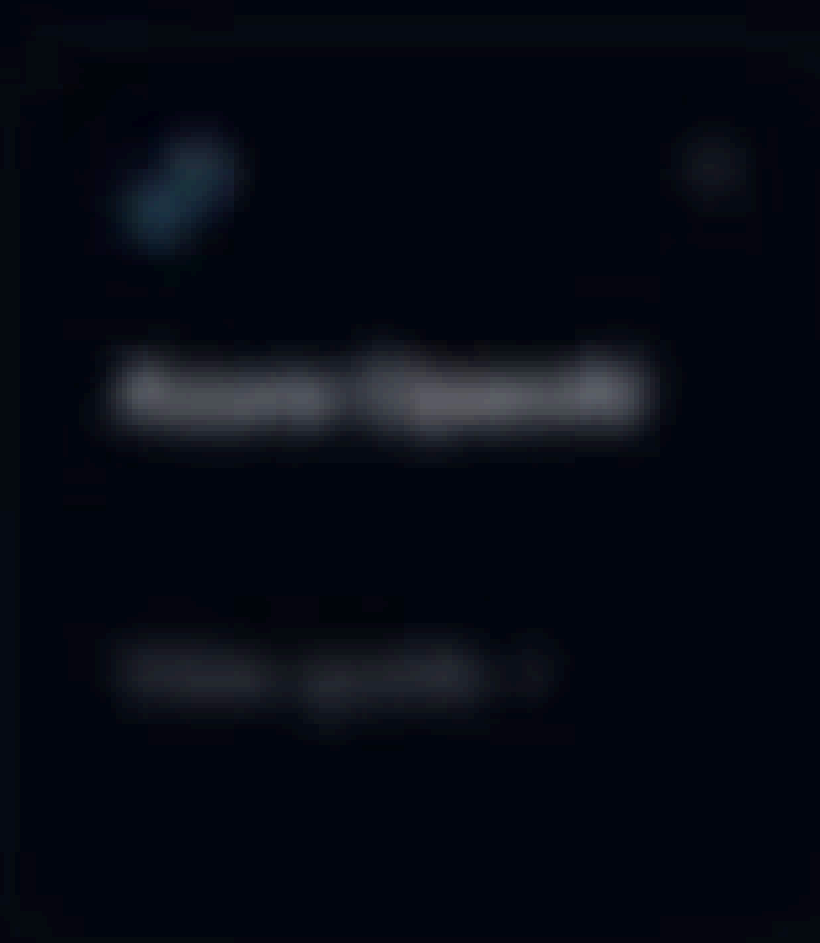
Recent searches

TypeScript > Integrations > General integrations

LLM integrations

Integrate with LLMs using LangChain JavaScript.

All LLMs



Excalidraw Whiteboard

Gemini API quickstart - genera

LangChain: Observe, Evaluate, >

LLM integrations - Docs by Lan

LangSmith

ChatGoogleGenerativeAI integr

+

docs.langchain.com/oss/javascript/integrations/chat/google_generative_ai

LangChain Docs

Build

Search...Ctrl K

Ask AI

GitHub

Try LangSmith

Overview

Deep Agents

LangChain

LangGraph

Integrations

Learn

Reference

Contribute

TypeScript

Overview

All providers

Popular Providers

OpenAI

Anthropic

Google

Overview

Chat

Tools

ChatGoogleGenerativeAI (Legacy)

ChatVertexAI (Legacy)

AWS

Microsoft

Popular Providers > Google

ChatGoogleGenerativeAI integration

Copy page

Google AI offers a number of different chat models, including the powerful Gemini series. For information on the latest models, their features, context windows, etc. head to the [Google AI docs](#).

This will help you getting started with `ChatGoogleGenerativeAI` [chat models](#). For detailed documentation of all `ChatGoogleGenerativeAI` features and configurations head to the [API reference](#).

ⓘ This library will be deprecated

This library is based on a deprecated library from Google and will be replaced by the [ChatGoogle](#) library. New implementations should use the [ChatGoogle](#) library instead and existing implementations should consider migrating.

On this page

Overview

Integration details

Model features

Setup

Credentials

Installation

Instantiation

Invocation

Safety settings

Tool calling

Built in

Code e

Context c

Gemini pro

API reference

Overview

Slide 200 - 53:14

LangChain Docs

Build

Search...Ctrl K

Ask AI

GitHub

Try LangSmith

Overview

Deep Agents

LangChain

LangGraph

Integrations

Learn

Reference

Contribute

TypeScript

Overview

All providers

Popular Providers

OpenAI

Anthropic

Google

Overview

Chat

Tools

ChatGoogleGenerativeAI (Legacy)

ChatVertexAI (Legacy)

AWS

Microsoft

Credentials

Get an API key here: <https://ai.google.dev/tutorials/setup>

Then set the `GOOGLE_API_KEY` environment variable:

```
export GOOGLE_API_KEY="your-api-key"
```

If you want to get automated tracing of your model calls you can also set your [LangSmith](#) API key by uncommenting below:

```
# export LANGSMITH_TRACING="true"
# export LANGSMITH_API_KEY="your-api-key"
```

Installation

The LangChain `ChatGoogleGenerativeAI` integration lives in the `@langchain/google-genai` package:

```
npm yarn pnpm
```

On this page

Overview

Integration details

Model features

Setup

Credentials

Installation

Instantiation

Invocation

Safety settings

Tool calling

Built in

Code e

Context c

Gemini pro

API reference

Slide 201 - 53:23

LangChain Docs

Build

Search...Ctrl K

Ask AI

GitHub

Try LangSmith

Overview

Deep Agents

LangChain

LangGraph

Integrations

Learn

Reference

Contribute

TypeScript

Overview

All providers

Popular Providers

OpenAI

Anthropic

Google

Overview

Chat

Tools

ChatGoogleGenerativeAI (Legacy)

ChatVertexAI (Legacy)

AWS

Microsoft

export GOOGLE_API_KEY="your-api-key"

If you want to get automated tracing of your model calls you can also set your [LangSmith](#) API key by uncommenting below:

export LANGSMITH_TRACING="true"
export LANGSMITH_API_KEY="your-api-key"

Installation

The LangChain `ChatGoogleGenerativeAI` integration lives in the `@langchain/google-genai` package:

npm yarn pnpm
npm install @langchain/google-genai @langchain/core

Instantiation

Now we can instantiate our model object and generate chat completions:

On this page

Overview

Integration details

Model features

Setup

Credentials

Installation

Instantiation

Invocation

Safety settings

Tool calling

Built in

Code e

Context c

Gemini pro

API reference



Slide 202 - 53:26

LangChain Docs

Build

Search...Ctrl K

Ask AI

GitHub

Try LangSmith

Overview

Deep Agents

LangChain

LangGraph

Integrations

Learn

Reference

Contribute

TypeScript

Overview

All providers

Popular Providers

OpenAI

Anthropic

Google

Overview

Chat

Tools

ChatGoogleGenerativeAI (Legacy)

ChatVertexAI (Legacy)

AWS

Microsoft

Instantiation

Now we can instantiate our model object and generate chat completions:

```
import { ChatGoogleGenerativeAI } from "@langchain/google-genai"

const llm = new ChatGoogleGenerativeAI({
  model: "gemini-2.5-pro",
  temperature: 0,
  maxRetries: 2,
  // other params...
})
```

Invocation

```
const aiMsg = await llm.invoke([
  [
    "system",
    "You are a helpful assistant that translates English to Hindi",
  ],
  ["human", "I love programming."],
])
```

On this page

Overview

Integration details

Model features

Setup

Credentials

Installation

Instantiation

Invocation

Safety settings

Tool calling

Built in tools

Code examples

Context c...

Gemini pro...

API reference



Slide 204 - 53:35

Excalidraw Whiteboard

Gemini API quickstart - general

LangChain: Observe, Evaluate, ...

LLM integrations - Docs by Lar...

LangSmith

ChatGoogleGenerativeAI integ...

OpenAI integration - Docs by l...

Ask Gemini

docs.langchain.com/oss/javascript/integrations/llms/openai

LangChain Docs

Build

Search...Ctrl K

Ask AI

GitHub

Try LangSmith

Overview

Deep Agents

LangChain

LangGraph

Integrations

Learn

Reference

Contribute

TypeScript

Overview

All providers

Popular Providers

OpenAI

Anthropic

Google

AWS

Microsoft

General Integrations

Chat models

Tools and Toolkits

LLMs

Middleware

Sandboxes

Instantiation

Now you can instantiate the model and generate text completions:

```
import { OpenAI } from "@langchain/openai"

const llm = new OpenAI({
  model: "gpt-3.5-turbo-instruct",
  temperature: 0,
  maxTokens: undefined,
  timeout: undefined,
  maxRetries: 2,
  apiKey: process.env.OPENAI_API_KEY,
  // other params...
})
```

Invocation

```
const inputText = "OpenAI is an AI company that "

const completion = await llm.invoke(inputText)
completion
```

On this page

Overview

Integration details

Setup

Credentials

Installation

Instantiation

Invocation

Custom URLs

API reference



Slide 206 - 540

Excalidraw Whiteboard

Gemini API quickstart - generai

LangChain: Observe, Evaluate, ...

LLM integrations - Docs by Lan

LangSmith

ChatGoogleGenerativeAI Integ

OpenAI integration - Docs by

Ask Gemini

docs.langchain.com/oss/javascript/integrations/chat/google_generative_ai

LangChain Docs

Build

Search...Ctrl K

Ask AI

GitHub

Try LangSmith

Overview

Deep Agents

LangChain

LangGraph

Integrations

Learn

Reference

Contribute

TypeScript

Overview

All providers

Popular Providers

OpenAI

Anthropic

Google

Overview

Chat

Tools

ChatGoogleGenerativeAI (Legacy)

ChatVertexAI (Legacy)

AWS

Microsoft

Get an API key here: <https://ai.google.dev/tutorials/setup>

Then set the `GOOGLE_API_KEY` environment variable:

```
export GOOGLE_API_KEY="your-api-key"
```

If you want to get automated tracing of your model calls you can also set your [LangSmith](#) API key by uncommenting below:

```
# export LANGSMITH_TRACING="true"
# export LANGSMITH_API_KEY="your-api-key"
```

Installation

The LangChain `ChatGoogleGenerativeAI` integration lives in the `@langchain/google-genai` package:

```
npm yarn pnpm

npm install @langchain/google-genai @langchain/core
```

On this page

Overview

Integration details

Model features

Setup

Credentials

Installation

Instantiation

Invocation

Safety settings

Tool calling

Built in

Code e

Context c

Gemini pro

API reference

Slide 207 - 55:24

Excalidraw Whiteboard

Gemini API quickstart - general

LangChain: Observe, Evaluate.

LLM integrations - Docs by Lan

LangSmith

ChatGoogleGenerativeAI integ

OpenAI integration - Docs by

Ask Gemini

docs.langchain.com/oss/javascript/integrations/chat/google_generative_ai

LangChain Docs

Build

Search...Ctrl K

Ask AI

GitHub

Try LangSmith

Overview

Deep Agents

LangChain

LangGraph

Integrations

Learn

Reference

Contribute

TypeScript

Overview

All providers

Popular Providers

OpenAI

Anthropic

Google

Overview

Chat

Tools

ChatGoogleGenerativeAI (Legacy)

ChatVertexAI (Legacy)

AWS

Microsoft

Installation

The LangChain ChatGoogleGenerativeAI integration lives in the @langchain/google-genai package:

npm yarn pnpm

npm install @langchain/google-genai @langchain/core

Instantiation

Now we can instantiate our model object and generate chat completions:

import { ChatGoogleGenerativeAI } from "@langchain/google-genai"

const llm = new ChatGoogleGenerativeAI({
 model: "gemini-2.5-pro",
 temperature: 0,
 maxRetries: 2,
 ...
})

On this page

Overview

Integration details

Model features

Setup

Credentials

Installation

Instantiation

Invocation

Safety settings

Tool calling

Built in tools

Code examples

Context management

Gemini pro

API reference



Slide 209 - 55:36

LangChain Docs

Build

Search...Ctrl K

Ask AI

GitHub

Try LangSmith

Overview

Deep Agents

LangChain

LangGraph

Integrations

Learn

Reference

Contribute

TypeScript

Overview

All providers

Popular Providers

OpenAI

Anthropic

Google

Overview

Chat

Tools

ChatGoogleGenerativeAI (Legacy)

ChatVertexAI (Legacy)

AWS

Microsoft

Instantiation

Now we can instantiate our model object and generate chat completions:

```
import { ChatGoogleGenerativeAI } from "@langchain/google-genai";

const llm = new ChatGoogleGenerativeAI({
  model: "gemini-2.5-pro",
  temperature: 0,
  maxRetries: 2,
  // other params...
});
```

Invocation

```
const aiMsg = await llm.invoke([
  [
    "system",
    "You are a helpful assistant that translates English to Hindi.",
  ],
  ["human", "I love programming."],
]);
```

On this page

Overview

Integration details

Model features

Setup

Credentials

Installation

Instantiation

Invocation

Safety settings

Tool calling

Built in tools

Code examples

Context management

Gemini pro

API reference

Slide 216 - 57:02

1

EXPLORER

SEARCH

ADVANCED BACKEND CO...

level4

node_modules

type-is

undefsafe

undici-types

unpipe

vary

web-streams-p...

wrappy

ws

zod

.package-lock.js...

.env

index.js

package-lock.json

package.json

TIMELINE

OUTLINE

package.json

package-lock.json

index.js

localhost:3000/api/auth/r..

.env

30 // })

31

32 // return res.status(200).js

33 // })

34

35 //with langchain

36

37

38

39 const llm=new ChatGoogleGenerativeAI()

40

41

42

43

44 app.get("/", (req, res) => {

45 | return res.json({ message: "hello from level4" })

46 | })

47

48

49 app.listen(port, () => {

50 | console.log("server started")

51 | })

ChatGoogleGenerativeAI(model: ModelParams["model"], fields?: Omit<GoogleGenerativeAIChatInput, "model">): ChatGoogleGenerativeAI

Model Name to use

1/2 Note: The format must follow the pattern - {model}

main

0

0

Ln 39, Col 38

Spaces: 4

UTF-8

CRLF

{ } JavaScript

Go Live

Excalidraw Whiteboard

Gemini API quickstart - generai

LangChain: Observe, Evaluate,

LLM Integrations - Docs by Lar

LangSmith

ChatGoogleGenerativeAI integi

OpenAI Integration - Docs by

Ask Gemini

docs.langchain.com/oss/javascript/integrations/chat/google_generative_ai

LangChain Docs

Build

Search...Ctrl K

Ask AI

GitHub

Try LangSmith

Overview

Deep Agents

LangChain

LangGraph

Integrations

Learn

Reference

Contribute

TypeScript

Overview

All providers

Popular Providers

OpenAI

Anthropic

Google

Overview

Chat

Tools

ChatGoogleGenerativeAI (Legacy)

ChatVertexAI (Legacy)

AWS

Microsoft

Instantiation

Now we can instantiate our model object and generate chat completions:

```
import { ChatGoogleGenerativeAI } from "@langchain/google-genai";

const llm = new ChatGoogleGenerativeAI({
  model: "gemini-2.5-pro",
  temperature: 0,
  maxRetries: 2,
  // other params...
});
```

Invocation

```
const aiMsg = await llm.invoke([
  "system",
  "You are a helpful assistant that translates English to
```

On this page

Overview

Integration details

Model features

Setup

Credentials

Installation

Instantiation

Invocation

Safety settings

Tool calling

Built in

Code e

Context c

Gemini pro

API reference

Slide 232 - 58:39

File

Edit

Selection

View

...

←

→

Advanced Backend Course

🗨

▼

🏠

🔍

🔗

🔧

📺

🔌

📦

👤

⚙️

🏠

🔍

🔄

🚫

⚠️

0

0

main

Slide 36 - 59:25

EXPLORED

SEARCH

ADVANCED BACKEND CO...

level4

node_modules

type-is

undefsafe

undici-types

unpipe

vary

web-streams-p...

wrappy

ws

zod

.package-lock.js...

.env

index.js

package-lock.json

package.json

TIMELINE

OUTLINE

package.json

package-lock.json

index.js

localhost:3000/api/auth/r..

.env

```
39 const llm=new ChatGoogleGenerativeAI({
40   model:"gemini-2.5-pro"
41 })
42
43 app.post("/ai", async (req, res) => {
44   const { input } = req.body
45   const response = await ai.models.generateContent({
46     model: "gemini-3.5-flash",
47     contents: [
48       {
49         role: "system",
50         parts: [{ text: "you are a assistant and your name is jarvis.if you
51 don't know the answer then don't give incorrect answer" }]
52       },
53       {
54         role: "user",
55         parts: [{ text: input }]
56       }
57     ]
58   })
59   return res.status(200).json({ "ai:": response.text })
60 })
61
```

🔍

Ln 45, Col 5

Spaces: 4

UTF-8

CRLF

{ }

JavaScript

🚫

Go Live

🔔

File

Edit

Selection

View

...

←

→

Advanced Backend Course

🗨

▼

🏠

|

🔍

🖨

📄

📁

⋮

📄

1

EXPLORER

...

> SEARCH

▼ ADVANCED BACKEND CO...

▼ level4

▼ node_modules

> type-is

> undefsafe

> undici-types

> unpipe

> vary

> web-streams-p...

> wrappy

> ws

> zod

{ } .package-lock.js...

🔧 .env

📄 index.js

📄 package-lock.json

📄 package.json

> TIMELINE

> OUTLINE

🔍

main

🔄

⊗ 0

⚠ 0

📄 package.json

📄 package-lock.json

📄 index.js

🔗 GET localhost:3000/api/auth/r..

🔧 .env

📄

...

38

39

40

41

42

43

44

45

46

47

48

49

50

51

52

53

54

55

56

57

58

59

60

```
const llm=new ChatGoogleGenerativeAI({
  model:"gemini-2.5-pro"
})

app.post("/ai", async (req, res) => {
  const { input } = req.body

  return res.status(200).json({ "ai:: response.text })
})

app.get("/", (req, res) => {
  return res.json({ message: "hello from level4" })
})

app.listen(port, () => {
  console.log("server started")
})
```

🔍

Ln 45, Col 5

Spaces: 4

UTF-8

CRLF

{ }

JavaScript

🔧

🔄 Go Live

🔔

Excalidraw Whiteboard

Gemini API quickstart - generai

LangChain: Observe, Evaluate,

LLM integrations - Docs by Lar

LangSmith

ChatGoogleGenerativeAI integ

OpenAI integration - Docs by

Ask Gemini

docs.langchain.com/oss/javascript/integrations/chat/google_generative_ai

LangChain Docs

Build

Search...Ctrl K

Ask AI

GitHub

Try LangSmith

Overview

Deep Agents

LangChain

LangGraph

Integrations

Learn

Reference

Contribute

TypeScript

Overview

All providers

Popular Providers

OpenAI

Anthropic

Google

Overview

Chat

Tools

ChatGoogleGenerativeAI (Legacy)

ChatVertexAI (Legacy)

AWS

Microsoft

you are a helpful assistant that translates english to

],

["human", "I love programming."],

])

aiMsg

AIMessage {

"content": "J'adore programmer. \n",

"additional_kwargs": {

"finishReason": "STOP",

"index": 0,

"safetyRatings": [

{

"category": "HARM_CATEGORY_SEXUALLY_EXPLICIT",

"probability": "NEGLIGIBLE"

},

{

"category": "HARM_CATEGORY_HATE_SPEECH",

"probability": "NEGLIGIBLE"

},

{

"category": "HARM_CATEGORY_HARASSMENT",

"probability": "NEGLIGIBLE"

},

On this page

Overview

Integration details

Model features

Setup

Credentials

Installation

Instantiation

Invocation

Safety settings

Tool calling

Built in

Code e

Context c

Gemini pro

API reference

Slide 243 - 60:19

THUNDER CLIE...

New Request

Activity

filter activity

POST localhost:3000/...

38 mins ago

POST localhost:8000/ai

2 days ago

GET localhost:8000/s...

7 days ago

GET localhost:8000/c...

9 days ago

GET localhost:5000

11 days ago

POST localhost:3000/...

15 days ago

GET localhost:8000/a...

package.jsonpackage-lock.jsonindex.jslocalhost:3000/api/auth/r...

.env

POST

http://localhost:5000/ai

Send

Query

Headers²

Auth

Body¹

Tests

Pre Run

JSON

XML

Text

Form

Form-encode

GraphQL

Binary

1

{

2

3

}

Status: 200 OK

Size: 144 Bytes

Time: 1m 7s

Response

1

{

2

"ai:": "I am sorry, but I do not know your name. Since I don't have access to your personal information, you would have to tell me what it is!"

3

}

Resp

Go Live

Slide 49 - 60:56

Excalidraw Whiteboard

Gemini API quickstart - general

LangChain: Observe, Evaluate,

LLM integrations - Docs by Lan

LangSmith

ChatGoogleGenerativeAI integ

OpenAI integration - Docs by

Ask Gemini

docs.langchain.com/oss/javascript/integrations/chat/google_generative_ai

LangChain Docs

Build

Search...Ctrl K

Ask AI

GitHub

Try LangSmith

Overview

Deep Agents

LangChain

LangGraph

Integrations

Learn

Reference

Contribute

TypeScript

Overview

All providers

Popular Providers

OpenAI

Anthropic

Google

Overview

Chat

Tools

ChatGoogleGenerativeAI (Legacy)

ChatVertexAI (Legacy)

AWS

Microsoft

Credentials

Get an API key here: <https://ai.google.dev/tutorials/setup>

Then set the `GOOGLE_API_KEY` environment variable:

```
export GOOGLE_API_KEY="your-api-key"
```

If you want to get automated tracing of your model calls you can also set your [LangSmith](#) API key by uncommenting below:

```
# export LANGSMITH_TRACING="true"
# export LANGSMITH_API_KEY="your-api-key"
```

Installation

The LangChain `ChatGoogleGenerativeAI` integration lives in the `@langchain/google-genai` package:

```
npm yarn pnpm
```

On this page

Overview

Integration details

Model features

Setup

Credentials

Installation

Instantiation

Invocation

Safety settings

Tool calling

Built in

Code e

Context c

Gemini pro

API reference

Slide 262 - 63:23

FileEditSelectionView

EXPLORER

SEARCH

ADVANCED BACKEND CO...

level1

level2

level3

level4

node_modules

.env

index.js

package-lock.json

package.json

TIMELINE

OUTLINE

package.jsonpackage-lock.jsonindex.jslocalhost:3000/api/auth/r... .env

1GOOGLE_API_KEY="AQ.Ab8RN6K83SorGP-PFaSePR6_3paD5Nd0YqG1FfnmBxjUY16Tew"

PROBLEMSOUTPUTDEBUG CONSOLETERMINALPORTS

node - level4

PS D:\desktop\Advanced Backend Course\level4> npm run dev

> nodemon index.js

[nodemon] 3.1.14

[nodemon] to restart at any time, enter `rs`

[nodemon] watching path(s): *.*

[nodemon] watching extensions: js,mjs,cjs,json

[nodemon] starting `node index.js`

◇ injected env (1) from .env // tip: % multiple files { path: ['.env.local',

server started

[nodemon] restarting due to changes...

[nodemon] starting `node index.js`

◇ injected env (1) from .env // tip: % custom filepath { path: '/custom/path/.e

server started

Ln 1, Col 71Spaces: 4UTF-8CRLF{ }DotenvGo Live



Excalidraw Whiteboard

Gemini API quickstart - generai

LangChain: Observe, Evaluate,

LLM integrations - Docs by Lan

LangSmith

ChatGoogleGenerativeAI integ

OpenAI integration - Docs by

Ask Gemini

docs.langchain.com/oss/javascript/integrations/chat/google_generative_ai

LangChain Docs

Build

Search...Ctrl K

Ask AI

GitHub

Try LangSmith

Overview

Deep Agents

LangChain

LangGraph

Integrations

Learn

Reference

Contribute

TypeScript

Overview

All providers

Popular Providers

OpenAI

Anthropic

Google

Overview

Chat

Tools

ChatGoogleGenerativeAI (Legacy)

ChatVertexAI (Legacy)

AWS

Microsoft

Credentials

Get an API key here: <https://ai.google.dev/tutorials/setup>

Then set the `GOOGLE_API_KEY` environment variable:

```
export GOOGLE_API_KEY="your-api-key"
```

If you want to get automated tracing of your model calls you can also set your [LangSmith](#) API key by uncommenting below:

```
# export LANGSMITH_TRACING="true"
# export LANGSMITH_API_KEY="your-api-key"
```

Installation

The LangChain `ChatGoogleGenerativeAI` integration lives in the `@langchain/google-genai` package:

```
npm yarn pnpm
```

On this page

Overview

Integration details

Model features

Setup

Credentials

Installation

Instantiation

Invocation

Safety settings

Tool calling

Built in

Code e

Context c

Gemini pro

API reference

Slide 265 - 63:55

Excalidraw Whiteboard

Gemini API quickstart - g

LangChain: Observe, Eval

LLM Integrations - Docs

LangSmith

ChatGoogleGenerativeAI

OpenAI integration - Doc

ChatGroq integration - D

Ask Gemini

docs.langchain.com/oss/javascript/integrations/chat/groq

LangChain Docs

Build

Search...Ctrl K

Ask AI

GitHub

Try LangSmith

Overview

Deep Agents

LangChain

LangGraph

Integrations

Learn

Reference

Contribute

TypeScript

Overview

All providers

Popular Providers

OpenAI

Anthropic

Google

AWS

Microsoft

General Integrations

Chat models

Tools and Toolkits

LLMs

Middleware

Sandboxes

Credentials

To use the Groq API, create an API key in the [Groq console](#). Then, you can set the API key as an environment variable in your terminal:

export GROQ_API_KEY="your-api-key"

If you want to get automated tracing of your model calls you can also set your [LangSmith](#) API key by uncommenting below:

export LANGSMITH_TRACING="true"
export LANGSMITH_API_KEY="your-api-key"

Installation

The LangChain ChatGroq integration lives in the `@langchain/groq` package:

npm yarn pnpm
npm install @langchain/groq @langchain/core

On this page

Overview

Integration details

Model features

Setup

Credentials

Installation

Instantiation

Invocation

Invoke with JSON

API reference

Slide 271 - 650

File

Edit

Selection

View

...

←

→

Advanced Backend Course

🗨

▼

🏠

🔍

📄

📁

📦

⚡

📦

👤

⚙️

🔗

EXPLORED

...

> SEARCH

> ADVA...

> level1

> level2

> level3

> level4

> node_modules

> .env

> index.js

> package-lock.json

> package.json

> TIMELINE

> OUTLINE

main

🔄

⊗ 0

⚠ 0

package.json

package-lock.json

index.js

localhost:3000/api/auth/r..

.env

32

//

return res.status(200).json({ "ai:: response.text })

33

// })

34

35

//with langchain

36

37

38

39

const llm=new ChatGoogleGenerativeAI({

40

model:"gemini-3.5-flash"

41

})

42

43

app.post("/ai", async (req, res) => {

44

const { input } = req.body

45

46

const response=await llm.invoke(input)

47

48

return res.status(200).json({ "ai:: response.content })

49

})

50

51

52

53

54

app.get("/", (req, res) => {

55

return res.json({ message: "hello from level4" })

Ln 40, Col 12 (16 selected)

Spaces: 4

UTF-8

CRLF

{ }

JavaScript

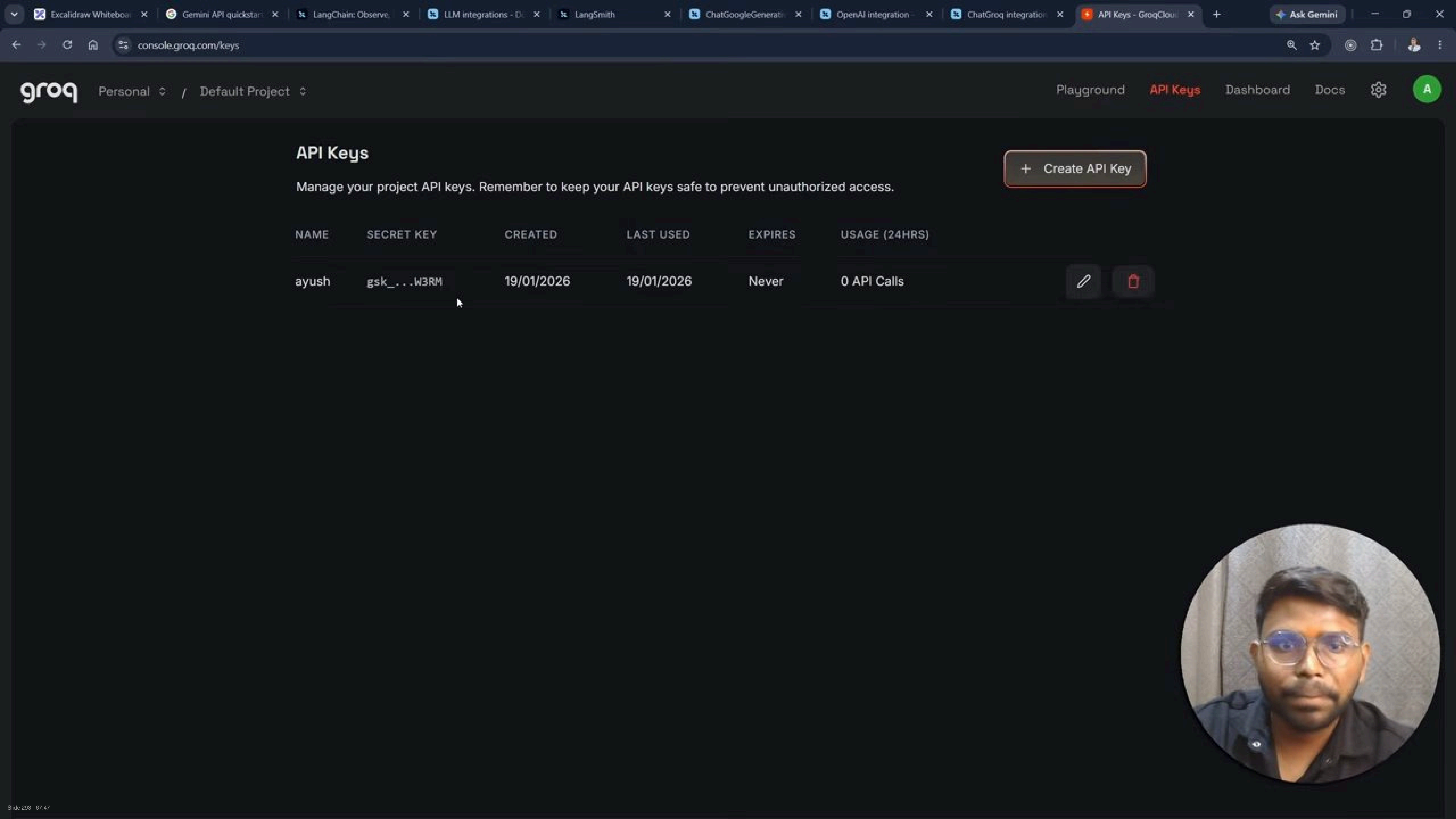
Go Live

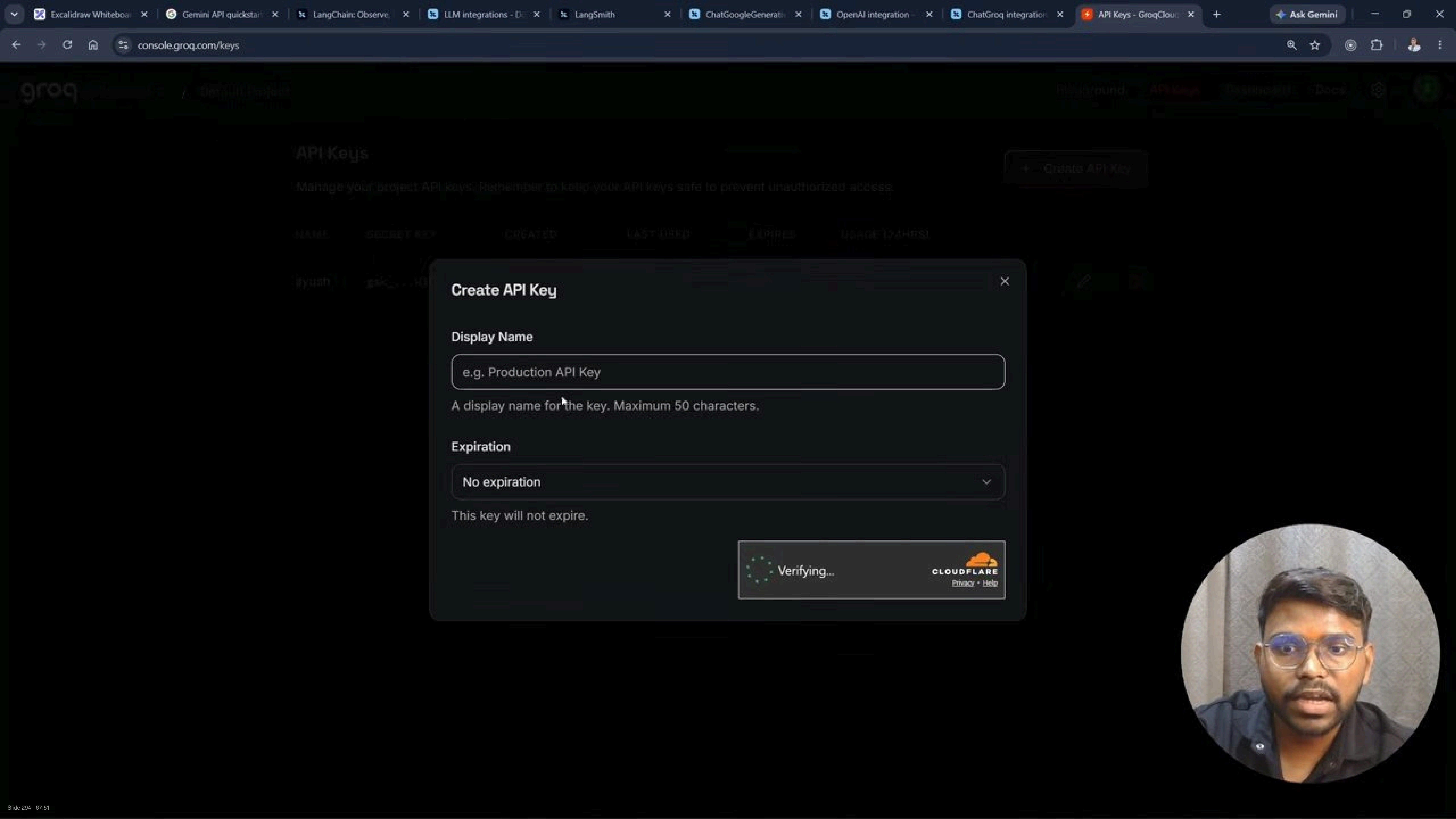
groq

Build ⚡ Fast

Loading...







groq

Products

Integrations

Support

Account

API Keys

API Keys

Manage your project API keys. Remember to keep your API keys safe to prevent unauthorized access.

Create API Key

Name	Environment	Created	Last Used	Expires	Usage (Last 30 Days)		
dev	gsk_...	10/01/2025	10/01/2026	Never	0 API Calls		
prod	gsk_...						

Create API Key

Your new API key has been created. Copy it now, as we will not display it again.

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

Copy

Done

Very hot weather

Slide 5 - 04:08

Search

Windows Taskbar

System Tray

14:08 13-06-2026

FileEditSelectionView...<→Advanced Backend Course

package.jsonpackage-lock.jsonindex.jslocalhost:3000/api/auth/r... .env

EXPLORERSERCHADVANCED BACKEND CO...level1level2level3level4node_modules.envindex.jspackage-lock.jsonpackage.json

TIMELINEOUTLINEmain00

POSThttp://localhost:5000/aiSend

QueryHeaders 2AuthBody 1TestsPre Run

JSONXMLTextFormForm-encodeGraphQLBinary

```
1 {
2   "input": "what is javascript?"
3 }
```

Status: 200 OKSize: 3.15 KBTime: 9.72 sResponse

```
1 {
2   "ai:": "**JavaScript (often abbreviated as JS)** is a popular, lightweight scripting language. It is best known as the language that powers the dynamic elements of websites. \n\nAlong with HTML and CSS, it is one of the three core technologies of the World Wide Web.\n\n-- House\" Analogy\nTo understand what JavaScript does, imagine building a house:\n*   **HTML (HyperText Markup Language)**--the walls, floors, and doors.\n*   **CSS (Cascading Style Sheet)**--the colors, fonts, and layout.\n*   **JavaScript**--the interactive elements that make the house livable.\n\nIn summary, JavaScript is a versatile language that allows developers to create dynamic and interactive web applications. It is essential for modern web development and is used by millions of websites around the world.\n\nI hope this helps you understand what JavaScript is! Feel free to ask if you have any more questions."
}
```

Go Live

FileEditSelectionView

EXPLORER

SEARCH

ADVANCED BACKEND CO...

level1

level2

level3

level4

node_modules

.env

index.js

package-lock.json

package.json

TIMELINE

OUTLINE

main

package.jsonpackage-lock.jsonindex.jslocalhost:3000/api/auth/r... .env

POSThttp://localhost:5000/aiSend

QueryHeaders 2AuthBody 1TestsPre Run

JSONXMLTextFormForm-encodeGraphQLBinary

```
1 {
2   "input": "hello , who are you?"
3 }
```

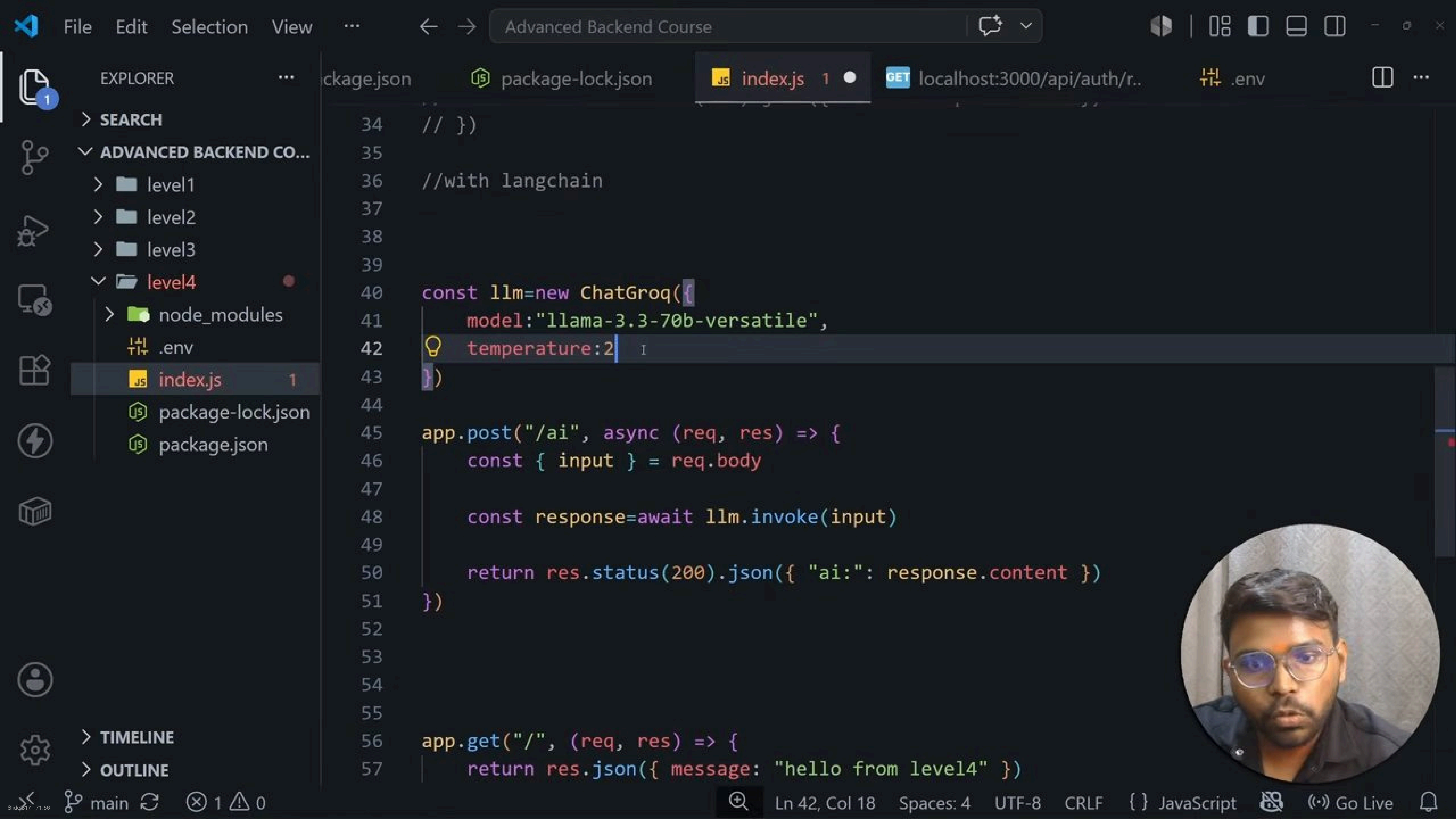
Status: 200 OKSize: 353 BytesTime: 512 msResponse

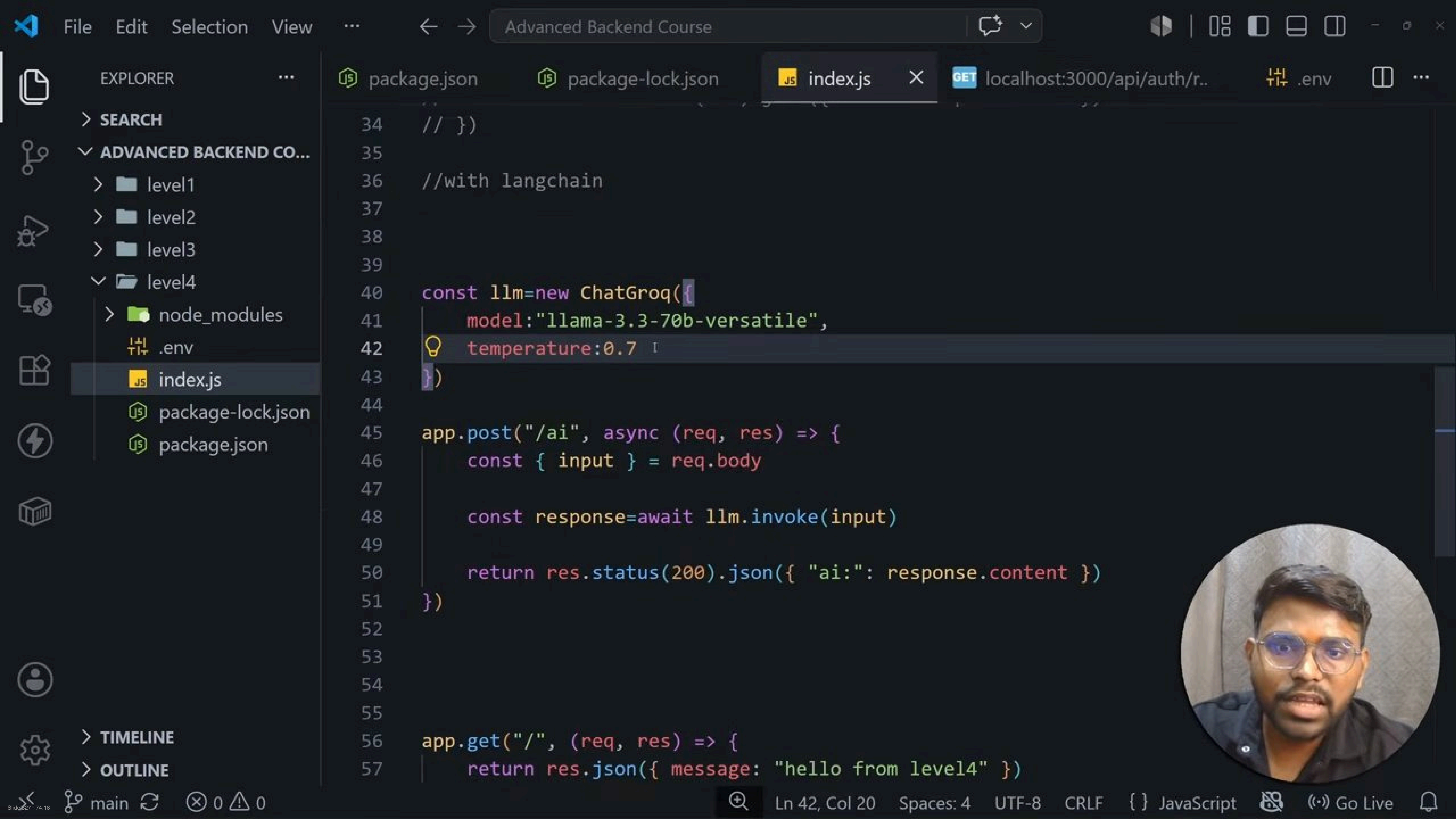
```
1 {
2   "ai": "Hello! I'm an artificial intelligence language model, which means I'm a computer program designed to understand and generate human-like text. I'm here to help you with your questions, provide information, and have conversations with you. I don't have a personal name, but you can think of me as a friendly AI assistant for today?"
3 }
```

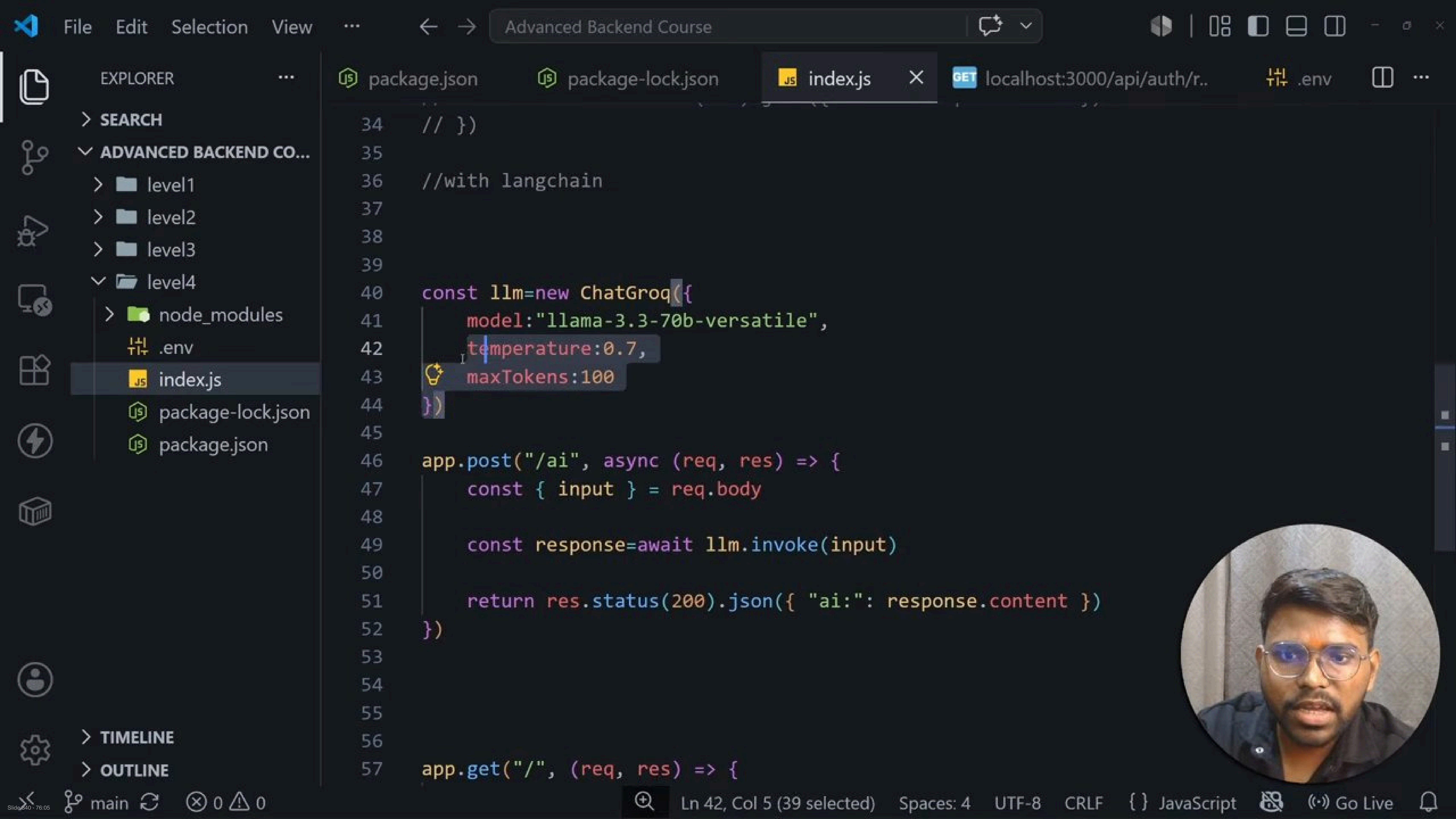
Go Live

13:05









LangChain Docs

Build

Search...Ctrl K

Ask AI

GitHub

Try LangSmith

Overview

Deep Agents

LangChain

LangGraph

Integrations

Learn

Reference

Contribute

TypeScript

Overview

All providers

Popular Providers

OpenAI

Anthropic

Google

AWS

Microsoft

General Integrations

Chat models

Tools and Toolkits

LLMs

Middleware

Sandboxes

completions:

```
import { ChatGroq } from "@langchain/groq"

const llm = new ChatGroq({
  model: "llama-3.3-70b-versatile",
  temperature: 0,
  maxTokens: undefined,
  maxRetries: 2,
  // other params...
})
```

Invocation

```
const aiMsg = await llm.invoke([
  {
    role: "system",
    content: "You are a helpful assistant that translates Eng
  },
  { role: "user", content: "I love programming." },
])

aiMsg
```

On this page

Overview

Integration details

Model features

Setup

Credentials

Installation

Instantiation

Invocation

Invoke with JSON

API reference



Slide 341 - 7600

LangChain Docs

Build

Search...Ctrl K

Ask AI

GitHub

Try LangSmith

Overview

Deep Agents

LangChain

LangGraph

Integrations

Learn

Reference

Contribute

TypeScript

Overview

All providers

Popular Providers

OpenAI

Anthropic

Google

AWS

Microsoft

General Integrations

Chat models

Tools and Toolkits

LLMs

Middleware

Sandboxes

```
model: "llama-3.3-70b-versatile",
temperature: 0,
maxTokens: undefined,
maxRetries: 2,
// other params...
})
```

Invocation

```
const aiMsg = await llm.invoke([
  {
    role: "system",
    content: "You are a helpful assistant that translates Eng
  },
  { role: "user", content: "I love programming." },
])
aiMsg
```

```
AIMessage {
  "content": "I enjoy programming. (The French translation is:
  "additional_kwargs": {
```

On this page

Overview

Integration details

Model features

Setup

Credentials

Installation

Instantiation

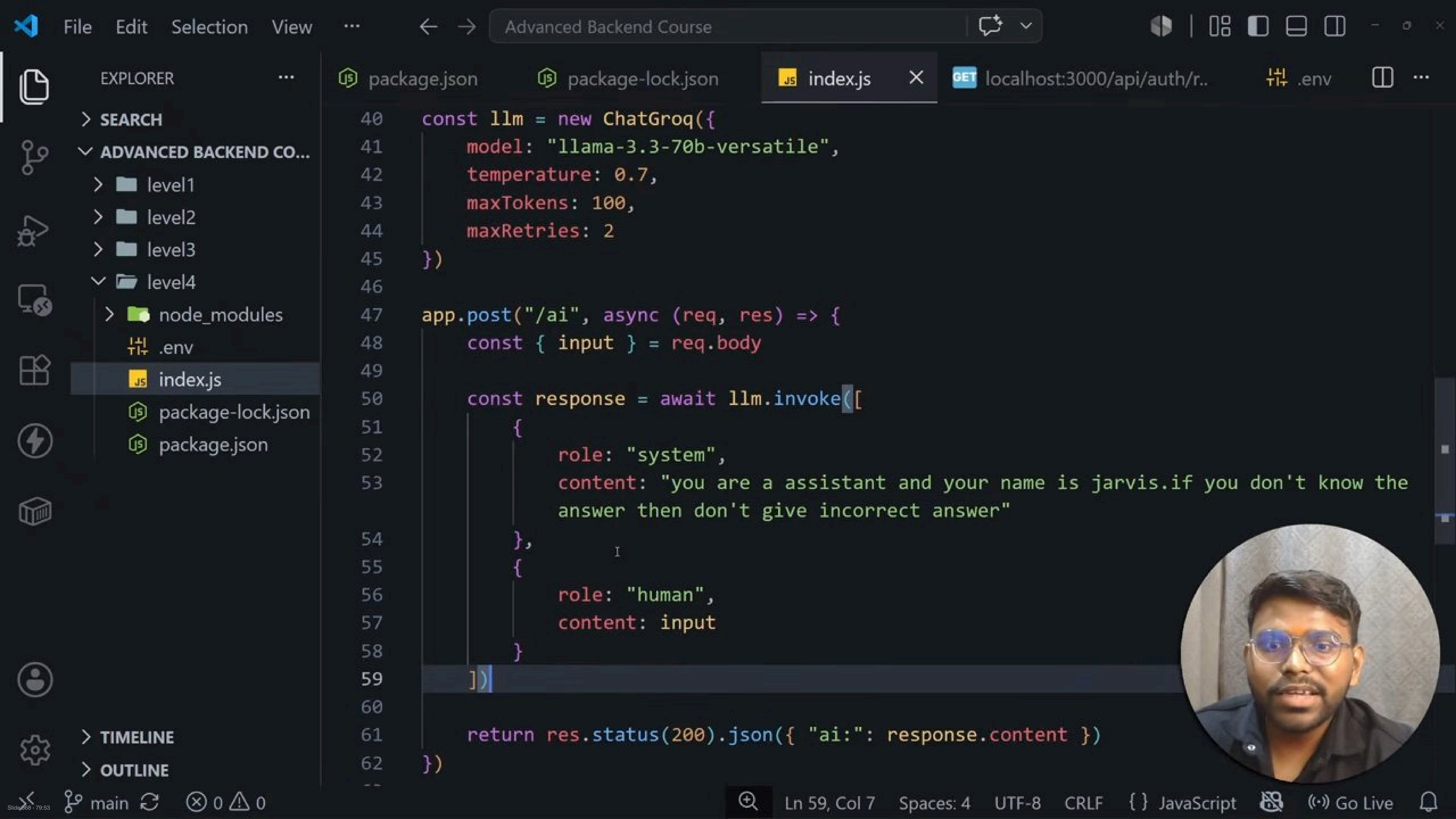
Invocation

Invoke with JSON

API reference

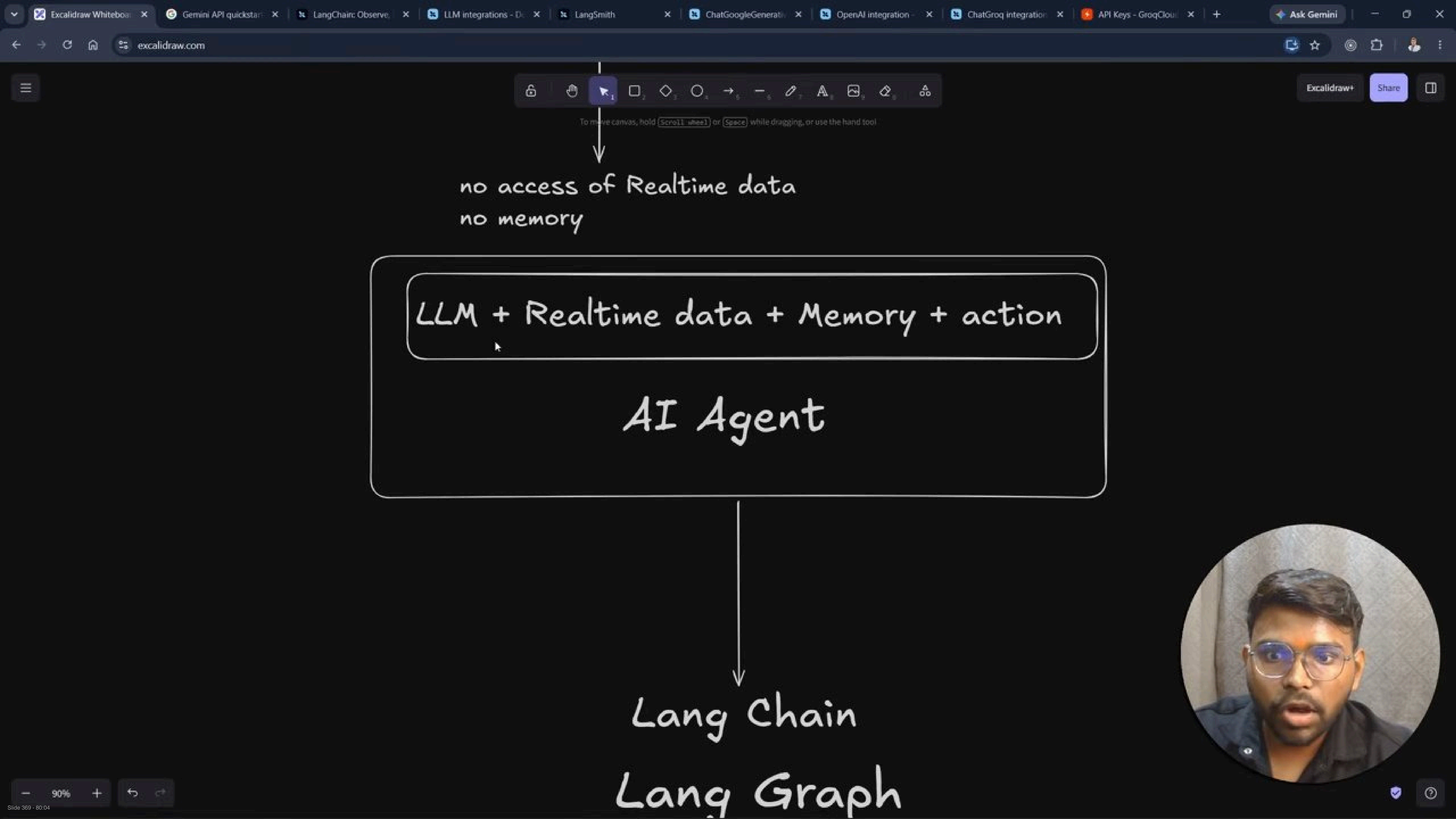
14:19

13-06-2026



```
40 const llm = new ChatGroq({
41   model: "llama-3.3-70b-versatile",
42   temperature: 0.7,
43   maxTokens: 100,
44   maxRetries: 2
45 })
46
47 app.post("/ai", async (req, res) => {
48   const { input } = req.body
49
50   const response = await llm.invoke([
51     {
52       role: "system",
53       content: "you are a assistant and your name is jarvis.if you don't know the
54         answer then don't give incorrect answer"
55     },
56     {
57       role: "human",
58       content: input
59     }
60   ])
61   return res.status(200).json({ "ai:": response.content })
62 })
```





Excalidraw Whiteboard

Gemini API quickstart

LangChain: Observe

LLM Integrations - D

LangSmith

ChatGoogleGenerati

OpenAI integration

ChatGroq integration

API Keys - GroqCloud

Ask Gemini

docs.langchain.com/oss/javascript/integrations/llms/openai

LangChain Docs

Build

Search...Ctrl K

Ask AI

GitHub

Try LangSmith

Overview

Deep Agents

LangChain

LangGraph

Integrations

Learn

Reference

Contribute

TypeScript

Overview

All providers

Popular Providers

OpenAI

Anthropic

Google

AWS

Microsoft

General Integrations

Chat models

Tools and Toolkits

LLMs

Middleware

Sandboxes

Instantiation

Now you can instantiate the model and generate text completions:

```
import { OpenAI } from "@langchain/openai"

const llm = new OpenAI({
  model: "gpt-3.5-turbo-instruct",
  temperature: 0,
  maxTokens: undefined,
  timeout: undefined,
  maxRetries: 2,
  apiKey: process.env.OPENAI_API_KEY,
  // other params...
})
```

Invocation

```
const inputText = "OpenAI is an AI company that "

const completion = await llm.invoke(inputText)
completion
```

On this page

Overview

Integration details

Setup

Credentials

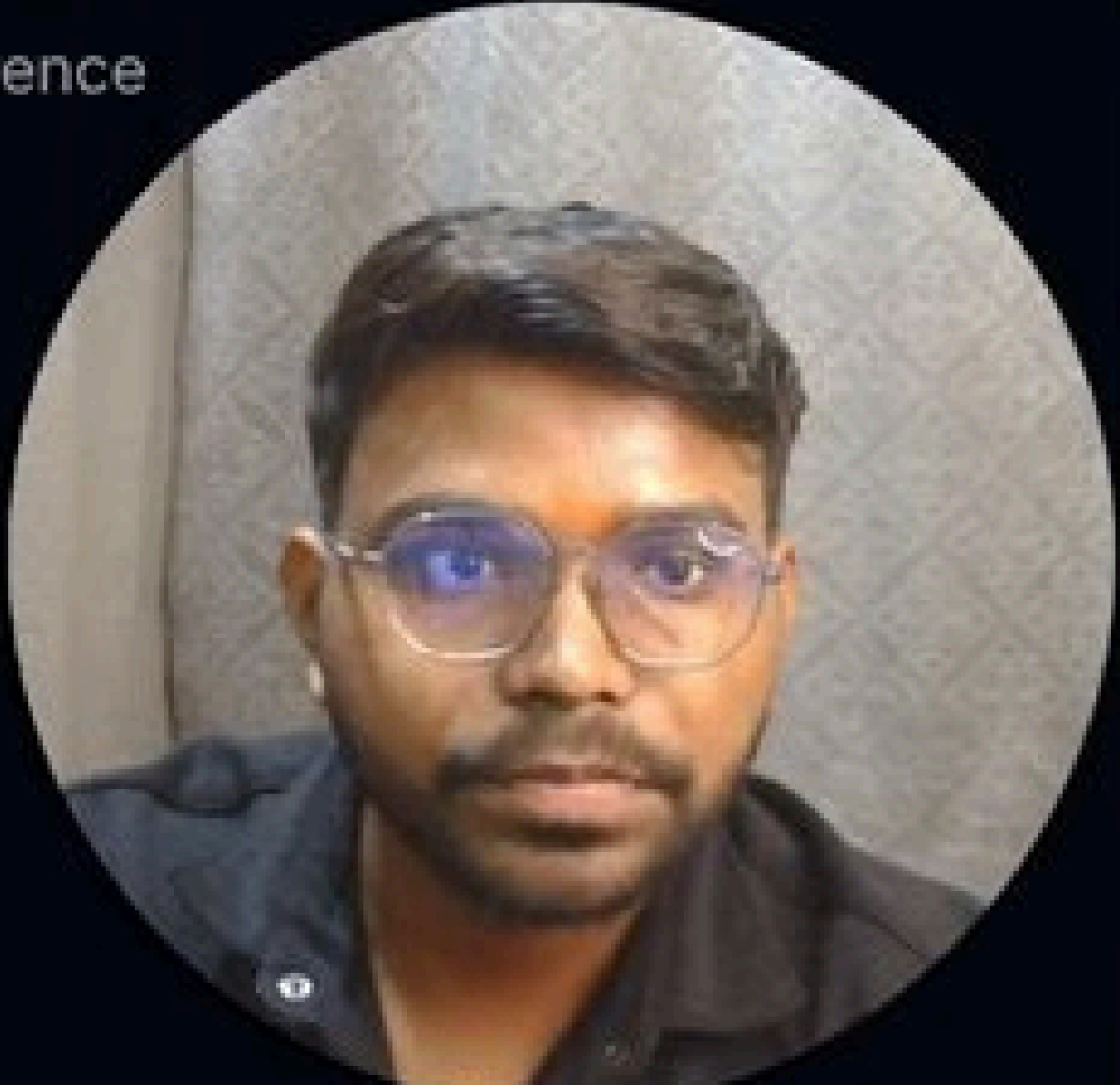
Installation

Instantiation

Invocation

Custom URLs

API reference



Excalidraw Whiteboard

LangGraph overview - Docs by

Ask Gemini

excalidraw.com

🔒 🗑️ 🖱️ 📐 🔍 ⬅️ ➡️ ✎ 🗑️ 🔗 🔄

GPT (open ai) or gemini while dragging, or use the hand tool

Excalidraw+ Share

user

write prompt

LLM

user

write prompt

LLM

no access of Realtime data
no memory

LLM + Realtime data + Memory + action

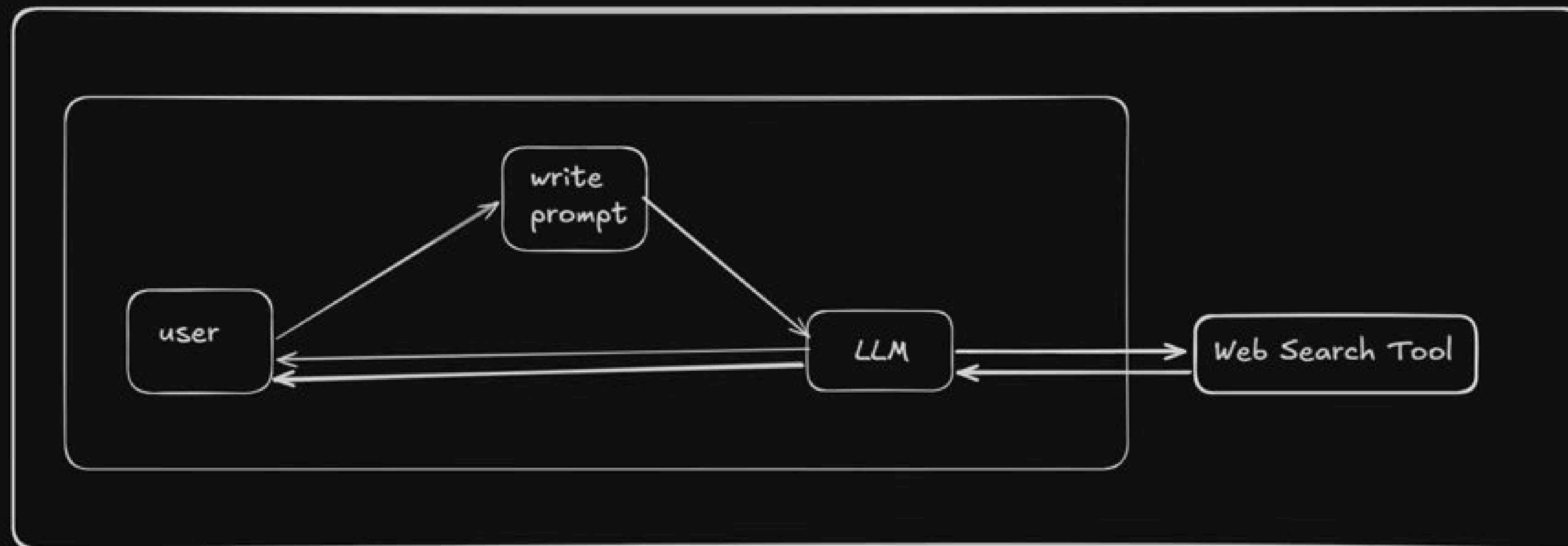
AI Agent



80%

Slide 379 - 81:51

🔒 ⌚



Langgraph → AI Workflows represented by graph



Excalidraw Whiteboard

LangGraph overview - Docs by

Ask Gemini

excalidraw.com

Share

Start

Agent

End

Web search tool

Edge

Node

Conditional Edges

Tool Node

80%

Slide 394 - 95:21



Excalidraw Whiteboard

LangGraph overview - Docs by

Ask Gemini

docs.langchain.com/oss/javascript/langgraph/overview

Search...Ctrl K

Ask AI

GitHub

Try LangSmith >

Overview

Deep Agents

LangChain

LangGraph

Integrations

Learn

Reference

Contribute

TypeScript

Overview

Get started

Install

Quickstart

Local server

Changelog

Thinking in LangGraph

Workflows + agents

Capabilities

Persistence

Fault tolerance

Checkpointers

Stores

Event streaming

Show how LangChain products fit together

Install

npm pnpm yarn bun

npm install @langchain/langgraph @langchain/core

Then, create a simple hello world example:

```
import { StateSchema, MessagesValue, type GraphNode, State }

const State = new StateSchema({
  messages: MessagesValue,
});

const mockLlm: GraphNode<typeof State> = (state) => {
  return { messages: [{ role: "ai", content: "hello world" }] };
};

const graph = new StateGraph(State)
  .addNode("mock_llm", mockLlm)
  .addEdge(START, "mock_llm")
```

On this page

Install

Core benefits

LangGraph ecosystem

Acknowledgements



Slide 398 - 9/25/24

A screenshot of the Visual Studio Code Explorer sidebar. The sidebar is dark-themed. At the top, there's a 'FILE EXPLORER' header with a document icon and a blue circle with the number '1'. Below the header, there's a search bar. The main area shows a file tree for a project named 'ADVANCED BACKEND CO...'. The tree structure is as follows: a root folder 'ADVANCED BACKEND CO...' containing three subfolders: 'level1', 'level2', and 'level3'. Under 'level3', there is a subfolder 'level4'. Inside 'level4', there are four files: 'node_modules' (a green folder icon), '.env' (a yellow file icon), 'index.js' (a yellow file icon with 'JS' in the corner, which is currently selected and highlighted with a blue background), 'package-lock.json' (a green file icon with 'JSON' in the corner), and 'package.json' (a green file icon with 'JSON' in the corner). On the left side of the sidebar, there are several icons for other views: a source control icon, a run and debug icon, a testing icon, a settings icon, a timeline icon, and an outline icon.

```

41  const llm = new ChatGroq({
44      maxTokens: 100,
45      maxRetries: 2
46  })
47
48
49
50  app.post("/ai", async (req, res) => {
51      const { input } = req.body
52
53      const response = await llm.invoke([
54          {
55              role: "system",
56              content: "you are a assistant and your name is jarvis.if you don't know the
57                  answer then don't give incorrect answer"
58          },
59          {
60              role: "human",
61              content: input
62          }
63      ])
64
65      return res.status(200).json({ "ai:": response.content })
66  })

```



Excalidraw Whiteboard

LangGraph overview - Docs by

Ask Gemini

docs.langchain.com/oss/javascript/langgraph/overview

LangChain Docs

Build

Search...Ctrl K

Ask AI

GitHub

Try LangSmith

Overview

Deep Agents

LangChain

LangGraph

Integrations

Learn

Reference

Contribute

TypeScript

Overview

Get started

Install

Quickstart

Local server

Changelog

Thinking in LangGraph

Workflows + agents

Capabilities

Persistence

Fault tolerance

Checkpointers

Stores

Event streaming

```
import { StateSchema, MessagesValue, type GraphNode, State }

const State = new StateSchema({
  messages: MessagesValue,
});

const mockLlm: GraphNode<typeof State> = (state) => {
  return { messages: [{ role: "ai", content: "hello world" }] };
};

const graph = new StateGraph(State)
  .addNode("mock_llm", mockLlm)
  .addEdge(START, "mock_llm")
  .addEdge("mock_llm", END)
  .compile();

await graph.invoke({ messages: [{ role: "user", content: "hi!" }]});
```

On this page

Install

Core benefits

LangGraph ecosystem

Acknowledgements

💡

Use [LangSmith](#) to trace requests, debug agent behavior, and evaluate outputs. Set `LANGSMITH_TRACING=true` and your API key to get started. Follow the [tracing quickstart](#) to get set up. We recommend you also set up [LangSmith Engine](#) which monitors your traces, detects issues, and proposes fixes.

Excalidraw Whiteboard

LangGraph overview - Docs by

Ask Gemini

excalidraw.com

Share

Stroke

Background

Stroke width

Stroke style

Sloppiness

Edges

Opacity

Layers

Actions

State={

Edge

}

Node

Conditional Edges

Tool Node

Start

Agent

End

Web search tool

State={

Edge

}

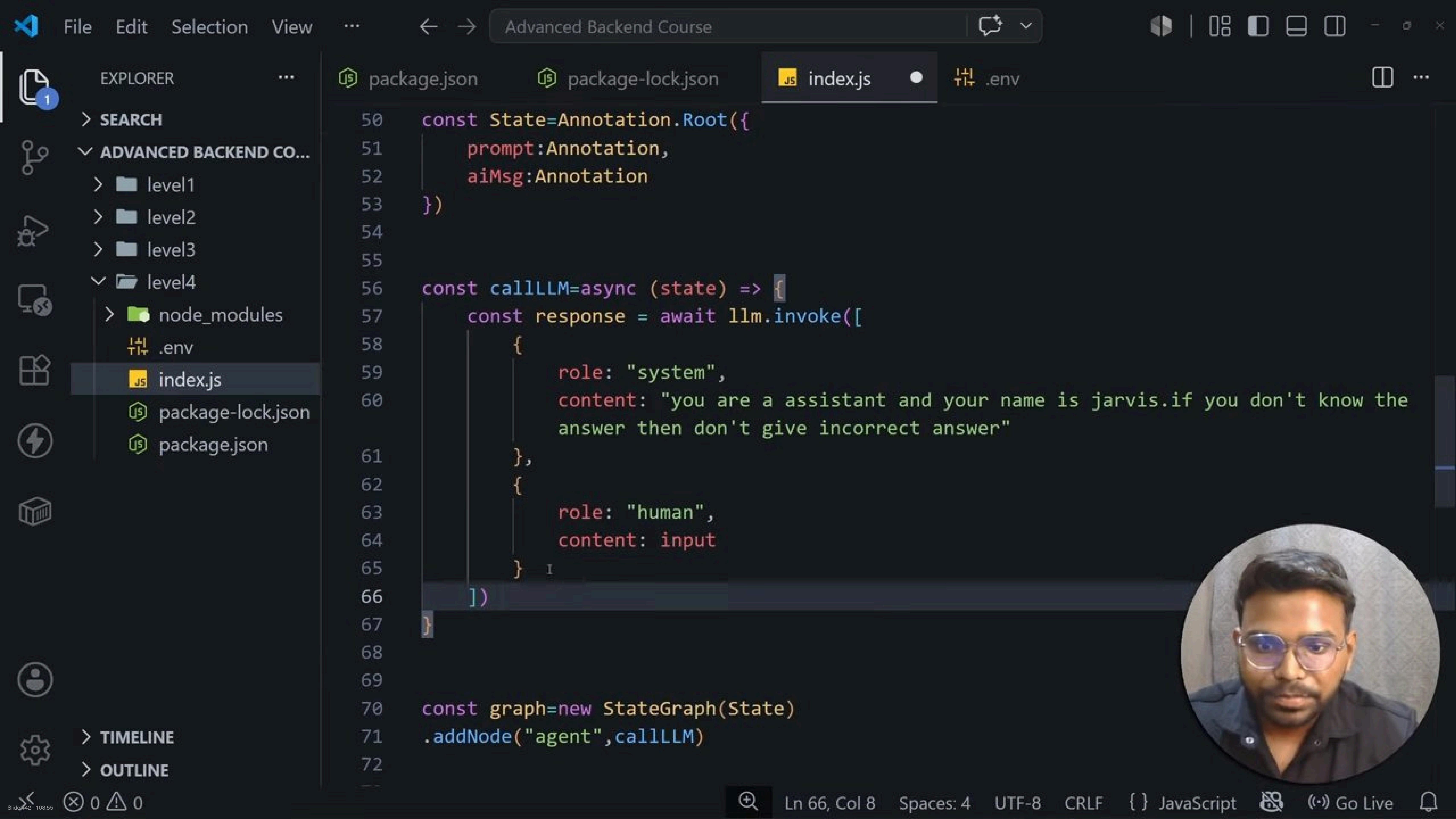
Node

Conditional Edges

Tool Node

80%

Slide 408 - 101:26





1

EXPLORER

...

> SEARCH

> ADVANCED BACKEND CO...

> level1

> level2

> level3

> level4

> node_modules

.env

index.js

package-lock.json

package.json

> TIMELINE

> OUTLINE

package.json

package-lock.json

index.js

.env

56

57

58

59

60

61

62

63

64

65

66

67

68

69

70

71

72

73

74

75

76

77

78

79

const callLLM=async (state) => {

console.log("state:",state)

const response = await llm.invoke([

{

role: "system",

content: "you are a assistant and your name is jarvis.if you don't know the

answer then don't give incorrect answer"

},

{

role: "human",

content: state.prompt

}

])

const graph=new StateGraph(State)

.addNode("agent",callLLM)

Ln 70, Col 5

Spaces: 4

UTF-8

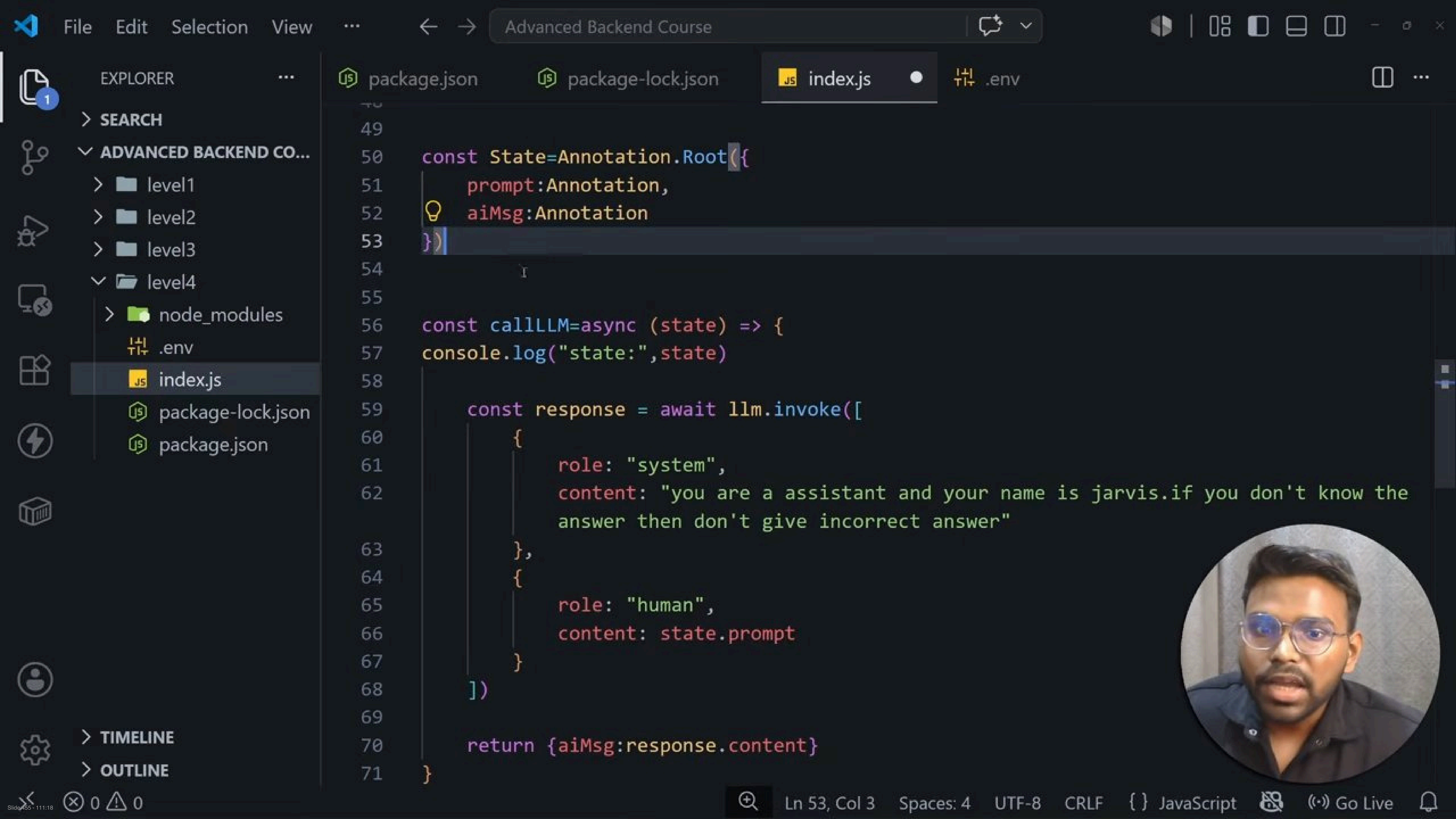
CRLF

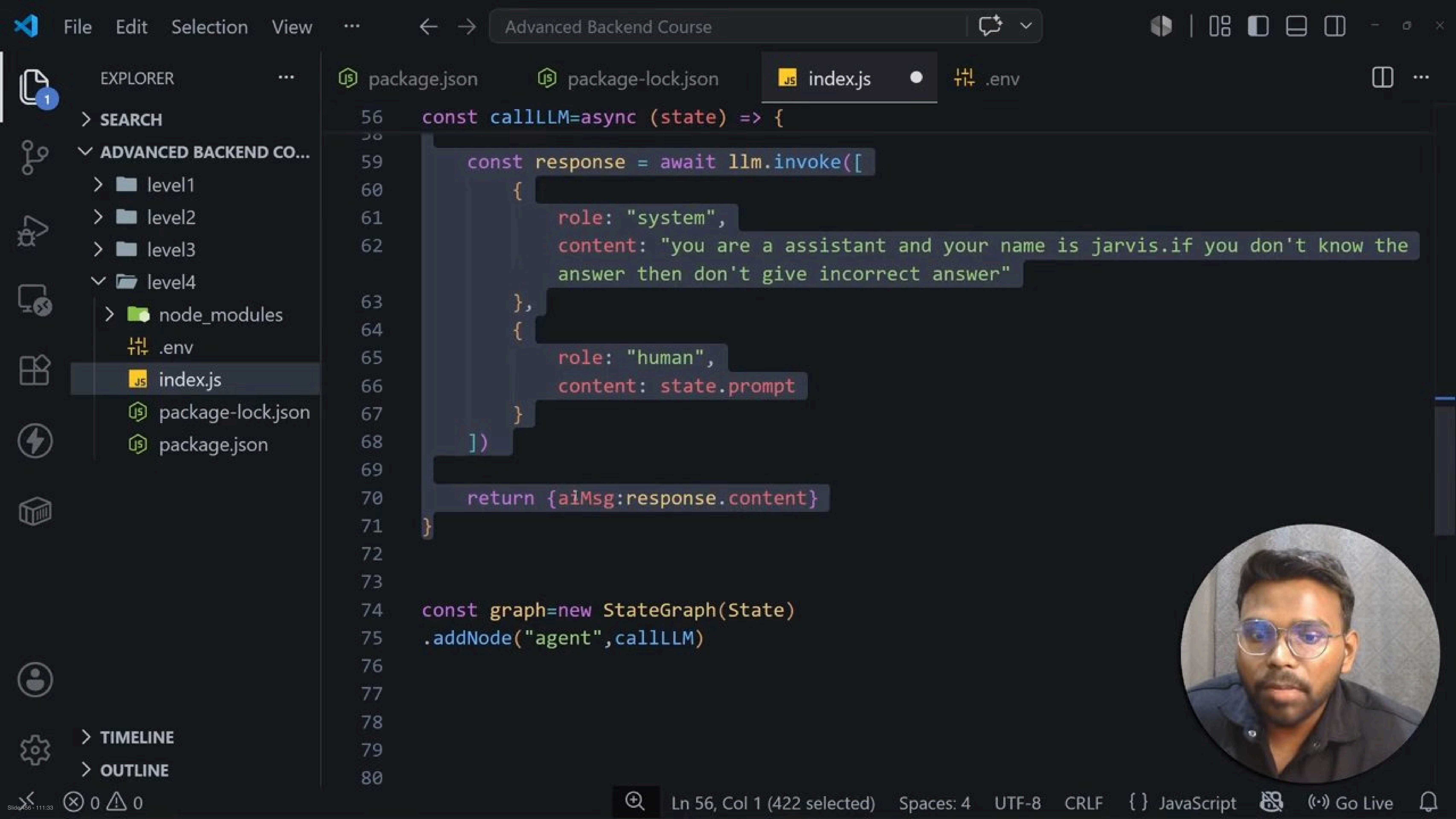
{ }

JavaScript

Go Live







```
56 const callLLM=async (state) => {
57
58
59   const response = await llm.invoke([
60     {
61       role: "system",
62       content: "you are a assistant and your name is jarvis.if you don't know the
        answer then don't give incorrect answer"
63     },
64     {
65       role: "human",
66       content: state.prompt
67     }
68   ])
69
70   return {aiMsg:response.content}
71 }
72
73
74 const graph=new StateGraph(State)
75 .addNode("agent",callLLM)
76
77
78
79
80
```



